JAVA 82R BATCH

Complete Java Notes

All 68 Topics | Sequential Order

Table of Contents

01. Code, Developers, Programming Languages
02. Introduction to Java
03. Java Basics
04. Access Modifiers ,keyword, Identifiers
05. Data Types and Variables
06. Operators in Java
07. Static Methods in Java
08. Non-Static Methods in Java
09. Method Access Method
10. Business Logic Class (BLC) and Executable Logic Class (ELC) in Java
11. Control Flow Statements in Java
12. if and if-else
13. if-else if-else
14. Scanner Class in Java
15. switch
16. for
17. while
18. do-while
19. jump statements
20. java packages, access modifiers , importing
21. Java Constructors
22. Encapsulation
23. Inheritance
24. Constructor Chaining
25. this() vs super()
26. Has-A Relationship in Java
27. Wrapper Classes in Java
28. Polymorphism in Java
29. final Keyword
30. Java Abstract Class
31. Abstraction in Java
32. Interfaces in Java
33. Java Arrays
34. Java Inner Classes
35. Functional Interface in Java
36. Lambda Expressions in Java
37. Enum in Java
38. Switch 14+ in Java
39. Object Class in Java
40. Exception Handling in Java

41. String in Java
42. StringBuilder and StringBuffer in Java
43. Multithreading in Java
44. Introduction to Collection API
45. Generics in Java
46. ArrayList in Java
47. LinkedList in Java
48. Vector in Java
49. Cursor in Java
50. Stack in Java
51. Queue in Java
52. PriorityQueue in Java
53. Deque and ArrayDeque in Java
54. Set in Java
55. HashSet in Java
56. LinkedHashSet in Java
57. TreeSet in Java
58. Map in Java
59. Hashtable in Java
60. HashMap in Java
61. LinkedHashMap in Java
62. TreeMap in Java
63. Comparable vs Comparator in Java
64. Collectors
65. Streams in Java
66. Records in Java
67. Optional Class in Java
68. File Handling in Java

Notes on Code, Developers, Programming Languages, and Applications

1. What is Code?

Code is a set of instructions written in a programming language that tells a computer how to perform specific tasks. It acts as a communication bridge between humans and computers, allowing software to function as intended.

2. Who is a Developer?

A developer (also known as a programmer or software engineer) is a person who writes, tests, and maintains code to build software applications. Developers specialize in different areas, such as:

- **Frontend Developer** – Works on the user interface of web and mobile apps.
- **Backend Developer** – Handles server-side logic, databases, and APIs.
- **Full-Stack Developer** – Manages both frontend and backend development.
- **Mobile Developer** – Builds applications for Android and iOS devices.
- **Game Developer** – Focuses on creating video games.

3. What is Coding?

Coding is the act of writing code using a programming language to develop software applications. It involves using syntax, logic, and algorithms to create instructions that a computer can execute. Coding is a crucial part of software development but does not always include other stages like design or debugging.

4. What is a Programming Language?

A programming language is a formal language used to write code that a computer can interpret and execute. Programming languages have their own syntax and rules.

Types of Programming Languages:

- **Low-Level Languages:** Machine code, Assembly language
- **High-Level Languages:** Python, Java, C++, JavaScript, etc.

Programming Paradigms:

- **Procedural Programming:** Uses functions and procedures (e.g., C, Pascal).
- **Object-Oriented Programming (OOP):** Uses objects and classes (e.g., Java, Python, C++).
- **Functional Programming:** Uses mathematical functions (e.g., Haskell, Lisp).
- **Scripting Languages:** Used for automation and web development (e.g., JavaScript, Python, PHP).

5. What is an Application?

An application (app) is a type of software designed to perform specific tasks for users, such as communication, entertainment, or productivity. Applications can run on different platforms like web browsers, mobile devices, and desktops.

6. Different Types of Applications

- **Web Applications:** Run on web browsers (e.g., Gmail, Facebook, YouTube).
- **Mobile Applications:** Designed for smartphones and tablets (e.g., WhatsApp, Instagram, Paytm).
- **Desktop Applications:** Installed on computers (e.g., Microsoft Word, Adobe Photoshop).
- **Enterprise Applications:** Used for business operations (e.g., ERP software, CRM tools).
- **Cloud Applications:** Hosted on the cloud and accessible via the internet (e.g., Google Drive, Dropbox).
- **Embedded Applications:** Run on embedded systems like ATMs, smart appliances, and IoT devices.

7. Statically Typed vs Dynamically Typed Languages

Statically Typed Languages:

- Variable types are declared explicitly before use.
- Type checking is done at compile time.
- Examples: Java, C, C++, Swift.
- **Pros:** Fewer runtime errors, better performance.
- **Cons:** Requires more code for type definitions.

Dynamically Typed Languages:

- Variable types are determined at runtime.
- No need to declare variable types explicitly.
- Examples: Python, JavaScript, Ruby, PHP.
- **Pros:** More flexible and easier to write.
- **Cons:** Higher risk of runtime errors.

8. Standalone Applications vs Web Applications

Standalone Applications:

- Installed and run directly on a device (e.g., desktop or mobile).
- Do not require an internet connection to function.
- Examples: Microsoft Word, Adobe Photoshop, VLC Media Player.
- **Pros:** Faster performance, works offline.
- **Cons:** Requires installation and manual updates.

Web Applications:

- Hosted on a server and accessed via a web browser.
- Requires an internet connection to function.
- Examples: Gmail, Google Docs, Facebook, YouTube.
- **Pros:** Accessible from any device, no installation needed.
- **Cons:** Slower performance, depends on internet availability.

CHAPTER 2

Introduction to Java

Introduction to Java

1. History of Java

Java was developed by **James Gosling** and his team at **Sun Microsystems** in 1991. Initially, it was called **Oak**, but it was later renamed to **Java** in 1995. The language was designed to be **platform-independent** and was initially used for **embedded systems**. Sun Microsystems publicly released Java in **1995**.

In **2010**, **Oracle Corporation** acquired Sun Microsystems, making **Java an Oracle product**. Over the years, Java has evolved through multiple versions, introducing new features and improvements with each update.

2. Introduction to Java

Java is a **high-level, object-oriented, and platform-independent** programming language. It follows the **WO RA (Write Once, Run Anywhere)** principle, utilizing the **Java Virtual Machine (JVM)** to run programs on any platform without modification.

Key Aspects of Java:

- Used in **web development, mobile applications, enterprise applications, and embedded systems**.
- Designed to be **simple, secure, and robust**, with **automatic memory management**.
- Java code is first **compiled into bytecode**, which is executed by the **JVM**, making it independent of the underlying operating system.

3. Features of Java

Java is a powerful and versatile language that offers several essential features:

1. **Platform Independence** – Java programs run on any system that has a **JVM**.
2. **Object-Oriented** – Follows **OOP principles** such as encapsulation, inheritance, and polymorphism.
3. **Simple and Easy to Learn** – Java has a clean syntax, similar to **C++**, but without complexities like pointers.
4. **Robust and Secure** – Provides features like **exception handling and a security manager** to prevent unauthorized access.
5. **Automatic Memory Management** – Uses **Garbage Collection** to efficiently manage memory.
6. **Multithreading** – Supports **concurrent execution** of multiple threads for improved performance.
7. **High Performance** – Utilizes **Just-In-Time (JIT) Compilation** for optimization.
8. **Distributed Computing** – Java provides APIs like **RMI (Remote Method Invocation)** for network-based applications.
9. **Rich API** – Offers a vast collection of **built-in libraries** for various functionalities.

10. **Community Support** – Has a large and active **developer community** that continuously enhances Java.

4. Editions of Java

Java has evolved into different editions, each designed for specific application domains:

1. Java Standard Edition (Java SE)

- The **core Java platform** used for **general-purpose programming**.
- Includes essential libraries like **collections, threading, networking, and JDBC**.
- Used for **desktop applications, command-line tools, and basic server-side applications**.

2. Java Enterprise Edition (Java EE) *(Now Jakarta EE)*

- Designed for **large-scale, enterprise-level applications**.
- Includes APIs for **web services, servlets, JSP, EJB, JPA, and more**.
- Used for building **distributed, scalable, and secure web applications**.

3. Java Micro Edition (Java ME)

- A lightweight version of Java for **mobile and embedded systems**.
- Used in **IoT devices, smart cards, and feature phones**.
- Contains a subset of Java SE with **optimizations for constrained environments**.

4. JavaFX

- A modern **UI framework** for building **rich internet applications (RIA)**.
- Replaces **Swing and AWT** for creating **desktop applications with advanced UI features**.
- Supports **CSS, FXML**, and integrates with **web technologies**.

CHAPTER 3

Java Basics

Java Basics

What is a Language?

A **language** is a way to communicate, either between people or between people and computers. It has two important parts:

1. Syntax

The rules that decide how to write in that language (like grammar in English or how to write code in Java)

2. Semantics

The meaning of what is written (like correct sentences in English or logical correctness in programming).

Simple Example:

- **Syntax** is like traffic rules—without them, driving would be confusing.
- **Semantics** is like reaching your destination—following rules without a goal does not help.

Statically vs. Dynamically Typed Languages

Statically Typed Languages

- You must mention a variable's type when creating it.
- The type of a variable cannot change later.
- **Examples:** C, C++, Java, C#

Example in Java:

```
int number = 10;  // Correct
number = "hello"; // Error
```

Dynamically Typed Languages

- You do not need to mention types.
- A variable can store different types of values at different times.
- **Examples:** JavaScript, Python

Example in Python:

```
x = 10      # Correct
x = "hello" # Also correct
```

Simple Example:

- **Static Typing**: Like a train—always runs on fixed tracks (fixed type).
- **Dynamic Typing**: Like a car—can go on different roads (flexible type).

Why Java is Popular: The Role of JVM

- **Compiler (`javac`)** – Converts Java code into **bytecode** (a form that computers understand).
- **JVM (Java Virtual Machine)** – Runs bytecode on any computer system by converting it into machine code.

Key Advantage: "Write Once, Run Anywhere" (WORA) – Java programs can run on different systems without changes.

Simple Example:

- JVM is like a movie player—different devices (Windows, Mac, Linux) need different players, but the movie (bytecode) remains the same.

How to Save, Compile, and Run Java Files

1. **Save**: Create a file with a `.java` extension (e.g., `Example.java`).
2. **Compile**: Convert the source code into bytecode:

```
javac Example.java  # Creates Example.class (bytecode)
```

3. **Run**: Use JVM to execute it:

```
java Example
```

Example Java Program:

```
public class Example {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

- **Save** as `Example.java` → **Compile** → **Run** → **Output:** `Hello, Java!`

Bytecode and Execution Process

Bytecode is the intermediate step between Java code and machine code.

- Java code is converted into **bytecode** (`.class` files).
- The **JVM** translates bytecode into machine-specific code.
- **Advantage:** Java works on multiple systems without modification.

Simple Example:

- Bytecode is like a universal script—translators (JVM) convert it into different languages (machine code) for different computers.

Compiler vs. Interpreter

Feature	**Compiler**	**Interpreter**
Translation	Translates all code at once.	Translates code line by line.
Output	Creates bytecode (`.class` file).	Directly runs the program.
Speed	Faster because it is pre-translated.	Slower because it translates while running.
Error Checking	Finds all errors at once.	Stops at the first error.
Platform	Output may be platform-specific.	Works on different platforms if an interpreter exists.

How Java Uses Both

1. **Compiler (`javac`)** – Converts `.java` to `.class` (bytecode).
2. **Interpreter (JVM)** – Reads bytecode and runs it line by line.
3. **JIT (Just-In-Time) Compilation** – Speeds up execution by converting frequently used code into machine code.

Simple Example:

- JVM's **JIT Compiler** is like a restaurant—prepares popular dishes in advance for faster service.

First Java Program: "Hello, Java!"

The first Java program is usually **"Hello, Java!"** to help beginners understand the basics.

Java Code Example

```
public class HelloJava {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
```

```
    }
}
```

Explanation

1. **Class Declaration (`public class HelloJava`)**
   - Defines a class named `HelloJava`.
   - Every Java program has at least one class.
2. **Main Method (`public static void main(String[] args)`)**
   - **Starting point** of every Java program.
   - `public` → Can be used from anywhere.
   - `static` → Runs without creating an object.
   - `void` → Does not return anything.
   - `main` → Special method where execution starts.
   - `String[] args` → Stores command-line arguments.
3. **Printing to Console (`System.out.println("Hello, Java!");`)**
   - `System.out` → Standard output stream.
   - `println` → Prints text and moves to a new line.

How to Run the Program

1. **Save** as `HelloJava.java`.
2. **Compile**:

```
javac HelloJava.java
```

3. **Run**:

```
java HelloJava
```

Output: `Hello, Java!`

CHAPTER 4

Access Modifiers ,keyword, Identifiers

Access Modifiers in Java

Access modifiers define the visibility and accessibility of classes, methods, and variables in Java.

1. Types of Access Modifiers

- **Public**: Accessible from anywhere (inside and outside the package).
- **Private**: Accessible only within the same class.
- **Default (Package-private)**: If no modifier is specified, the member is accessible only within the same package.
- **Protected**: Accessible within the same package and subclasses (even if they are in different packages).

Access Modifiers for Top-Level Classes

- Only **public** and **default** are allowed for top-level classes.
- **private** and **protected** are not allowed for top-level classes.

Classification of Keywords in Java

Java keywords are reserved words that have a predefined meaning in the language.

1. Java Keywords

These are the core reserved words used in Java programming. They serve various functions such as defining data types, control flow, exception handling, etc.

2. Literals in Java

Literals are predefined constant values in Java that are not considered keywords but have a special meaning.

3. Reserved Words for Future Use

These words are reserved by Java but are not yet implemented or used.

Identifiers in Java

Identifiers are names given to variables, methods, classes, and interfaces in Java.

Rules for Identifiers

- Can contain letters (A-Z, a-z), digits (0-9), underscore (_), and dollar sign ($).
- Cannot start with a digit (e.g., `1variable` is invalid).
- Cannot be a Java keyword (e.g., `int`, `class`, `void` cannot be used as an identifier).
- Case-sensitive (`Variable` and `variable` are different).
- No special characters except _ and `$` are allowed.
- Can be of any length, but meaningful names are recommended.

Examples

```
int age;       // Valid
int _count;    // Valid
int $salary;   // Valid
int 1number;   // Invalid
int class;     // Invalid (class is a keyword)
```

Naming Conventions in Java

Naming conventions improve code readability and maintainability.

1. Class and Interface Names

- Use **PascalCase** (each word starts with a capital letter).
- **Examples**: `StudentDetails`, `BankAccount`, `EmployeeRecord`.

2. Variable and Method Names

- Use **camelCase** (first word is lowercase, subsequent words start with uppercase).
- **Examples**: `studentName`, `calculateSalary()`.

3. Constant Names

- Use **UPPER_CASE** with underscores (_).
- **Examples**: `PI`, `MAX_VALUE`, `MIN_BALANCE`.

4. Package Names

- Use all lowercase letters, usually in reverse domain name format.
- **Examples**: `com.example.project`, `org.company.module`.

CHAPTER 5

Data Types and Variables

Variables in Java

A variable is a container for storing data values during the execution of a program. In Java, every variable has a **name**, a **type**, and a **value**.

Types of Variables and Their Memory Storage Areas

Java provides three types of variables:

1. Local Variables

- **Definition:** Declared inside a method, constructor, or block.
- **Scope:** Limited to the block in which they are declared.
- **Memory Location:** Stored in **Stack Memory**.
- **Initialization:** Must be explicitly initialized before use.
- **Example:**

```
public class Example {
    public void display() {
        int number = 10;  // Local variable
        System.out.println("Number: " + number);
    }
}
```

2. Instance Variables (Non-static fields)

- **Definition:** Declared inside a class but outside any method.
- **Scope:** Belongs to an instance of the class, and each object gets its own copy.
- **Memory Location:** Stored in **Heap Memory**.
- **Initialization:** Default values are provided (0 for numbers, `null` for objects, `false` for boolean).
- **Example:**

```
class Student {
    String name;  // Instance variable
    int age;      // Instance variable
}
```

3. Class Variables (Static fields)

- **Definition:** Declared inside a class with the **static** keyword.
- **Scope:** Shared among all instances of the class.
- **Memory Location:** Stored in the **Method Area (Class Memory)**.
- **Initialization:** Default values are provided like instance variables.

- **Example:**

```
class Employee {
    static String company = "10000 coders"; // Class variable
}
```

Declaring Variables in Java

Java allows two ways to declare variables:

1. Separate Declaration and Initialization

You can first declare a variable and then assign a value later.

```
int number;  // Declaration
number = 25; // Initialization
```

2. Variable Initialization (Declaration + Initialization)

Declare and initialize a variable in a single step.

```
int number = 25;
```

Data Types in Java

Java is a **statically typed** language, meaning every variable must have a predefined type.

Java data types are categorized into **Primitive Data Types** and **Non-Primitive Data Types**.

![data types](https://media.geeksforgeeks.org/wp-content/uploads/20240809125618/Java-Data-Types.png "data types")

1. Primitive Data Types

Primitive data types store simple values directly in memory. They have a fixed size and do not require additional methods for manipulation.

Data Type	Size	Default Value	Description
`byte`	1 byte (8 bits)	0	Stores small integers (-128 to 127).
`short`	2 bytes (16 bits)	0	Stores medium-sized integers (-32,768 to 32,767).
`int`	4 bytes (32 bits)	0	Stores standard integers (-2^31 to 2^31-1).

`long`	8 bytes (64 bits)	0L	Stores large integers (-2^63 to 2^63-1).
`float`	4 bytes (32 bits)	0.0f	Stores decimal numbers with single precision.
`double`	8 bytes (64 bits)	0.0d	Stores decimal numbers with double precision.
`char`	2 bytes (16 bits)	'\u0000'	Stores a single character (Unicode values).
`boolean`	1 bit	false	Stores `true` or `false`.

Examples of Primitive Data Types

```
public class PrimitiveExample {
    public static void main(String[] args) {
        byte a = 127;
        short b = 32000;
        int c = 100000;
        long d = 10000000000L;
        float e = 5.75f;
        double f = 19.99d;
        char g = 'A';
        boolean h = true;

        System.out.println("byte: " + a);
        System.out.println("short: " + b);
        System.out.println("int: " + c);
        System.out.println("long: " + d);
        System.out.println("float: " + e);
        System.out.println("double: " + f);
        System.out.println("char: " + g);
        System.out.println("boolean: " + h);
    }
}
```

2. Non-Primitive Data Types

Non-primitive data types are more complex and can store multiple values. They store the **memory address** of the actual data rather than the data itself.

Data Type	Description
`String`	Stores sequences of characters (text).
`Arrays`	Stores multiple values of the same type.
`Classes`	User-defined blueprints for objects.
`Interfaces`	Defines a contract that a class must follow.

Examples of Non-Primitive Data Types

```
public class NonPrimitiveExample {
    public static void main(String[] args) {
        // String example
        String name = "John Doe";
```

```
        System.out.println("Name: " + name);

        // Array example
        int[] numbers = {10, 20, 30, 40};
        System.out.println("First number: " + numbers[0]);
    }
}
```

Key Differences Between Primitive and Non-Primitive Data Types

Feature	Primitive Data Types	Non-Primitive Data Types
Memory Storage	Stores actual values in memory	Stores memory address of values
Size	Fixed size	Varies based on implementation
Methods	Cannot use methods on them	Have built-in methods
Examples	`int`, `char`, `boolean`	`String`, `Array`, `Class`

CHAPTER 6

Operators in Java

Operators in Java

What are Operators and Operands?

In Java, **operators** are special symbols used to perform operations on variables and values. The **operand** is the value or variable on which the operator performs the operation.

Example

```
int a = 10 + 5;
```

Here:

- `+` is the **operator** (Addition Operator).
- `10` and `5` are **operands**.
- The result `15` is stored in `a`.

Types of Operators in Java

Java provides several types of operators:

1. **Arithmetic Operators**
2. **Relational (Comparison) Operators**
3. **Logical Operators**
4. **Bitwise Operators**
5. **Assignment Operators**
6. **Increment & Decrement Operators**
7. **Ternary Operator**
8. **Instanceof Operator**
9. **Type Casting Operators**

Each of these operator types is explained below in detail.

1. Arithmetic Operators

These operators are used to perform basic mathematical operations.

Operator	Description	Example
`+`	Addition	`a + b`

-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b (Integer division truncates decimal part)
%	Modulus (Remainder)	a % b

Example

```
public class ArithmeticExample {
    public static void main(String[] args) {
        int a = 10, b = 5;
        System.out.println("Addition: " + (a + b)); // 15
        System.out.println("Subtraction: " + (a - b)); // 5
        System.out.println("Multiplication: " + (a * b)); // 50
        System.out.println("Division: " + (a / b)); // 2
        System.out.println("Modulus: " + (a % b)); // 0
    }
}
```

2. Relational (Comparison) Operators

These operators compare two values and return a boolean result (`true` or `false`).

Operator	Description	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal to	a >= b
<=	Less than or equal to	a <= b

Example

```
public class RelationalExample {
    public static void main(String[] args) {
        int a = 10, b = 5;
        System.out.println(a == b); // false
        System.out.println(a != b); // true
        System.out.println(a > b);  // true
        System.out.println(a < b);  // false
        System.out.println(a >= b); // true
        System.out.println(a <= b); // false
    }
}
```

3. Logical Operators

Used to perform logical operations, mainly used in decision-making.

Operator	Description	Example
&&	Logical AND	(a > 5 && b < 10)
||	Logical OR	(a > 5 || b < 2)
!	Logical NOT	!(a > 5)

Example

```
public class LogicalExample {
    public static void main(String[] args) {
        int a = 10, b = 5;
        System.out.println((a > 5 && b < 10)); // true
        System.out.println((a > 5 || b < 2));  // true
        System.out.println(!(a > 5));          // false
    }
}
```

4. Bitwise Operators

Used to perform bit-level operations.

Operator	Description	Example
&	Bitwise AND	a & b
|	Bitwise OR	a | b
^	Bitwise XOR	a ^ b
~	Bitwise Complement	~a
<<	Left Shift	a << 2
>>	Right Shift	a >> 2

Example

```
public class BitwiseExample {
    public static void main(String[] args) {
        int a = 5, b = 3; // 5 -> 0101, 3 -> 0011
        System.out.println("Bitwise AND: " + (a & b)); // 0001 -> 1
        System.out.println("Bitwise OR: " + (a | b));  // 0111 -> 7
        System.out.println("Bitwise XOR: " + (a ^ b)); // 0110 -> 6
        System.out.println("Bitwise Complement: " + (~a)); // -6
        System.out.println("Left Shift: " + (a << 1)); // 1010 -> 10
        System.out.println("Right Shift: " + (a >> 1)); // 0010 -> 2
    }
```

```
}
```

5. Assignment Operators

Used to assign values to variables.

Operator	Description	Example
`=`	Assign	`a = 10`
`+=`	Add and assign	`a += 5` (same as `a = a + 5`)
`-=`	Subtract and assign	`a -= 5` (same as `a = a - 5`)
`*=`	Multiply and assign	`a *= 5` (same as `a = a * 5`)
`/=`	Divide and assign	`a /= 5` (same as `a = a / 5`)
`%=`	Modulus and assign	`a %= 5` (same as `a = a % 5`)

Example

```
public class AssignmentExample {
    public static void main(String[] args) {
        int a = 10;
        a += 5; // a = 15
        System.out.println(a);
        a *= 2; // a = 30
        System.out.println(a);
        a -= 5; // a = 25
        System.out.println(a);
    }
}
```

6. Increment & Decrement Operators

Used to increase or decrease the value of a variable.

Operator	Description	Example
`++`	Increment	`a++` (Post-increment) / `++a` (Pre-increment)
`--`	Decrement	`a--` (Post-decrement) / `--a` (Pre-decrement)

Example

```
public class IncrementExample {
    public static void main(String[] args) {
        int a = 5;
        System.out.println(a++); // 5 (then a becomes 6)
```

```
        System.out.println(++a); // 7
        System.out.println(a--); // 7 (then a becomes 6)
        System.out.println(--a); // 5
    }
}
```

7. Ternary Operator

Used as a shorthand for if-else statements.

Operator	Description	Example
`? :`	Ternary	`result = (a > b) ? a : b;`

Example

```
public class TernaryExample {
    public static void main(String[] args) {
        int a = 10, b = 20;
        int max = (a > b) ? a : b;
        System.out.println("Maximum: " + max);
    }
}
```

8. Instanceof Operator

Used to check if an object belongs to a specific class.

Example

```
class Animal { }
class Dog extends Animal { }

public class InstanceofExample {
    public static void main(String[] args) {
        Dog d = new Dog();
        System.out.println(d instanceof Dog); // true
        System.out.println(d instanceof Animal); // true
    }
}
```

9. Type Casting Operators

Used to convert one data type into another.

Example

```
public class TypeCastingExample {
    public static void main(String[] args) {
        int a = 10;
        double b = a; // Implicit Casting
        System.out.println(b);

        double c = 9.78;
        int d = (int) c; // Explicit Casting
        System.out.println(d);
    }
}
```

CHAPTER 7

Static Methods in Java

Static Methods in Java

What are Static Methods?

Static methods in Java are methods that belong to the class rather than instances of the class. They are declared using the `static` keyword and can be called without creating an instance of the class.

Declaration of Static Methods

A static method is defined using the `static` keyword before the return type:

```
class Example {
    static void displayMessage() {
        System.out.println("This is a static method.");
    }
}
```

How to Call Static Methods?

Since static methods belong to the class, they can be called using the class name:

```
public class Test {
    public static void main(String[] args) {
        Example.displayMessage(); // Calling static method using class name
    }
}
```

Characteristics of Static Methods

1. **Belong to Class:** Static methods are associated with the class itself rather than any specific object.
2. **Can Access Static Members Only:** Static methods can only access static variables and other static methods directly.
3. **Cannot Use `this` or `super`:** Static methods cannot refer to instance variables or methods using `this` or `super`.
4. **Improves Memory Efficiency:** Since static methods do not require object instantiation, they help optimize memory usage.
5. **Shared Among All Instances:** A static method can be accessed by all instances of the class without duplication.

Example: Static Method Usage

```
class MathOperations {
    static int add(int a, int b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        int result = MathOperations.add(10, 20);
        System.out.println("Addition Result: " + result);
    }
}
```

Accessing Static Methods Within the Same Class

If the static method is within the same class, it can be called directly without using the class name:

```
class Sample {
    static void greet() {
        System.out.println("Hello, Welcome!");
    }

    public static void main(String[] args) {
        greet(); // Directly calling static method
    }
}
```

Three Ways to Access Static Members in Java:

1. **Using the Class Name** (Recommended)

```
Example.display(); // Accessing using class name
```

2. **Using an Object (Not Recommended but Possible)**

```
Example obj = new Example();
obj.display(); // Warning: Should be accessed using class name
```

3. **Calling Directly (Within the Same Class)**

```
display(); // Direct call within the same class
```

Best Practice:

- Always use **ClassName.methodName()** to call static methods.
- Avoid using objects to access static methods.

- Within the same class, direct calls are acceptable.

Restrictions of Static Methods

1. **Cannot Access Instance Members:** Static methods cannot access instance variables or instance methods directly.
2. **Cannot Override in Subclasses:** Static methods cannot be overridden but can be hidden if redefined in a subclass.
3. **Cannot Use `this` or `super`:** As static methods are not tied to an object, they cannot use `this` or `super` keywords.

Example: Attempting to Access Instance Members in Static Method (Error)

```
class Demo {
    int x = 10;

    static void show() {
          // System.out.println(x); // Compilation Error: Cannot access instance variable in static
method
    }
}
```

When to Use Static Methods?

- When a method does not depend on instance variables.
- For utility or helper methods (e.g., `Math.sqrt()`, `Collections.sort()`).
- When a function performs an independent operation (e.g., database connection utility methods).

Summary

- Static methods belong to the class, not instances.
- They can be called using the class name.
- They can access only static members.
- They cannot use `this` or `super`.
- They cannot be overridden but can be hidden in subclasses.
- Best suited for utility functions and independent operations.

CHAPTER 8

Non-Static Methods in Java

Non-Static Methods in Java

Introduction

In Java, methods can be classified as **static** or **non-static**. Non-static methods, also known as **instance methods**, belong to objects of a class rather than the class itself. These methods require an instance of the class to be invoked.

Characteristics of Non-Static Methods

1. **Instance-Specific**: Non-static methods operate on instance variables and can access other non-static members of the class.
2. **Object Dependency**: They can only be invoked through an object of the class.
3. **Can Access Both Static and Non-Static Members**: Unlike static methods, which can only directly access static members, non-static methods can access both.
4. **Memory Allocation**: Non-static methods are stored in the heap memory along with the object of the class.
5. **Supports Method Overriding**: Non-static methods can be overridden in subclasses.

Syntax of a Non-Static Method

```
class Example {
    int number = 10; // Instance variable

    // Non-static method
    void display() {
        System.out.println("Number: " + number);
    }
}

public class Main {
    public static void main(String[] args) {
        Example obj = new Example(); // Creating an instance
        obj.display(); // Calling the non-static method
    }
}
```

Explanation

- `display()` is a **non-static method** inside the `Example` class.
- It accesses the instance variable `number`.

- The method is called using an object (`obj.display();`).

Types of Non-Static Methods

1. **Instance Methods with Parameters and Return Type**

A non-static method can have parameters and return a value.

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        int sum = calc.add(5, 10);
        System.out.println("Sum: " + sum);
    }
}
```

2. **Instance Methods Without Parameters and Return Type**

```
class Greeting {
    void sayHello() {
        System.out.println("Hello, Welcome to Java!");
    }
}

public class Main {
    public static void main(String[] args) {
        Greeting greet = new Greeting();
        greet.sayHello();
    }
}
```

Accessing Non-Static Methods in the Same Class

A non-static method in the same class can be accessed using `this` keyword or directly.

```
class Demo {
    void show() {
        System.out.println("Inside show method");
    }
    void display() {
        show(); // Direct call
        this.show(); // Using 'this' keyword
    }
```

```
}
```

Difference Between Static and Non-Static Methods

Feature	Static Method	Non-Static Method
Belongs to	Class	Object (Instance)
Access	Only static members	Both static and non-static members
Invocation	ClassName.method()	ObjectName.method()
Memory Allocation	Class area	Heap (with the object)
Method Overriding	Cannot be overridden	Can be overridden

Use Cases of Non-Static Methods

- **Encapsulation**: Non-static methods allow data hiding and controlled access.
- **Instance-Specific Operations**: Useful when behavior varies per object.
- **OOP Concepts**: Used in inheritance, method overriding, and polymorphism.

Conclusion

Non-static methods play a crucial role in Java's object-oriented paradigm. They allow interaction with instance variables and provide flexibility in programming. Understanding their behavior is essential for designing efficient Java applications.

CHAPTER 9

Method Access Method

Method Access Method

In Java, methods can be **static** or **non-static (instance methods)**. How one method calls another depends on whether they are static or non-static.

There are four possible combinations when one method tries to access another:

1. Static Method Calling Another Static Method

- Both are class-level methods stored in the **static memory area**.
- A static method can call another static method **directly** without creating an object.

Example:

```
class MyClass {
    static void methodA() {
        methodB();  // Direct call
    }

    static void methodB() {
        System.out.println("Static Method B");
    }

    public static void main(String[] args) {
        methodA();  // Calling static method from main
    }
}
```

2. Non-Static Method Calling a Static Method

- A non-static (instance) method can call a static method **directly**.
- Since static methods belong to the class, they are accessible from any instance.

Example:

```
class MyClass {
    void methodA() {
        methodB();  // Direct call
    }

    static void methodB() {
```

```
        System.out.println("Static Method B");
    }

    public static void main(String[] args) {
        MyClass obj = new MyClass();
        obj.methodA();  // Calling non-static method
    }
}
```

3. Static Method Calling a Non-Static Method

- A static method **cannot** directly call a non-static method because non-static methods belong to an instance.
- To access a non-static method, an object must be created inside the static method.

Example:

```
class MyClass {
    static void methodA() {
        // methodB();  // Not allowed directly
        MyClass obj = new MyClass();
        obj.methodB();  // Correct way
    }

    void methodB() {
        System.out.println("Non-static Method B");
    }

    public static void main(String[] args) {
        methodA();  // Calling static method from main
    }
}
```

4. Non-Static Method Calling Another Non-Static Method

- Non-static methods can call other non-static methods **directly** because they belong to the same object.

Example:

```
class MyClass {
    void methodA() {
        methodB();  // Direct call
    }

    void methodB() {
        System.out.println("Non-static Method B");
    }
```

```
    public static void main(String[] args) {
        MyClass obj = new MyClass();
        obj.methodA();  // Calling non-static method
    }
}
```

Summary

From → To	Possible Without Object?	Explanation
Static → Static	Yes	Both are class-level (static area)
Non-Static → Static	Yes	Non-static can access static easily
Static → Non-Static	No	Needs an object because non-static methods belong to instances
Non-Static → Non-Static	Yes	Both belong to the same object

Business Logic Class (BLC) and Executable Logic Class (ELC) in Java

In Java, **BLC (Business Logic Class)** and **ELC (Executable Logic Class)** are design concepts that separate concerns in an application, ensuring maintainability and scalability.

1. Business Logic Class (BLC)

The **BLC** is responsible for handling business rules, calculations, and validations. It is independent of the presentation or execution layer and does not contain UI-related logic.

Key Responsibilities:

- Encapsulates business rules and domain logic.
- Ensures reusability and testability.
- Can interact with databases through DAO (Data Access Object) but does not handle direct execution.

Example: Circle Calculation

```
// Business Logic Class (BLC)
public class Circle {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double calculateArea() {
        return Math.PI * radius * radius;
    }

    public double calculateCircumference() {
        return 2 * Math.PI * radius;
    }

    public void displayCircleDetails() {
        System.out.println("Circle Radius: " + radius);
        System.out.println("Circle Area: " + calculateArea());
        System.out.println("Circle Circumference: " + calculateCircumference());
    }
}
```

2. Executable Logic Class (ELC)

The **ELC** acts as a controller or execution class that interacts with the user and invokes methods of the **BLC**.

Key Responsibilities:

- Handles user input and output.
- Calls the BLC methods and displays results.
- Acts as a bridge between UI and business logic.
- Typically contains the `main()` method for execution.

Example: Circle Calculation Execution

```
// Executable Logic Class (ELC)
import java.util.Scanner;

public class CircleApp {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the radius of the circle: ");
        double radius = scanner.nextDouble();

        Circle circle = new Circle(radius);
        circle.displayCircleDetails();

        scanner.close();
    }
}
```

Key Differences:

Feature	Business Logic Class (BLC)	Executable Logic Class (ELC)
Purpose	Contains business rules and logic.	Handles user interaction and execution flow.
Independence	Independent of UI and execution logic.	Depends on BLC for processing data.
Reusability	Can be reused in different applications.	Not reusable, specific to execution.
Contains `main()`?	No	Yes

Why Use This Approach?

Separation of Concerns – BLC focuses on logic, ELC handles execution.

Reusability – `Circle` class can be used in different applications.

Maintainability – If logic changes, update only BLC.

Testability – `Circle` class can be tested independently without modifying the UI.

This separation is commonly used in **MVC (Model-View-Controller)** architecture:

- **Model (BLC)** represents data and business logic.
- **View** handles UI.
- **Controller (ELC)** manages execution and user interaction.

CHAPTER 11

Control Flow Statements in Java

Control Flow Statements in Java

Control flow statements determine the execution order of statements in a Java program. They help in decision-making, looping, and branching. Java provides three main types of control flow statements:

1. **Decision-Making Statements (Conditional Statements)**
2. **Looping Statements (Iteration Statements)**
3. **Jump Statements (Branching Statements)**

1. Decision-Making Statements (Conditional Statements)

These statements allow the program to execute different code blocks based on conditions.

1.1 `if` Statement

Executes a block of code only if the condition is `true`.

Syntax:

```
if (condition) {
    // Code to execute if condition is true
}
```

1.2 `if-else` Statement

Provides an alternative block of code if the condition is `false`.

Syntax:

```
if (condition) {
    // Code to execute if condition is true
} else {
    // Code to execute if condition is false
}
```

1.3 `if-else if-else` Statement

Used when multiple conditions need to be checked in sequence.

Syntax:

```
if (condition1) {
    // Code to execute if condition1 is true
} else if (condition2) {
```

```
    // Code to execute if condition2 is true
} else {
    // Code to execute if all conditions are false
}
```

1.4 `switch` Statement

Executes different blocks based on the value of an expression.

Syntax:

```
switch (expression) {
    case value1:
        // Code to execute if expression == value1
        break;
    case value2:
        // Code to execute if expression == value2
        break;
    default:
        // Code to execute if no case matches
}
```

2. Looping Statements (Iteration Statements)

Loops are used to execute a block of code multiple times.

2.1 `for` Loop

Executes a block of code a fixed number of times.

Syntax:

```
for (initialization; condition; update) {
    // Code to execute in each iteration
}
```

2.2 `while` Loop

Executes a block of code repeatedly as long as the condition is true.

Syntax:

```
while (condition) {
    // Code to execute in each iteration
}
```

2.3 `do-while` Loop

Ensures that the block of code is executed at least once before checking the condition.

Syntax:

```
do {
    // Code to execute in each iteration
} while (condition);
```

3. Jump Statements (Branching Statements)

Jump statements alter the flow of execution.

3.1 `break` Statement

Used to exit a loop or a `switch` statement.

Example:

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) break;
    System.out.println(i);
}
```

3.2 `continue` Statement

Skips the current iteration and moves to the next.

Example:

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    System.out.println(i);
}
```

3.3 `return` Statement

Exits from a method and optionally returns a value.

Example:

```
public static int sum(int a, int b) {
    return a + b;
}

public static void main(String[] args) {
    int result = sum(5, 10);
    System.out.println("Sum: " + result);
}
```

Summary of Java Control Flow Statements

Type	Statement	Description
Decision-Making	`if`, `if-else`, `if-else if-else`	Executes different blocks of code based on conditions
	`switch`	Executes code based on a variable's value
Looping	`for`	Repeats code a fixed number of times
	`while`	Repeats code while a condition is true
	`do-while`	Executes code at least once, then checks the condition
Jump	`break`	Exits a loop or `switch` statement
	`continue`	Skips the current iteration and moves to the next
	`return`	Exits a method and optionally returns a value

CHAPTER 12

if and if-else

if and if-else

Conditional statements in Java allow us to control the flow of execution based on conditions.

1. `if` Statement

- **`if` Statement** – Executes a block of code when a condition is `true`.

The `if` statement is the simplest conditional statement in Java. It checks whether a condition evaluates to `true` and executes the code inside its block. If the condition is `false`, the block is skipped.

Syntax:

```
if (condition) {
    // Code to execute if condition is true
}
```

- The **condition** inside parentheses `( )` must be a boolean expression that evaluates to `true` or `false`.
- If the condition is `true`, the statements inside the curly braces `{ }` execute.
- If the condition is `false`, the block is skipped, and execution continues after the `if` statement.

Flow - chart :

![if](https://res.cloudinary.com/dtoupvd2u/image/upload/v1704797597/1_2_7d52ce0c83.png "if")

Example 1: Checking if a number is positive

```
public class IfExample {
    public static void main(String[] args) {
        int number = 10;

        if (number > 0) { // Checks if the number is greater than 0
            System.out.println("The number is positive.");
        }

        System.out.println("End of program.");
    }
}
```

Output:

```
The number is positive.
```

```
End of program.
```

Explanation:

- Since `number > 0` is `true`, the `if` block executes.
- The message **"The number is positive."** is printed.
- After the `if` block, execution continues normally.

Example 2: `if` Statement Skipping Execution

```
public class IfExample {
    public static void main(String[] args) {
        int number = -5;

        if (number > 0) { // This condition is false
            System.out.println("The number is positive.");
        }

        System.out.println("End of program.");
    }
}
```

Output:

```
End of program.
```

Explanation:

- Since `number > 0` is `false`, the `if` block is skipped.
- The program simply prints **"End of program."**

2. `if-else` Statement

- **`if-else` Statement** – Executes one block if the condition is `true`, otherwise executes another block.

The `if-else` statement is used when we want to execute one block of code if the condition is `true` and another block if the condition is `false`.

Syntax:

```
if (condition) {
    // Code to execute if condition is true
} else {
    // Code to execute if condition is false
}
```

- If the condition is `true`, the code inside the `if` block executes.
- If the condition is `false`, the code inside the `else` block executes.

Flow - chart :

![if-else](https://media.geeksforgeeks.org/wp-content/uploads/if-else-2-1.jpg "if-else")

Example 1: Checking if a number is positive or not

```
public class IfElseExample {
    public static void main(String[] args) {
        int number = -5;

        if (number > 0) {
            System.out.println("The number is positive.");
        } else {
            System.out.println("The number is not positive.");
        }

        System.out.println("End of program.");
    }
}
```

Output:

```
The number is not positive.
End of program.
```

Explanation:

- Since `number > 0` is `false`, the `else` block executes.
- The message **"The number is not positive."** is printed.

Example 2: Checking if a number is even or odd

```
public class EvenOddCheck {
    public static void main(String[] args) {
        int number = 7;

        if (number % 2 == 0) { // Checks if number is divisible by 2
            System.out.println("The number is even.");
        } else {
            System.out.println("The number is odd.");
        }

        System.out.println("End of program.");
    }
}
```

Output:

```
The number is odd.
End of program.
```

Explanation:

- The condition `number % 2 == 0` checks if `7` is divisible by `2`.
- Since `7 % 2` equals `1`, the condition is `false`, so the `else` block executes.

Comparison: `if` vs. `if-else`

Feature	`if` Statement	`if-else` Statement
Executes a block if the condition is `true`		
Skips execution if condition is `false`		(Executes `else` block instead)
Handles alternative cases		
Example Scenario	Check if a number is positive	Check if a number is positive or not

---

Key Points to Remember

1. The condition inside `if` or `if-else` must evaluate to `true` or `false`.
2. Curly braces `{ }` are **optional** for single-line statements but recommended for readability.
3. The `if` block executes only when the condition is `true`; otherwise, it is skipped.
4. The `if-else` ensures that one block always executes—either `if` or `else`.

CHAPTER 13

if-else if-else

`if-else if-else` Statement

- **`if-else if-else` Statement** – Used when we have **multiple conditions** to check one after another.

The `if-else if-else` ladder helps us choose between **many possible options**. It evaluates conditions from top to bottom, and **executes the block** of the first condition that is `true`.

Syntax:

```
if (condition1) {
    // Executes if condition1 is true
} else if (condition2) {
    // Executes if condition2 is true
} else if (condition3) {
    // Executes if condition3 is true
} else {
    // Executes if all above conditions are false
}
```

- Each condition is checked in order.
- As soon as a condition evaluates to `true`, its corresponding block is executed and the rest are skipped.
- If **none** of the conditions are `true`, the `else` block executes.

Flow - chart:

![if-else if -else](https://embetronicx.b-cdn.net/wp-content/uploads/2023/07/else-if-ladder.webp "if-else if -else")

Example 1: Grade Checker

```
public class GradeCheck {
    public static void main(String[] args) {
        int marks = 75;

        if (marks >= 90) {
            System.out.println("Grade: A");
        } else if (marks >= 75) {
            System.out.println("Grade: B");
        } else if (marks >= 60) {
            System.out.println("Grade: C");
```

```
        } else if (marks >= 40) {
            System.out.println("Grade: D");
        } else {
            System.out.println("Grade: F (Fail)");
        }

        System.out.println("End of program.");
    }
}
```

Output:

```
Grade: B
End of program.
```

Explanation:

- `marks = 75`, so `marks >= 90` is `false`, but `marks >= 75` is `true`.
- So it prints **"Grade: B"** and skips the remaining conditions.

Example 2: Temperature Range Check

```
public class TemperatureCheck {
    public static void main(String[] args) {
        int temp = 5;

        if (temp >= 30) {
            System.out.println("It's hot.");
        } else if (temp >= 15) {
            System.out.println("It's warm.");
        } else if (temp >= 0) {
            System.out.println("It's cold.");
        } else {
            System.out.println("It's freezing!");
        }

        System.out.println("End of program.");
    }
}
```

Output:

```
It's cold.
End of program.
```

Comparison: `if` vs `if-else` vs `if-else if`

Feature	`if`	`if-else`	`if-else if`
Checks one condition			
Handles two outcomes			
Handles multiple conditions			
Executes only one block			
Skips remaining conditions after one is true			

Key Points to Remember

1. Conditions are checked **in order** from top to bottom.
2. Only the **first true condition's block** gets executed.
3. The `else` block is **optional** but used to handle "none of the above" cases.
4. Helps avoid writing multiple separate `if` statements when options are related.

Scanner Class in Java

1. What is `Scanner` in Java?

- The **`Scanner`** class is part of the `java.util` package.
- It is used to **read input** from various sources: keyboard (System.in), files, strings, etc.
- Mainly used for **interactive input** in console-based Java applications.

2. How to Import Scanner?

```
import java.util.Scanner;
```

3. Basic Syntax for Using `Scanner`

```
Scanner sc = new Scanner(System.in);
```

> *`System.in` is the standard input stream (keyboard).*

4. Commonly Used Scanner Methods

Method	Description	Example
`next()`	Reads one word (up to space)	`sc.next()`
`nextLine()`	Reads an entire line	`sc.nextLine()`
`nextInt()`	Reads an integer	`sc.nextInt()`
`nextDouble()`	Reads a double	`sc.nextDouble()`
`nextBoolean()`	Reads a boolean	`sc.nextBoolean()`
`nextFloat()`	Reads a float	`sc.nextFloat()`
`nextLong()`	Reads a long	`sc.nextLong()`

5. Example Program Using Scanner

```
import java.util.Scanner;

public class InputExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine(); // reads full name

        System.out.print("Enter your age: ");
        int age = sc.nextInt();

        System.out.println("Name: " + name);
        System.out.println("Age: " + age);

        sc.close();
    }
}
```

6. Common Drawbacks of `Scanner`

a. **Mixing `nextLine()` with other methods causes skipping**

- After using `nextInt()`, `nextLine()` seems to "skip" input.

Why?

- `nextInt()` doesn't consume the newline character (`\n`) after the number.
- So, when `nextLine()` is called next, it reads that leftover newline.

Example Problem Code:

```
System.out.print("Enter age: ");
int age = sc.nextInt();  // user enters 25 + Enter

System.out.print("Enter name: ");
String name = sc.nextLine();  // skipped due to leftover \n
```

Solution: Add an extra `sc.nextLine()`

```
int age = sc.nextInt();
sc.nextLine(); // consume the leftover newline
String name = sc.nextLine();
```

b. **Does not validate input types automatically**

If the user enters wrong input (e.g., string instead of int), it throws an exception.

Example:

```
int num = sc.nextInt(); // If user enters "abc", InputMismatchException occurs
```

Solution: Use `hasNextInt()` to check

```
if (sc.hasNextInt()) {
    int num = sc.nextInt();
} else {
    System.out.println("Invalid input. Enter an integer.");
}
```

---

c. **Resource leak warning**

- If `Scanner` is not closed, you might get a warning in IDE.
- But closing `Scanner(System.in)` may close the standard input stream permanently.

Best practice:
Only close `Scanner` when you're sure input is no longer needed **or use try-with-resources** (for file input).

```
try (Scanner sc = new Scanner(System.in)) {
    // read input
}
```

---

7. Alternatives to Scanner

Class	Use Case
`BufferedReader`	Faster input, especially for competitive coding
`Console`	For secure inputs like passwords (not supported in IDEs)
`DataInputStream`	Low-level binary data input
`Scanner(File)`	Reads from file

---

8. Drawbacks of Scanner

Drawback	Description	Fix/Alternative
Input skipping	`nextLine()` skipped after numeric input	Add `sc.nextLine()` after numeric input
Type mismatch error	InputMismatchException if wrong input type	Use `hasNextInt()`, `hasNextDouble()` etc.
Resource warning	IDE shows warning if scanner not closed	Close with `sc.close()` or use try-with-resources
Slow in large input	Not fast for competitive coding	Use `BufferedReader` instead

9. Alternative: BufferedReader (for fast input)

```
import java.io.*;

BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
String name = br.readLine();          // Read full line
int age = Integer.parseInt(br.readLine()); // Read number after converting
```

10. Comparison: Scanner vs BufferedReader

Feature	Scanner	BufferedReader
Syntax	Simple	Requires try-catch
Performance	Slower	Faster
Reads	Tokens and primitives	Lines only
Parsing	Built-in	Manual parsing needed

11. Conclusion

- `Scanner` is beginner-friendly, useful for basic I/O operations.
- Be careful with `nextLine()` after numeric inputs.
- For large data or performance-critical apps, consider `BufferedReader`.

CHAPTER 15

switch

switch statement in java

The `switch` statement in Java is a control flow statement used to execute one block of code from multiple options based on the value of an expression. It is an alternative to using multiple `if-else` statements when dealing with multiple possible values of a variable.

1. Syntax of `switch` Statement

```
switch (expression) {
    case value1:
        // Code to execute if expression == value1
        break;
    case value2:
        // Code to execute if expression == value2
        break;
    case value3:
        // Code to execute if expression == value3
        break;
    default:
        // Code to execute if no case matches
}
```

2. How It Works

1. The `expression` is evaluated.
2. The value of `expression` is compared with each `case` label.
3. If a match is found, the corresponding block of code is executed.
4. The `break` statement is used to exit the `switch` after executing a case.
5. If no match is found, the `default` block is executed (if present).

Flow chart :

![switch](https://www.infobrother.com/web/library/cplusplus/img/switch-statement-cpp.webp "switch")

3. Important Rules

1. The `expression` **must evaluate to an integer, character, string, or enum** (allowed types: `byte`, `short`, `int`, `char`, `String`, `Enum`, `Wrapper classes` like `Integer`).
2. Each `case` must have a constant value or a `final` variable.
3. The `break` statement is optional but prevents fall-through.
4. The `default` case is optional but recommended.
5. Duplicate `case` values are not allowed.

4. Example: Basic `switch` with `int`

```
public class SwitchExample {
    public static void main(String[] args) {
        int day = 3;

        switch (day) {
            case 1:
                System.out.println("Monday");
                break;
            case 2:
                System.out.println("Tuesday");
                break;
            case 3:
                System.out.println("Wednesday");
                break;
            case 4:
                System.out.println("Thursday");
                break;
            case 5:
                System.out.println("Friday");
                break;
            default:
                System.out.println("Weekend");
        }
    }
}
```

Output:

`Wednesday`

5. Example: `switch` with `String`

```
public class SwitchStringExample {
    public static void main(String[] args) {
        String language = "Java";
```

```
        switch (language) {
            case "Java":
                System.out.println("You selected Java.");
                break;
            case "Python":
                System.out.println("You selected Python.");
                break;
            default:
                System.out.println("Unknown language.");
        }
    }
}
```

Output:

```
You selected Java.
```

6. Example: `switch` with `char`

```
public class SwitchCharExample {
    public static void main(String[] args) {
        char grade = 'B';

        switch (grade) {
            case 'A':
                System.out.println("Excellent!");
                break;
            case 'B':
                System.out.println("Good job!");
                break;
            case 'C':
                System.out.println("You can improve.");
                break;
            default:
                System.out.println("Invalid grade.");
        }
    }
}
```

Output:

```
Good job!
```

7. Example: `switch` with `enum`

```
enum Day {
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
```

```
}

public class SwitchEnumExample {
    public static void main(String[] args) {
        Day today = Day.WEDNESDAY;

        switch (today) {
            case MONDAY:
                System.out.println("Start of the week.");
                break;
            case FRIDAY:
                System.out.println("Weekend is near.");
                break;
            case SUNDAY:
                System.out.println("Weekend!");
                break;
            default:
                System.out.println("A regular day.");
        }
    }
}
```

Output:

```
A regular day.
```

8. Fall-Through Behavior (Without `break`)

If the `break` statement is **not** used, execution continues into the next case.

Example: Fall-Through

```
public class SwitchFallThrough {
    public static void main(String[] args) {
        int number = 2;

        switch (number) {
            case 1:
                System.out.println("Case 1");
            case 2:
                System.out.println("Case 2");
            case 3:
                System.out.println("Case 3");
                break;
            default:
                System.out.println("Default Case");
        }
    }
}
```

Output:

```
Case 2
Case 3
```

- Since there is no `break` in case `2`, execution continues into case `3`.

9. `switch` with Multiple Case Labels

You can combine multiple cases for the same code block.

```
public class SwitchMultipleCases {
    public static void main(String[] args) {
        int number = 2;

        switch (number) {
            case 1:
            case 2:
            case 3:
                System.out.println("Number is 1, 2, or 3.");
                break;
            default:
                System.out.println("Number is not 1, 2, or 3.");
        }
    }
}
```

Output:

```
Number is 1, 2, or 3.
```

10. Enhanced `switch` (Java 14+)

Java 14 introduced an **enhanced `switch`** expression that returns a value and uses the arrow `->` syntax.

Example: New `switch` Syntax

```
public class EnhancedSwitch {
    public static void main(String[] args) {
        int day = 3;
        String dayType = switch (day) {
            case 1, 2, 3, 4, 5 -> "Weekday";
            case 6, 7 -> "Weekend";
            default -> "Invalid";
        };
        System.out.println(dayType);
    }
```

```
}
```

Output:

```
Weekday
```

Advantages of Enhanced `switch`

1. **No need for `break`.**
2. **Can return values.**
3. **More concise and readable.**

11. When to Use `switch` vs `if-else`

Feature	`switch`	`if-else`
Use case	Multiple fixed values	Complex conditions, ranges, expressions
Readability	Better for multiple options	Better for complex logic
Performance	Faster for large sets of options	Slower in some cases

Use `switch` when you have multiple possible values for a single variable. Use `if-else` when conditions involve **ranges, boolean expressions, or relational operators**.

Conclusion

- The `switch` statement is useful for checking multiple values.
- It works with `int`, `char`, `String`, `enum`, and `Wrapper classes`.
- The `break` statement prevents fall-through behavior.
- Java 14 introduced an enhanced `switch` with `->` syntax.
- Use `switch` when dealing with **fixed values** and `if-else` for **complex conditions**.

CHAPTER 16

for

for Loop in Java

The `for` loop in Java is a control flow statement used to **repeat a block of code** a specific number of times. It is ideal when the number of iterations is known in advance.

1. Syntax of `for` Loop

```
for (initialization; condition; update) {
    // Code to be executed in each iteration
}
```

2. How It Works

1. **Initialization** – executed once at the beginning.
2. **Condition** – checked before every iteration; loop continues while it's `true`.
3. **Update** – executed after every iteration.
4. **Loop Body** – executes if condition is `true`.

3. Flowchart

![for](https://dpfdothtmbwzx.cloudfront.net/images/java-for-loop.svg "for")

4. Example: Print Numbers from 1 to 5

```
public class ForLoopExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            System.out.println(i);
        }
    }
}
```

Output:

```
1
2
3
4
5
```

5. Example: Print Even Numbers from 2 to 10

```
public class EvenNumbers {
    public static void main(String[] args) {
        for (int i = 2; i <= 10; i += 2) {
            System.out.println(i);
        }
    }
}
```

Output:

```
2
4
6
8
10
```

6. Example: Reverse Counting

```
public class ReverseCount {
    public static void main(String[] args) {
        for (int i = 5; i >= 1; i--) {
            System.out.println(i);
        }
    }
}
```

Output:

```
5
4
3
2
1
```

7. Infinite `for` Loop

```
for (;;) {
    // runs forever unless you use 'break' or 'return'
}
```

8. Nested `for` Loops

You can place one `for` loop inside another.

Example: Print 3x3 Grid

```
public class NestedLoop {
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 3; j++) {
                System.out.print("(" + i + "," + j + ") ");
            }
            System.out.println();
        }
    }
}
```

Output:

```
(1,1) (1,2) (1,3)
(2,1) (2,2) (2,3)
(3,1) (3,2) (3,3)
```

9. When to Use `for` Loop

Use Case	Loop Type
Known number of iterations	`for` loop
Unknown number, condition-based	`while` loop
Iterating over arrays/collections	`for-each` loop

Conclusion

- `for` loops are best when the **number of iterations is known**.

- Use `for-each` to loop through **arrays and collections**.
- You can nest `for` loops to handle **multi-dimensional** logic.

CHAPTER 17

while

`while` Loop in Java

The `while` loop in Java is a control flow statement used to **repeat a block of code as long as a condition is true**. It is ideal when the number of iterations is **not known in advance**.

1. Syntax of `while` Loop

```
while (condition) {
    // Code to be executed
}
```

2. How It Works

1. **Condition** – evaluated **before** each iteration.
2. **Loop Body** – executes **only if** the condition is `true`.
3. **Update** – must be done **manually inside** the loop.
4. **Stops** when the condition becomes `false`.

3. Flowchart

![while](https://images.tpointtech.com/tutorial/dart/images/dart-while-loop.png "while")

4. Example: Print Numbers from 1 to 5

```
public class WhileLoopExample {
    public static void main(String[] args) {
        int i = 1;
        while (i <= 5) {
            System.out.println(i);
            i++;
        }
    }
```

```
}
```

Output:

```
1
2
3
4
5
```

5. Example: Print Even Numbers from 2 to 10

```
public class EvenNumbers {
    public static void main(String[] args) {
        int i = 2;
        while (i <= 10) {
            System.out.println(i);
            i += 2;
        }
    }
}
```

Output:

```
2
4
6
8
10
```

6. Example: Reverse Counting

```
public class ReverseCount {
    public static void main(String[] args) {
        int i = 5;
        while (i >= 1) {
            System.out.println(i);
            i--;
        }
    }
}
```

Output:

```
5
4
```

```
3
2
1
```

7. Infinite `while` Loop

```
while (true) {
    // runs forever unless you use 'break' or 'return'
}
```

8. Nested `while` Loops

You can place one `while` loop inside another.

Example: Print 3x3 Grid

```
public class NestedWhile {
    public static void main(String[] args) {
        int i = 1;
        while (i <= 3) {
            int j = 1;
            while (j <= 3) {
                System.out.print("(" + i + "," + j + ") ");
                j++;
            }
            System.out.println();
            i++;
        }
    }
}
```

Output:

```
(1,1) (1,2) (1,3)
(2,1) (2,2) (2,3)
(3,1) (3,2) (3,3)
```

9. When to Use `while` Loop

Use Case	Loop Type

Unknown number of iterations	`while` loop
Known number of iterations	`for` loop
Iterate through collections	`for-each` loop

Conclusion

- Use `while` when the **end condition is not known**.
- Be careful to **update variables inside the loop** to avoid infinite loops.
- Great for **reading input** or **waiting for a condition**.

CHAPTER 18

do-while

`do-while` Loop in Java

The `do-while` loop in Java is a control flow statement that **executes the loop body at least once**, and then repeats as long as the condition is `true`. It is ideal when the loop must run **at least once** before checking the condition.

1. Syntax of `do-while` Loop

```
do {
    // Code to be executed
} while (condition);
```

2. How It Works

1. **Loop Body** – executes **once before** checking the condition.
2. **Condition** – evaluated **after** each iteration.
3. If the condition is `true`, the loop continues.
4. The loop **always runs at least once**, even if the condition is `false` initially.

3. Flowchart

![do-while](https://media.geeksforgeeks.org/wp-content/uploads/20221006152307/dowhileloopinc.png "do-while")

4. Example: Print Numbers from 1 to 5

```
public class DoWhileExample {
    public static void main(String[] args) {
        int i = 1;
        do {
            System.out.println(i);
            i++;
```

```
        } while (i <= 5);
    }
}
```

Output:

```
1
2
3
4
5
```

5. Example: Print Even Numbers from 2 to 10

```
public class EvenNumbers {
    public static void main(String[] args) {
        int i = 2;
        do {
            System.out.println(i);
            i += 2;
        } while (i <= 10);
    }
}
```

Output:

```
2
4
6
8
10
```

6. Example: Reverse Counting

```
public class ReverseCount {
    public static void main(String[] args) {
        int i = 5;
        do {
            System.out.println(i);
            i--;
        } while (i >= 1);
    }
}
```

Output:

```
5
4
3
2
1
```

7. Infinite `do-while` Loop

```
do {
    // runs forever unless you use 'break' or 'return'
} while (true);
```

8. Nested `do-while` Loops

You can place one `do-while` loop inside another.

Example: Print 3x3 Grid

```
public class NestedDoWhile {
    public static void main(String[] args) {
        int i = 1;
        do {
            int j = 1;
            do {
                System.out.print("(" + i + "," + j + ") ");
                j++;
            } while (j <= 3);
            System.out.println();
            i++;
        } while (i <= 3);
    }
}
```

Output:

```
(1,1) (1,2) (1,3)
(2,1) (2,2) (2,3)
(3,1) (3,2) (3,3)
```

9. When to Use `do-while` Loop

| Use Case | Loop Type |

|---|---|
Must execute at least once	`do-while` loop
Known number of iterations	`for` loop
Condition-based, may skip execution	`while` loop

Conclusion

- `do-while` is useful when the loop must **run at least once**.
- Condition is **checked after** the loop body.
- Use it for **menus, retry logic**, or input loops where at least one user interaction is needed.

Jump Statements in Java

Java provides **jump statements** to control the flow of execution explicitly. These are used to **exit loops, skip iterations**, or **return from methods**.

1. `break` Statement

The `break` statement is used to **exit a loop or switch** statement **immediately**.

Syntax:

```
break;
```

How It Works

- When `break` is encountered, control jumps **outside the loop or switch**.
- Commonly used to **terminate loops early** based on a condition.

Example: Break on Number 3

```
public class BreakExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            if (i == 3) {
                break;
            }
            System.out.println(i);
        }
    }
}
```

Output:

```
1
2
```

2. `continue` Statement

The `continue` statement is used to **skip the current iteration** and **move to the next** one.

Syntax:

```
continue;
```

How It Works

- Skips the remaining code in the current loop iteration.
- Loop proceeds to the **next cycle**.

Example: Skip Number 3

```
public class ContinueExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            if (i == 3) {
                continue;
            }
            System.out.println(i);
        }
    }
}
```

Output:

```
1
2
4
5
```

3. `return` Statement

The `return` statement is used to **exit from a method**. It can also **return a value** if the method is not `void`.

Syntax (no value):

```
return;
```

Syntax (with value):

```
return value;
```

How It Works

- Ends method execution.
- Optionally sends a value back to the calling method.

Example 1: Return from `void` Method

```
public class ReturnVoid {
    public static void greet() {
        System.out.println("Hello!");
        return;
        // Any code here will not be executed
    }

    public static void main(String[] args) {
        greet();
    }
}
```

Output:

```
Hello!
```

Example 2: Return Value from Method

```
public class ReturnValue {
    public static int square(int num) {
        return num * num;
    }

    public static void main(String[] args) {
        int result = square(4);
        System.out.println("Square: " + result);
    }
}
```

Output:

```
Square: 16
```

4. Summary Table

Statement	Purpose	Common Usage
`break`	Exits loop or switch	Stop loop when condition met
`continue`	Skips current iteration and goes to next	Skip specific condition in loop
`return`	Exits method, optionally returns a value	End method, return result

Conclusion

- Use **`break`** to stop loops early.
- Use **`continue`** to skip iterations.
- Use **`return`** to exit methods and optionally send values back.

CHAPTER 20

java packages, access modifiers , importing

Java Packages

A **package** is a namespace that organizes Java classes and interfaces. It helps group **related classes** and provides **modular structure**, just like folders in a file system.

1. Types of Packages

Type	Description	Example
Built-in	Provided by Java SDK	`java.util`, `java.io`, `java.lang`
User-defined	Created by developers to structure their own applications	`com.myapp.services`
Third-party	Provided by external libraries/frameworks (via JAR files or Maven)	`org.apache.commons.lang3`

2. Built-in Package Example

```
import java.util.Scanner;

public class InputExample {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        String name = sc.nextLine();
        System.out.println("Hello " + name);
    }
}
```

3. User-defined Package Example

Structure:

```
com/
 └── myapp/
```

```
      └── Greet.java
Main.java
```

File: Greet.java

```
package com.myapp;

public class Greet {
    public void sayHello() {
        System.out.println("Hello from user-defined package!");
    }
}
```

File: Main.java

```
import com.myapp.Greet;

public class Main {
    public static void main(String[] args) {
        Greet g = new Greet();
        g.sayHello();
    }
}
```

4. Third-party Package Example

Add to your project using Maven or JAR:

```
<!-- Apache Commons Lang -->
<dependency>
  <groupId>org.apache.commons</groupId>
  <artifactId>commons-lang3</artifactId>
  <version>3.12.0</version>
</dependency>
```

Example usage:

```
import org.apache.commons.lang3.StringUtils;

public class Test {
    public static void main(String[] args) {
        String text = "  hello  ";
        System.out.println(StringUtils.capitalize(text.trim()));
    }
}
```

Ways to Import Packages in Java

There are **4 ways** to use/import classes and packages:

1. Single Class Import

```
import java.util.Scanner;
```

Imports **only** the `Scanner` class.

2. On-Demand (Wildcard) Import

```
import java.util.*;
```

Imports **all classes** in `java.util`, but not sub-packages.

3. Fully Qualified Name (No import statement)

```
public class Example {
    public static void main(String[] args) {
        java.util.Scanner sc = new java.util.Scanner(System.in);
        System.out.println("Enter age: ");
        int age = sc.nextInt();
    }
}
```

Directly uses class with package name.

4. Static Import (for static members)

```
import static java.lang.Math.*;

public class StaticImport {
    public static void main(String[] args) {
        System.out.println(sqrt(16));   // no need for Math.sqrt
        System.out.println(PI);         // no need for Math.PI
    }
}
```

Access Modifiers in Java

Access modifiers **control the visibility** (scope) of classes, variables, methods, and constructors.

Types of Access Modifiers

Modifier	Same Class	Same Package	Subclass (other package)	Other Packages
`public`				
`protected`				
default				
`private`				

1. `public`

- Can be accessed **from anywhere**.

```
public class MyClass {
    public void show() {
        System.out.println("Public method");
    }
}
```

2. `private`

- Access restricted to the **same class only**.

```
public class MyClass {
    private int number = 10;

    private void display() {
        System.out.println(number);
    }
}
```

3. `protected`

- Accessible in:
  - Same package
  - Subclasses (even in other packages)

```
protected void message() {
    System.out.println("Protected method");
}
```

4. Default (no keyword)

- Accessible **within the same package only**.

```
void greet() {
    System.out.println("Default method");
}
```

Where You Can Use Access Modifiers

Element	public	protected	default	private
Class				
Variable				
Method				
Constructor				

Conclusion

- Use **packages** for better organization and reuse.
- Use **import statements** smartly (based on need).
- Use **access modifiers** to **encapsulate data** and **control access**.
- Combine them wisely to write **modular**, **secure**, and **scalable** Java code.

CHAPTER 21

Java Constructors

Java Constructors

1. Introduction to Constructors

A **constructor** is a special method-like block that runs **automatically when an object is created**. It is used to **initialize fields** and prepare the object for use.

2. Constructor Syntax

```
class ClassName {
    ClassName() {
        // initialization code
    }
}
```

Key Characteristics:

- Same name as the class.
- No return type (not even `void`).
- Can be overloaded.

3. How Constructors Work (Behind the Scenes)

When you create an object like:

```
MyClass obj = new MyClass();
```

The process involves:

1. Memory allocation for the object.
2. Constructor is automatically called.
3. Fields are initialized.
4. Reference is assigned to the variable `obj`.

4. Types of Constructors

4.1 Default Constructor (Compiler-Generated)

Provided automatically by Java when no constructor is defined.

```
class Book {
    int pages;
    String title;
}
```

```
Book b = new Book();  // Default constructor is used
```

4.2 No-Argument Constructor (User-Defined)

You can define a constructor without parameters.

```
class Book {
    Book() {
        System.out.println("Book created");
    }
}
```

4.3 Parameterized Constructor

Used to assign values during object creation.

```
class Book {
    String title;
    int pages;

    Book(String t, int p) {
        title = t;
        pages = p;
    }
}
```

```
Book b = new Book("Java", 500);
```

4.4 Constructor Overloading

Multiple constructors with different parameter types or counts.

```
class Student {
    String name;
    int age;

    Student() {
        name = "Default";
        age = 18;
    }

    Student(String n, int a) {
        name = n;
        age = a;
    }
}
```

```
Student s1 = new Student();              // Calls no-arg constructor
Student s2 = new Student("Alice", 22);   // Calls parameterized constructor
```

5. Constructor Rules

- The **constructor name must match the class name**.
- Constructors **do not have a return type**.
- A class can have **multiple constructors** (overloading is allowed).
- If no constructor is defined, Java provides a **default constructor**.
- If any constructor is defined, the default constructor is **not added automatically**.
- Constructors can use **access modifiers** (`public`, `private`, `protected`, or default).
- Constructors **cannot be `final`, `abstract`, or `static`**.
- A constructor **cannot be inherited**, but a subclass can call the superclass constructor using `super()`.
- Constructors can call other constructors in the same class using `this()`, but this call must be **the first line**.

6. The `this` Keyword in Constructors

`this` refers to the **current object**.

Usage:

6.1 Referring to Current Object's Fields

```
class Book {
    String title;

    Book(String title) {
        this.title = title;  // differentiates field and parameter
    }
}
```

6.2 Calling Another Constructor in the Same Class

```
class Box {
    int length, width;

    Box() {
        this(10, 20);  // Calls the parameterized constructor
    }

    Box(int l, int w) {
        this.length = l;
        this.width = w;
    }
}
```

> *Note: `this()` must be the first statement in the constructor.*

7. Using Getters and Setters with Constructors

Getters and setters are used to safely access and modify private fields of a class.

```
class Book {
    private String title;
    private int pages;

    Book(String title, int pages) {
        this.title = title;
        this.pages = pages;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
```

```
    }

    public int getPages() {
        return pages;
    }

    public void setPages(int pages) {
        this.pages = pages;
    }
}
```

8. Summary Table – Choosing a Constructor

Situation	Constructor Type
Create a simple, empty object	Default constructor
Set up object values at creation	Parameterized constructor
Provide multiple ways to create object	Overloaded constructors
Control and restrict object creation	Private constructor (Singleton)

9. Important Notes

- Constructors are not inherited but can be accessed using `super()`.
- Use `this()` to reduce code duplication in overloaded constructors.
- Always use `private` fields and access them via getters and setters to follow encapsulation.

CHAPTER 22

Encapsulation

Encapsulation in Java

What is Encapsulation?

- **Encapsulation** is a fundamental concept of **Object-Oriented Programming (OOP)**.
- In simple words, **Encapsulation** means:

Binding (combining) the data (variables) and methods (functions) together into a single unit.

- In Java, **Encapsulation** is mainly achieved by:

1. **Making variables `private`**.
2. **Providing `public` getter and setter methods** to access and update them.

Real-world meaning:

Think of a **capsule** (medicine) — it wraps medicine inside a cover to protect it. Similarly, in programming, Encapsulation **hides** the internal details and **protects** the data.

Why Use Encapsulation?

Reason	Meaning
Data hiding	Private variables cannot be accessed directly outside the class.
Controlled access	Only through getter/setter methods.
Increased security	Sensitive data is protected.
Better code maintainability	Easy to change internal code without affecting other parts.
Reusability	Once a class is encapsulated properly, you can reuse it confidently.

How to Achieve Encapsulation?

Here's the **standard rule**:

1. Declare variables as **`private`**.
2. Provide **`public` getter** methods to **read** the data.
3. Provide **`public` setter** methods to **modify** the data.

Example of Encapsulation

Full Example Code:

```
// Employee.java
public class Employee {
    // 1. Private Data Members
    private int id;
    private String name;
    private double salary;

    // 2. Public Getter for 'id'
    public int getId() {
        return id;
    }

    // 3. Public Setter for 'id'
    public void setId(int id) {
        this.id = id;
    }

    // 4. Getter for 'name'
    public String getName() {
        return name;
    }

    // 5. Setter for 'name'
    public void setName(String name) {
        this.name = name;
    }

    // 6. Getter for 'salary'
    public double getSalary() {
        return salary;
    }

    // 7. Setter for 'salary'
    public void setSalary(double salary) {
        if (salary > 0) {  // basic validation
            this.salary = salary;
        }
    }
}
```

Another Class to Use It:

```
public class TestEmployee {
    public static void main(String[] args) {
```

```
        Employee emp = new Employee();

        emp.setId(101);
        emp.setName("John Doe");
        emp.setSalary(50000);

        System.out.println("Employee ID: " + emp.getId());
        System.out.println("Employee Name: " + emp.getName());
        System.out.println("Employee Salary: " + emp.getSalary());
    }
}
```

Key Points to Notice

- Outside classes **cannot** access `id`, `name`, and `salary` directly.
- They **must** use `getId()`, `getName()`, and `getSalary()` to read.
- They **must** use `setId()`, `setName()`, and `setSalary()` to update.
- **Validation** (e.g., salary must be > 0) can be done inside **setter methods**.

Real Life Analogy

Imagine **ATM Machine**:

- You **cannot** directly access bank database.
- You **request** money through an ATM interface (input PIN, select amount).
- ATM **validates** your PIN and **then** allows access.

Same way in Java:

- **Variables (data)** are private (bank database).
- **Methods (functions)** provide controlled access (ATM machine).

Extra Tip: Encapsulation + Data Validation

You can also **add checks** inside setters to improve data safety!

Example:

```
public void setAge(int age) {
    if (age > 0 && age < 150) {
        this.age = age;
```

```
    } else {
        System.out.println("Invalid age!");
    }
}
```

Helps to protect the object from incorrect values.

Summary

Topic	Meaning
Encapsulation	Wrapping variables and methods into one unit + Hiding data from direct access.
How	Private variables + Public getters/setters.
Benefit	Data security, flexibility, maintainability, validation, reusability.

Simple Formula

```
Encapsulation = Data Hiding + Controlled Access
```

CHAPTER 23

Inheritance

Inheritance in Java

1. What is Inheritance?

- Inheritance is one of the core concepts of **Object-Oriented Programming (OOP)**.
- In Java, **Inheritance** means **one class (child class) acquiring the properties (fields) and behaviors (methods) of another class (parent class)**.
- It helps in **code reusability** and **method overriding**.

2. Syntax of Inheritance

```
class Parent {
    // fields and methods
}

class Child extends Parent {
    // fields and methods
}
```

Here, `Child` class **inherits** from `Parent` class using the **extends** keyword.

3. Types of Inheritance in Java

Type	Description
Single Inheritance	One class inherits from one other class
Multilevel Inheritance	Class inherits from a child class (chain-like)
Hierarchical Inheritance	Multiple classes inherit from a single parent
Multiple Inheritance	One class inherits from multiple classes (Not supported via classes)
Hybrid Inheritance	Combination of two or more types (Handled using interfaces)

4. Single Inheritance

One child class inherits from one parent class.
One child class inherits from one parent class.
![single level](https://www.trytoprogram.com/images/single-inheritance.jpg "single level")

```
class Animal {
    void eat() {
        System.out.println("Animal eats food");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
```

`Dog` inherits `eat()` method from `Animal`.

5. Multilevel Inheritance

One class inherits from a class which itself is inherited from another class.

![multi level](https://www.trytoprogram.com/images/multi-level-inheritance.jpg "multi level")

```
class Animal {
    void eat() {
        System.out.println("Animal eats food");
    }
}

class Mammal extends Animal {
    void walk() {
        System.out.println("Mammal walks");
    }
}

class Dog extends Mammal {
    void bark() {
        System.out.println("Dog barks");
    }
}
```

`Dog` indirectly inherits `eat()` and `walk()` methods.

6. Hierarchical Inheritance

Multiple child classes inherit from one parent class.
![heirarchical-inheritance](https://www.trytoprogram.com/images/heirarchical-inheritance.jpg "heirarchical-inheritance.")

```
class Animal {
    void eat() {
        System.out.println("Animal eats food");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Cat meows");
    }
}
```

Both `Dog` and `Cat` inherit `eat()` from `Animal`.

7. Multiple Inheritance (Classes)

- Java **does not support multiple inheritance with classes** to **avoid ambiguity problems** (known as the **Di amond Problem**).
- **Multiple Inheritance** means a class trying to inherit from **more than one class**.

Example that is **NOT allowed** in Java:

```
class A {
    void show() {
        System.out.println("A's show");
    }
}

class B {
    void show() {
        System.out.println("B's show");
    }
}
```

```
// This is NOT allowed
class C extends A, B {   // Error: Class cannot extend multiple classes
    // Ambiguity: Which show() to inherit?
}
```

Result:

- Java compiler will throw an error:

`Class C cannot extend multiple classes`

Diamond Problem in Java

What is the Diamond Problem?

- It happens when **one class inherits from two classes that have a common superclass**.
- The **problem is ambiguity**:

If two parent classes have the **same method**, and a child class inherits both,
then **which method should the child inherit**?

Classic Diamond Shape Diagram:

```
     A
    / \
   B   C
    \ /
     D
```

Where:

- Class B and Class C inherit from A
- Class D tries to inherit from both B and C

Problem: If A, B, and C all have a method `display()`, and D inherits from both B and C,
then which `display()` should D inherit? B's or C's?
→ **AMBIGUITY!**

Why Java does not allow Multiple Inheritance using Classes?

- **To avoid ambiguity** at runtime and compile time.
- **Simplify the design** — no confusion about which method to inherit.

- **Prevent complexity** — keeps Java simple and easy to understand.

Interfaces and Multiple Inheritance

Java solves this problem using **interfaces**.

In **interfaces**, even if two interfaces have the same method signature, the implementing class **must override** the method and provide its own definition — so **no ambiguity**.

Final Conclusion:

Situation	Java Behavior
Multiple Inheritance using Classes	Not allowed (to prevent Diamond Problem)
Multiple Inheritance using Interfaces	Allowed (child must override methods)

Important Keywords:

Keyword	Meaning
`extends`	Used for class inheritance
`implements`	Used for interface inheritance
`super`	To call parent class constructor/method
`this`	Current object reference

Quick Summary (Important Points)

- Java supports **single, multilevel, and hierarchical** inheritance via classes.
- Java **does not** support **multiple inheritance via classes** to avoid **Diamond Problem**.
- Java **supports** multiple inheritance **via interfaces**.
- **Private** members of the parent are **not inherited**.
- **Constructors** are **not inherited**, but the child can call them using `super()`.

CHAPTER 24

Constructor Chaining

Object Creation, Type Casting, Object Class & Constructor Chaining

1. Object Creation in Java (Child and Parent Class)

- **Can I create object of child class? → Yes**
- **Can I create object of parent class? → Yes**

Object Reference Scenarios:

Case	Syntax	Possible?	Access
1	`Child refVar = new Child();`	Yes	Child + Parent properties
2	`Parent refVar = new Parent();`	Yes	Parent properties only
3	`Parent refVar = new Child();`	Yes (**Upcasting**)	Parent properties only, overridden methods from Child
4	`Child refVar = new Parent();`	No	Compile-time error

Important Points:

- When **upcasting** (`Parent ref = new Child();`), only **parent properties** are accessible.
- If **overridden methods** are called through parent reference, **child's version** is executed (Runtime Polymorphism).
- **Direct Downcasting** (`Child ref = new Parent();`) is **NOT allowed** and causes:

```
java.lang.ClassCastException: class Parent cannot be cast to class Child
```

2. Type Casting in Java (Upcasting and Downcasting)

Upcasting

- **Definition:** Converting child class reference to parent class type.
- **Type:** Implicit (Automatic) — **Widening**
- **Example:**

```
Parent p = new Child();
```

Downcasting

- **Definition:** Converting parent class reference back to child class type.
- **Type:** Explicit (Manual) — **Narrowing**
- **Example:**

```
Parent p = new Child();  // Upcasting first
Child c = (Child) p;     // Then Downcasting
```

Note:
You **must** perform upcasting first before downcasting.
Direct downcasting is **not possible** and results in **ClassCastException**.

Type	Casting	Safe?	Example
Upcasting	Implicit	Safe	`Parent p = new Child();`
Downcasting	Explicit	Risky (Exception if wrong)	`Child c = (Child) p;`

---

3. Object Class in Java (Super Most Class)

- In Java, **`Object`** is the **root class** for all other classes.
- Every Java class is **implicitly inherited** from `Object` class if no other parent is specified.
- `Object` class contains **11 predefined methods**

Methods of `Object` Class:

1. **`toString()`**:
   - Converts the object to a string representation.
   - Can be overridden to provide meaningful information.
2. **`hashCode()`**:
   - Returns a unique integer hash code for the object.
   - Used in collections like `HashMap` for efficient retrieval.

3. **`equals(Object obj)`**:
   - Compares the current object with the provided one for equality.
   - Can be overridden to define custom equality checks.
4. **`clone()`**:
   - Creates a copy of the current object.
   - Must implement the `Cloneable` interface for cloning to work.
5. **`finalize()`**:
   - Called just before the object is garbage collected.
   - Used for cleanup, though its use is generally discouraged.
6. **`getClass()`**:
   - Returns the `Class` object associated with the object.
   - Useful for reflection.
7. **`notify()`**:
   - Wakes up a single thread that is waiting on this object's monitor.
8. **`notifyAll()`**:
   - Wakes up all threads that are waiting on this object's monitor.
9. **`wait()`**:
   - Causes the current thread to wait until another thread invokes `notify()` or `notifyAll()`.
10. **`wait(long timeout)`**:
    - Causes the current thread to wait for the specified amount of time or until notified.
11. **`wait(long timeout, int nanos)`**:
    - Similar to `wait(long timeout)`, but allows more precise waiting time using nanoseconds.

---

4. Constructor Chaining in Java

What is Constructor Chaining?

- **Constructor Chaining** means calling **one constructor from another constructor** in the same class or the parent class.
- Helps **re-use constructor logic** and **reduce duplicate code**.

Types of Constructor Chaining:

(i) Within Same Class → Using `this()`

```
class Example {
    Example() {
        this(10);  // Calling parameterized constructor
        System.out.println("Default Constructor");
    }

    Example(int x) {
        System.out.println("Parameterized Constructor with value: " + x);
    }
}
```

Output:

```
Parameterized Constructor with value: 10
Default Constructor
```

(ii) From Parent Class → Using `super()`

```
class Parent {
    Parent() {
        System.out.println("Parent Constructor");
    }
}

class Child extends Parent {
    Child() {
        super();  // Calls Parent Constructor
        System.out.println("Child Constructor");
    }
}
```

Output:

```
Parent Constructor
Child Constructor
```

Important Rules in Constructor Chaining:

- `this()` and `super()` must be the **first statement** inside the constructor.
- **Only one** (`this()` or `super()`) can be used in a constructor, not both.
- If no `super()` is explicitly called, Java automatically calls **superclass's default constructor**.
- Constructor chaining helps achieve **better code reuse** and **proper initialization**.

Final Quick Summary

Topic	Key Point
Object Creation	Possible for both child and parent
Upcasting	Implicit, Safe
Downcasting	Explicit, Risky
Object Class	Supermost class in Java
Constructor Chaining	Achieved via `this()` and `super()`

CHAPTER 25

this() vs super()

`this()` vs `super()` in Java:

Feature	**`this()`**	**`super()`**
Purpose	Refers to the current instance of the class.	Refers to the parent class (superclass) of the current class.
Usage in Constructors	Used to call another constructor in the same class.	Used to call a constructor in the superclass.
Position in Constructor	Must be the **first statement** in the constructor.	Must be the **first statement** in the constructor.
Class	Can only be used within the **same class**.	Can only be used in a **subclass** to call a parent class constructor.
Behavior	If used, it calls another constructor in the same class, either default or parameterized.	If used, it calls the parent class constructor, and if no constructor is explicitly called, the parent class's default constructor is called automatically.
Access to Parent Class	Cannot directly access parent class methods or fields.	Used to access methods or constructors from the parent class.
Constructor Chaining	Used for chaining constructors within the same class.	Used for chaining constructors from the parent class.
Multiple Calls	Can only call one constructor of the same class (`this()`), but you can pass different parameters to it.	Can only call one constructor of the parent class (`super()`), but you can call different parent constructors with various parameters.

Key Differences Between `this()` and `super()`

1. **`this()`** is used to call a constructor of the **same class**, while **`super()`** is used to call a constructor of the **parent class**.
2. **`this()`** is used for **constructor chaining within the current class**, whereas **`super()`** is used for **constructor chaining to the parent class**.
3. Both **`this()`** and **`super()`** must be the **first statement** inside the constructor, meaning you can't have any other code before them.
4. **`this()`** calls a constructor with the same name (from the current class), while **`super()`** calls the constructor of the **parent class**.

Example:

```
class Parent {
    Parent() {
        System.out.println("Parent Constructor");
    }

    Parent(String msg) {
        System.out.println("Parent Constructor with message: " + msg);
    }
}

class Child extends Parent {
    Child() {
        // Calling Parent's default constructor
        super();
        System.out.println("Child Constructor");
    }

    Child(String msg) {
        // Calling Parent's parameterized constructor
        super(msg);
        System.out.println("Child Constructor with message: " + msg);
    }
}

class Demo {
    public static void main(String[] args) {
        System.out.println("Creating Child object with default constructor:");
        Child c1 = new Child();

        System.out.println("\nCreating Child object with parameterized constructor:");
        Child c2 = new Child("Hello from Child");
    }
}
```

Output:

```
Creating Child object with default constructor:
Parent Constructor
Child Constructor

Creating Child object with parameterized constructor:
Parent Constructor with message: Hello from Child
Child Constructor with message: Hello from Child
```

In this example:

- **`super()`** is used to invoke the parent class's constructor, either the default or parameterized.
- **`this()`** could have been used in the **Child class** to call a different constructor within the **Child class** itself (though we didn't use it here).

When to Use `this()` vs `super()`?

- Use **`this()`** when you need to call another constructor of the **current class** (usually for constructor chaining within the same class).
- Use **`super()`** when you need to call a constructor from the **parent class** (usually for constructor chaining from the parent class).

Has-A Relationship in Java (Composition vs Aggregation)

In Java, a **Has-A relationship** means that **one class contains a reference to another class** as a field or member variable. It's used to achieve **code reusability** through object composition.

1. **Composition**

Definition:

> *Composition is a strong **Has-A relationship** where the containing class **owns** the object it contains.*
>
> *If the container is destroyed, so are the contained objects.*

Example: `Person` has-a `Heart`

```
class Heart {
    void beat() {
        System.out.println("Heart is beating...");
    }
}

class Person {
    private Heart heart = new Heart(); // Strong Has-A

    void live() {
        heart.beat();
        System.out.println("Person is alive.");
    }
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person();
        p.live();
    }
}
```

Key Points:

- Object (`Heart`) is **created inside** the `Person` class.
- Tight coupling.
- If `Person` is deleted, the `Heart` also gets deleted.

2. **Aggregation**

Definition:

> *Aggregation is a **weaker Has-A relationship**, where one class **uses** or **refers** to another class, but **does not own** it.*

> *The contained object can exist independently.*

Example: `School` has-a `Student`

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }

    void show() {
        System.out.println("Student: " + name);
    }
}

class School {
    Student student; // Weak Has-A

    School(Student student) {
        this.student = student;
    }

    void showStudent() {
        student.show();
    }
}

public class Main {
    public static void main(String[] args) {
        Student s = new Student("Rahul");
        School sch = new School(s);
        sch.showStudent();
    }
}
```

Key Points:

- Object (`Student`) is **passed from outside**.
- Loose coupling.
- If `School` is destroyed, `Student` can still exist independently.

Comparison Table

Feature	Composition	Aggregation
Type	Strong Has-A	Weak Has-A
Object creation	Inside the class	Outside and passed in
Lifespan	Dependent	Independent
Coupling	Tight	Loose
Example	Person has-a Heart	School has-a Student

CHAPTER 27

Wrapper Classes in Java

Wrapper Classes in Java

What are Wrapper Classes?

In Java, **wrapper classes** are used to **wrap primitive data types** into **objects**.
Each primitive type has a corresponding wrapper class in the `java.lang` package.

Why Use Wrapper Classes?

1. **Collections API**

Java **collections (like ArrayList, HashMap)** work only with objects — not with primitive types.
Wrapper classes allow primitives to be used in collections.

2. **Object Manipulation**

In some cases (like method arguments), objects are required. Wrapper classes make it possible to **treat primitives as objects**.

3. **Utility Methods**

Wrapper classes provide useful **methods** for parsing, comparison, and conversion.

Primitive Types and Their Wrapper Classes

Primitive Type	Wrapper Class
`byte`	`Byte`
`short`	`Short`
`int`	`Integer`
`long`	`Long`
`float`	`Float`
`double`	`Double`

`char`	`Character`
`boolean`	`Boolean`

Manual Boxing and Unboxing (Before Java 5)

Manual Boxing:

> *Converting primitive → wrapper **explicitly***

```
int a = 10;
Integer obj = new Integer(a); // Manual boxing (Deprecated but valid)
```

Manual Unboxing:

> *Converting wrapper → primitive **explicitly***

```
Integer obj = new Integer(20);
int b = obj.intValue(); // Manual unboxing
```

Autoboxing and Unboxing (Java 5 Onward)

Java provides **automatic conversion** between primitives and wrapper objects.

Autoboxing:

> *Java **automatically converts** a primitive into its corresponding wrapper object.*

```
int x = 30;
Integer obj = x; // Autoboxing
```

Unboxing:

> *Java **automatically converts** a wrapper object into its corresponding primitive.*

```
Integer obj = 40;
int y = obj; // Unboxing
```

Example: Using Wrapper Classes

```
public class WrapperExample {
    public static void main(String[] args) {
        int a = 5;

        // Autoboxing
        Integer obj = a;

        // Unboxing
        int b = obj;

        System.out.println("Primitive: " + a);
        System.out.println("Object: " + obj);
        System.out.println("Unboxed: " + b);
    }
}
```

Common Methods in Wrapper Classes

`parseXXX(String s)`

Converts a **String to primitive**.

```
int num = Integer.parseInt("123");
double d = Double.parseDouble("45.6");
```

`valueOf(String s)`

Converts a **String to Wrapper**.

```
Integer x = Integer.valueOf("100");
Boolean y = Boolean.valueOf("true");
```

`xxxValue()`

Converts a **wrapper to another primitive type**.

```
Integer i = 10;
double d = i.doubleValue();  // 10.0
```

Notes:

- All wrapper classes are **immutable** (cannot change state).
- Wrapper classes implement **Comparable** and are **final**.
- Can be **null**, unlike primitives — be careful when using in logic.
- Wrapper classes wrap primitive types in objects.
- Needed for working with collections, object methods, and utilities.
- Java supports **autoboxing/unboxing** from Java 5 onward.
- They belong to java.lang package.
- Avoid using new (manual boxing) from Java 9 onward — it is deprecated.
- Provides useful static methods for parsing and conversion.

What Is Caching in Wrapper Classes?

In Java, some wrapper classes **cache object instances** of certain values **to improve performance and memory efficiency**.

This means:

- For specific value ranges (like -128 to 127), Java **reuses the same object** instead of creating a new one every time.

Which Wrapper Classes Use Caching?

Wrapper Class	Cached Range
`Byte`	-128 to 127
`Short`	-128 to 127
`Integer`	-128 to 127
`Long`	-128 to 127
`Character`	0 to 127
`Boolean`	true / false
`Float` / `Double`	No caching

Example: Integer Caching

```
public class CacheDemo {
    public static void main(String[] args) {
        Integer a = 100;
        Integer b = 100;

        Integer x = 200;
        Integer y = 200;

        System.out.println(a == b); // true (within cache range)
        System.out.println(x == y); // false (outside cache range)
    }
}
```

Explanation:

- `a` and `b` refer to the **same object** (cached), so `a == b` is `true`.
- `x` and `y` are **different objects** (outside cache range), so `x == y` is `false`.

How It Works Internally

For example, in `Integer.valueOf(int i)`:

```
public static Integer valueOf(int i) {
    if (i >= -128 && i <= 127)
        return IntegerCache.cache[i + 128];
    else
        return new Integer(i);
}
```

> *`IntegerCache` is an internal static class that stores cached Integer objects for values between -128 and 127.*

Why Is This Important?

- Reduces memory usage.
- Improves performance when working with common values.
- Be cautious when using `==` for wrapper comparison — use `.equals()` for reliable value comparison.

Best Practice

```
Integer a = 100;
Integer b = 100;

System.out.println(a.equals(b));  //  Always safe for value comparison
```

Summary

- Wrapper classes like `Integer`, `Short`, `Long`, etc., **cache commonly used values**.
- Caching helps in **performance optimization**.
- `==` compares object references — so be careful outside the cached range.
- Always use `.equals()` to compare wrapper class **values**.

CHAPTER 28

Polymorphism in Java

Polymorphism in Java

What is Polymorphism?

Polymorphism = "Many Forms"

In Java, **polymorphism** allows objects to be treated in multiple forms, enabling the same method name to perform different tasks based on the object or parameters involved.

Types of Polymorphism in Java

Type	When Resolved	Achieved By	Also Known As
Compile-time Polymorphism	At Compile Time	Method Overloading	Static / Early Binding
Runtime Polymorphism	At Runtime	Method Overriding	Dynamic / Late Binding

1. **Method Overloading** – *Compile-Time Polymorphism*

Definition:

Method Overloading allows multiple methods with the **same name** but **different parameters** within the **same class**.

Rules for Method Overloading

1. **Changed Parameter Count**

```
void show() {}
void show(int a) {}  // Valid
```

2. **Changed Parameter Type**

```
void show(int a) {}
void show(double b) {}  // Valid
```

3. **Changed Parameter Sequence**

```
void show(int a, String b) {}
void show(String b, int a) {}  // Valid
```

4. **Return Type Alone is Not Enough**

```
int get() { return 1; }
// double get() { return 1.0; }  // Invalid – causes ambiguity
```

5. **Changing Access Modifier Only is Not Allowed**

```
public void print() {}
// private void print() {}  // Invalid – same signature
```

Overloading with Wrapper Classes

Java supports **primitives** and their **wrapper classes** (`int` → `Integer`, etc.)

Key Concepts:

Concept	Description
Autoboxing	Java automatically converts primitives to wrappers
Unboxing	Converts wrapper to primitive
Widening vs Boxing	Java prefers **widening** over **boxing**
Precedence Order	1. Exact Match → 2. Widening → 3. Autoboxing → 4. Varargs

---

Example:

```
void display(int x) { System.out.println("int"); }
void display(Integer x) { System.out.println("Integer"); }
```

```
display(5);  // Output: int (exact match preferred)
```

2. **Method Overriding** – *Runtime Polymorphism*

Definition:

Method Overriding is when a **subclass** provides its own version of a **method already defined in the superclass**.

Rules for Method Overriding

1. **Same Method Signature**

- Name, parameters, and return type must match
- Return type can be **covariant** (subclass type)

2. **Access Modifier Must Be Same or Less Restrictive**

Modifier Hierarchy	public > protected > default > private

```
// Superclass
protected void greet() {}

// Subclass
public void greet() {}  //  More accessible – valid
```

```
// Superclass
public void greet() {}

// Subclass
protected void greet() {}  //  Less accessible – invalid
```

3. **Private Methods Cannot Be Overridden**

```
class A {
```

```
    private void secret() {}
}

class B extends A {
    void secret() {}  // This is NOT overriding – it’s a new method
}
```

Covariant Return Types

A subclass can override a method and return a **subtype** of the return type of the overridden method.

```
class Parent {
    Object getObject() { return null; }
}

class Child extends Parent {
    String getObject() { return "Hello"; }  // Valid
}
```

Example of Runtime Polymorphism

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Test {
    public static void main(String[] args) {
        Animal obj = new Dog();  // Reference is Animal, object is Dog
        obj.sound();  // Output: Dog barks (runtime binding)
    }
}
```

Quick Comparison Table

Feature	Method Overloading	Method Overriding
Type of Polymorphism	Compile-time	Runtime
Inheritance Needed	No	Yes
Method Signature	Must differ (params)	Must be same
Return Type	Can differ	Must be same or covariant
Access Modifier	Any	Same or more accessible
Private Methods	Can overload	Cannot override
Purpose	Code clarity, multiple tasks	Runtime behavior flexibility

Why Use Polymorphism?

Benefit	Description
Flexibility	One interface can support different behaviors
Code Reuse	Reduces duplication, improves maintenance
Extensibility	Easily add new behaviors without changing existing code
Loose Coupling	Enhances scalability and modularity

`final` Keyword

In Java, the `final` keyword is a **non-access modifier** used to apply **restrictions** on:

- **Variables**
- **Methods**
- **Classes**

It indicates that **once something is assigned or defined, it cannot be changed or overridden**.

1. `final` Local Variable

A. Initialize at Declaration

```
void testMethod() {
    final int x = 100;
    System.out.println(x);
}
```

B. Initialize Later (Before Use)

```
void testMethod() {
    final int y;
    y = 200;  // Only once
    System.out.println(y);
}
```

Reassigning → Compilation Error

```
void testMethod() {
    final int z = 10;
    z = 20; //  Error: cannot assign a value to final variable 'z'
}
```

2. `final` Instance Variable

A. Initialize at Declaration

```
public class Person {
    final String name = "John";
}
```

B. Initialize in Constructor

```
public class Person {
    final String name;

    Person(String name) {
        this.name = name;
    }
}
```

C. Initialize in Instance Initializer Block

```
public class Person {
    final String name;

    {
        name = "Default Name";
    }
}
```

Multiple Assignments → Error

```
public class Person {
    final String name;

    Person() {
        name = "John";
        name = "Doe"; //  Error: cannot assign a value twice
    }
}
```

3. `final static` Variable

A. Initialize at Declaration

```
public class Config {
    public static final int TIMEOUT = 5000;
}
```

B. Initialize in Static Block

```
public class Config {
    public static final int TIMEOUT;

    static {
        TIMEOUT = 5000;
    }
}
```

Reassignment Later → Error

```
public class Config {
    public static final int TIMEOUT = 3000;

    void changeTimeout() {
        // TIMEOUT = 4000; //  Error
    }
}
```

4. `final` Method

A. Prevent Overriding in Subclass

```
class Animal {
    final void sound() {
        System.out.println("Animal sound");
    }
}
```

```
class Dog extends Animal {
    // void sound() { } //  Error: Cannot override final method
}
```

B. Final Method Can Be Overloaded

```
class Animal {
    final void sound() {
        System.out.println("Animal sound");
    }

    void sound(String type) {
        System.out.println("Animal sound: " + type);
    }
}
```

5. `final` Class

A. Final Class Cannot Be Extended

```
final class Utility {
    static void print() {
        System.out.println("Utility class");
    }
}

// class SubUtility extends Utility {} //  Error
```

B. You Can Use Final Class Normally

```
public class Test {
    public static void main(String[] args) {
        Utility.print(); //  Works
    }
}
```

6. `final` Object Reference

A. Modify Object Contents (Allowed)

```
final List<String> names = new ArrayList<>();
names.add("Alice"); //
names.set(0, "Bob"); //
```

B. Reassign Reference (Not Allowed)

```
final List<String> names = new ArrayList<>();
// names = new ArrayList<>(); //  Error: cannot reassign final reference
```

Java Abstract Class

1. What is an Abstract Class?

An `abstract class` in Java is a **special class** that:

- Cannot be instantiated (no objects can be created directly)
- Can contain:
  - **Abstract methods** (no body)
  - **Concrete methods** (with body)
  - Variables (static/final/instance)
  - Constructors

It's used to **provide partial implementation** and define a **common base for subclasses**.

2. Syntax

```
abstract class ClassName {
    abstract void method1();   // Abstract method
    void method2() {           // Concrete method
        System.out.println("Concrete method");
    }
}
```

3. Key Features

Feature	Abstract Class Support
Instantiation	Not allowed directly
Abstract methods	Optional
Concrete methods	Yes
Constructor	Yes
Fields (variables)	Instance, static, final

| Inheritance | via `extends` |
| Static/final methods | Allowed (not abstract) |

Key Rules for Abstract Classes:

1. Declared using the `abstract` keyword
2. Cannot be instantiated directly (cannot use `new Vehicle()`)
3. May contain both abstract and concrete methods
4. Abstract methods must be declared with the `abstract` keyword and have no body
5. If a class contains even one abstract method, the class must be declared abstract
6. Subclasses must implement all abstract methods or be declared abstract themselves
7. Can have constructors (even though you can't instantiate the class directly)
8. Can have fields, static methods, and instance methods with implementation
9. Can extend another class (abstract or concrete)

4. Design Principle (Important!)

> ***If a class contains only static methods or static fields, declare it as `abstract` to prevent object creation.***

Example:

```
abstract class MathUtils {
    public static final double PI = 3.14;

    public static int square(int n) {
        return n * n;
    }
}
```

> *This ensures the utility class is not instantiated accidentally.*

5. When to Use Abstract Classes?

- When you want to **provide a base class** with **some common implementation**
- When some methods must be **implemented by subclasses**
- When you want to **restrict object creation**

6. Abstract Class vs Interface

Feature	Abstract Class	Interface
Methods	Abstract + Concrete	All abstract (Java 7), default/static (Java 8+)
Variables	Any type	Only `public static final`
Constructor	Yes	No
Multiple Inheritance	No	Yes (Multiple interfaces)
Inheritance keyword	`extends`	`implements`

7. Example: Abstract Class in Action

```
abstract class Animal {
    abstract void makeSound();

    void eat() {
        System.out.println("This animal eats food");
    }
}

class Dog extends Animal {
    void makeSound() {
        System.out.println("Bark");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.makeSound(); // Bark
        a.eat();       // This animal eats food
    }
}
```

8. Abstract Class with Constructor

```
abstract class Vehicle {
    Vehicle() {
        System.out.println("Vehicle created");
    }
```

```
    abstract void run();
}

class Bike extends Vehicle {
    Bike() {
        super(); // Calls Vehicle's constructor
    }

    void run() {
        System.out.println("Bike is running");
    }
}
```

9. Abstract Class with Variables

```
abstract class Shape {
    final double PI = 3.14; // final field
    int side = 5;           // instance variable

    abstract double area();
}
```

Abstraction in Java

1. **What is Abstraction?**

- Abstraction is an OOP principle that **hides complex implementation details** and shows only the **essential features**.
- It focuses on **what an object does**, not **how it does it**.
- It helps manage complexity by **separating interface from implementation**.

---

2. **Why Use Abstraction?**

- To reduce complexity.
- To enhance code maintainability.
- To provide security by hiding sensitive info.
- To promote loose coupling and improve flexibility.
- To enable multiple implementations of the same abstract behavior.

---

3. **How Abstraction is Achieved in Java?**

- Using **Abstract Classes**
- Using **Interfaces**

---

4. **Abstract Class**

- Cannot be instantiated directly.
- Can contain abstract methods (no body) and concrete methods (with body).
- Subclasses must implement all abstract methods or be abstract themselves.

Syntax Example:

```
abstract class Animal {
    abstract void sound(); // abstract method

    void sleep() {
        System.out.println("Sleeping...");
```

```
    }
}
```

5. **Interface**

- Provides 100% abstraction (before Java 8).
- All methods are abstract by default.
- Since Java 8, interfaces can have default and static methods with implementation.
- Supports multiple inheritance.

Syntax Example:

```
interface Animal {
    void sound();  // implicitly public and abstract
}
```

6. **Difference Between Abstract Class and Interface**

Feature	Abstract Class	Interface
Methods	Abstract + concrete methods	Abstract methods (default & static allowed)
Variables	Instance variables allowed	Only public static final constants
Multiple Inheritance	Not supported	Supported
Constructors	Allowed	Not allowed
Usage	For shared base class behavior	For contract specification

7. **Example of Abstract Class Implementation**

```
abstract class Vehicle {
    abstract void start();

    void stop() {
        System.out.println("Vehicle stopped.");
    }
}

class Car extends Vehicle {
    void start() {
        System.out.println("Car started with key.");
    }
```

```
}
```

8. **Example of Interface Implementation**

```
interface Vehicle {
    void start();
    void stop();
}

class Bike implements Vehicle {
    public void start() {
        System.out.println("Bike started with button.");
    }

    public void stop() {
        System.out.println("Bike stopped.");
    }
}
```

9. **Helper Class Pattern for Abstraction**

- Implement all abstract methods in a concrete subclass.
- Create a **helper class** with a **helper method** that returns an instance of the implemented concrete class but typed as the abstract class.
- This **hides the concrete implementation** from the user and promotes loose coupling.

10. **Example: Abstract Class + Helper Class**

```
// Abstract class with abstract method
abstract class Animal {
    abstract void sound();

    void sleep() {
        System.out.println("Sleeping...");
    }
}

// Concrete implementation
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
```

```
    }
}

// Helper class to return abstract type object
class AnimalHelper {
    public static Animal getAnimal() {
        return new Dog();  // returning concrete class instance as abstract type
    }
}

// Testing the abstraction with helper
public class TestAbstractionHelper {
    public static void main(String[] args) {
        Animal myAnimal = AnimalHelper.getAnimal();

        myAnimal.sound();  // Dog barks
        myAnimal.sleep();  // Sleeping...
    }
}
```

11. **Explanation**

- `Animal` is abstract, cannot be instantiated directly.
- `Dog` is a concrete subclass implementing `sound()`.
- `AnimalHelper.getAnimal()` hides the concrete class from client code by returning `Animal` type.
- Client works with the abstract class only, increasing **abstraction and flexibility**.
- You can change `Dog` to another subclass in `AnimalHelper` without changing client code.

12. **Key Points to Remember**

- Abstraction hides implementation and shows only relevant details.
- Abstract classes can have both abstract and concrete methods.
- Interfaces provide full abstraction but can also have default methods (Java 8+).
- Concrete classes implement all abstract methods.
- Helper classes can be used to **return objects of concrete classes typed as abstract classes** to hide implementations.
- This pattern promotes loose coupling and better design.

13. **When to Use Abstraction?**

- When you want to separate interface from implementation.

- When you want to enforce a contract for subclasses.
- When you want to hide internal implementation details.
- When designing frameworks or APIs.
- When different implementations share common behavior.

Interfaces in Java

1. Introduction

An **interface** in Java is a reference type, similar to a class, that is a collection of abstract methods. Interfaces are used to achieve **abstraction** and **multiple inheritance** in Java.

2. Characteristics of Interfaces

- Cannot be instantiated.
- Can only contain abstract methods (Java 7 and earlier).
- All variables declared in interfaces are:
  - `public static final` by default
- Interface methods are:
  - `public abstract` by default (for abstract methods)

3. Syntax

```
interface InterfaceName {
    // Abstract method
    void method1();
}
```

4. Implementing an Interface

```
class MyClass implements InterfaceName {
    public void method1() {
        System.out.println("Implemented method1");
    }
}
```

5. Default and Static Methods (Java 8+)

From Java 8 onward, interfaces can include concrete methods using `default` and `static` keywords.

Default Method

Used when we want to provide a common implementation for all implementing classes.

```
interface InterfaceName {
    default void method2() {
        System.out.println("Default method");
    }
}
```

Static Method

Belongs to the interface, not to the instance.

```
interface InterfaceName {
    static void method3() {
        System.out.println("Static method");
    }
}
```

6. Multiple Interfaces

Java allows a class to implement multiple interfaces:

```
interface A {
    void show();
}

interface B {
    void display();
}

class Demo implements A, B {
    public void show() {
        System.out.println("Showing A");
    }
    public void display() {
        System.out.println("Displaying B");
    }
}
```

7. Interface vs Abstract Class

Feature	Interface	Abstract Class
Methods	Only abstract (Java 7), default & static (Java 8+)	Abstract + concrete methods
Variables	`public static final`	Any type (with/without modifiers)
Constructors	Not allowed	Allowed
Multiple Inheritance	Supported	Not supported
Use Case	Full abstraction	Partial abstraction

8. Real-Life Example

```
interface Remote {
    void turnOn();
    void turnOff();
}

class TV implements Remote {
    public void turnOn() {
        System.out.println("TV ON");
    }
    public void turnOff() {
        System.out.println("TV OFF");
    }
}
```

9. Interface Variables

All variables in interfaces are implicitly:

- `public`
- `static`
- `final`

Example:

```
interface Constants {
    int MAX = 100; // public static final int MAX = 100;
}
```

Cannot be changed:

```
Constants.MAX = 200; // Error: cannot assign a value to final variable
```

10. Functional Interfaces (Java 8)

- Interface with only one abstract method.
- Can be implemented using lambda expressions.

```
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}

Calculator c = (a, b) -> a + b;
System.out.println(c.add(5, 3));
```

11. Private Methods in Interface (Java 9+)

Used to support code reusability inside `default` methods.

```
interface Helper {
    default void methodA() {
        common();
    }

    default void methodB() {
        common();
    }

    private void common() {
        System.out.println("Common Logic");
    }
}
```

12. Example Programs

Program 1: Basic Interface

```
interface Shape {
```

```
    void draw();
}

class Circle implements Shape {
    public void draw() {
        System.out.println("Drawing Circle");
    }
}
```

Program 2: Interface Polymorphism

```
interface Vehicle {
    void start();
}

class Bike implements Vehicle {
    public void start() {
        System.out.println("Bike Started");
    }
}

class Car implements Vehicle {
    public void start() {
        System.out.println("Car Started");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v1 = new Bike();
        Vehicle v2 = new Car();
        v1.start();
        v2.start();
    }
}
```

13. Types of Interfaces

Type	Description	Example
Normal Interface	Has one or more abstract methods.	`Runnable`, `Comparable`
Functional Interface	Has only one abstract method (Java 8+). Can use lambdas.	`Runnable`, `Callable`
Marker Interface	Has no methods or fields. Used as a tag to convey metadata.	`Serializable`, `Cloneable`

Nested Interface	Interface declared inside another interface or class.	`Map.Entry`

14. Summary

- Interfaces are used for abstraction and multiple inheritance.
- From Java 8+, interfaces can contain default and static methods.
- Interfaces support loose coupling and polymorphism.
- Functional interfaces allow concise syntax with lambda expressions.

CHAPTER 33

Java Arrays

Java Arrays

What is an Array?

An **array** in Java is a **container object** that holds a **fixed number of values** of a **single type**.
The length of an array is **established when the array is created** and **cannot be changed**.

Why Use Arrays?

- To store multiple values in a **single variable**.
- To manage **large sets of data** efficiently.
- To access elements via **indexing** (fast random access).
- Helps avoid creating multiple variables (e.g., `int a, b, c, d;` becomes `int[] arr = new int[4];`).

Array Syntax

1. **Declaration**

```
int[] arr;        // preferred
// OR
int arr[];
```

2. **Instantiation**

```
arr = new int[5];  // array of 5 integers
```

3. **Initialization**

```
arr[0] = 10;
arr[1] = 20;
...
```

4. **Declaration + Instantiation + Initialization**

```
int[] arr = new int[5];                // default values (0 for int)
int[] arr2 = {10, 20, 30, 40, 50};     // direct initialization
```

Default Values in Arrays

Data Type	Default Value
int	0
double	0.0
boolean	false
String / Object	null

Accessing Array Elements

```
System.out.println(arr[0]); // prints first element
arr[2] = 99;                // change value at index 2
```

Indexing starts at `0`

Last element is at `array.length - 1`

Traversing Arrays

Using `for` loop:

```
for(int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

Using `for-each` loop:

```
for(int value : arr) {
    System.out.println(value);
}
```

ArrayIndexOutOfBoundsException

Thrown when you try to access an index **outside the bounds** of the array.

```
int[] arr = new int[3];
arr[3] = 10; //  Error: Index 3 is out of bounds for length 3
```

Arrays in Methods

Passing an array to a method:

```
void display(int[] arr) {
    for(int val : arr) {
        System.out.println(val);
    }
}
```

Returning an array from a method:

```
int[] getArray() {
    return new int[]{1, 2, 3, 4, 5};
}
```

Multidimensional Arrays (2D Array)

Declaration & Initialization:

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6}
};
```

Accessing Elements:

```
System.out.println(matrix[0][1]); // Output: 2
```

Traversing:

```
for(int i = 0; i < matrix.length; i++) {
    for(int j = 0; j < matrix[i].length; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
```

Array Class Methods (java.util.Arrays)

Import first:

```
import java.util.Arrays;
```

Method	Description
`Arrays.sort(arr)`	Sort array in ascending order
`Arrays.toString(arr)`	Convert array to string
`Arrays.equals(arr1, arr2)`	Compare arrays
`Arrays.copyOf(arr, newLength)`	Copy array with new size
`Arrays.fill(arr, value)`	Fill all elements with a value

Key Points to Remember

- Arrays are **fixed in size**.
- Store **homogeneous elements** (same data type).
- Indexing starts from `0`.
- Can be of **primitive types** or **objects** (e.g., `String[]`, `int[]`).
- Use **ArrayList** if you need a resizable array.

Sample Program

```
public class ArrayDemo {
    public static void main(String[] args) {
        int[] marks = {85, 90, 78, 92};
```

```
        int sum = 0;
        for(int mark : marks) {
            sum += mark;
        }

        System.out.println("Total: " + sum);
        System.out.println("Average: " + (sum / marks.length));
    }
}
```

CHAPTER 34

Java Inner Classes

Java Inner Classes

In Java, an **inner class** is a class defined **within another class**. It is associated with the **outer class** and can access its members, including private ones.

Why Use Inner Classes?

- To logically group classes that are only used in one place.
- To increase encapsulation.
- To make code more readable and maintainable.

Types of Inner Classes in Java

1. **Non-static (Regular) Inner Class**

- Associated with an instance of the outer class.
- Can access all members of the outer class, including private members.

```
class Outer {
    private String message = "Hello from Outer";

    class Inner {
        void display() {
            System.out.println(message);  // Accessing private field of outer class
        }
    }
}

public class Test {
    public static void main(String[] args) {
        Outer outer = new Outer();
        Outer.Inner inner = outer.new Inner();  // Creating inner class object
        inner.display();
    }
}
```

2. **Static Nested Class**

- Declared with the `static` keyword.
- Can **only access static members** of the outer class.
- Doesn't need an object of the outer class to be instantiated.

```
class Outer {
    static String msg = "Static Nested Example";

    static class Nested {
        void show() {
            System.out.println(msg);
        }
    }
}

public class Test {
    public static void main(String[] args) {
        Outer.Nested obj = new Outer.Nested();
        obj.show();
    }
}
```

3. **Local Inner Class**

- Defined inside a method.
- Can access local variables of the method **only if they are final or effectively final**.

```
class Outer {
    void outerMethod() {
        int number = 10;  // effectively final

        // Local Inner Class
        class LocalInner {
            void print() {
                System.out.println("Number is: " + number);
            }
        }

        // Creating object of local inner class and calling method
        LocalInner inner = new LocalInner();
        inner.print();
    }
}

public class LocalInnerExample {
    public static void main(String[] args) {
```

```
        Outer outer = new Outer();
        outer.outerMethod();
    }
}
```

4. **Anonymous Inner Class**

- Class without a name.
- Used when you need to override methods of a class or interface **on the spot**.

```
abstract class Animal {
    abstract void sound();
}

public class Test {
    public static void main(String[] args) {
        Animal dog = new Animal() {
            void sound() {
                System.out.println("Bark");
            }
        };

        dog.sound();
    }
}
```

Summary Table

Type	Static?	Can Access Outer Class Members	Typical Use Case
Regular Inner Class	No	Yes (including private)	Grouping logic closely tied to outer class
Static Nested Class	Yes	Only static members	Group utility/helper classes
Local Inner Class	No	Yes (if final/effectively final)	Use in methods for local tasks
Anonymous Inner Class	No	Yes	Quick implementation (e.g., for listeners)

CHAPTER 35

Functional Interface in Java

Functional Interface in Java

Definition:

A **Functional Interface** in Java contains **exactly one abstract method**, and can have any number of default or static methods.

> *It's the foundation of **Lambda expressions** and **method references** introduced in Java 8.*

Annotation: `@FunctionalInterface`

```
@FunctionalInterface
interface MyFunctionalInterface {
    void display();  // single abstract method
}
```

Rules of Functional Interface

- One **and only one** abstract method.
- Can include multiple **default** and **static** methods.
- May override methods from `Object` class (like `toString()`).
- Use of `@FunctionalInterface` is **optional but recommended**.

Built-in Functional Interfaces & Abstract Methods

Java provides several built-in functional interfaces in the **`java.util.function`** package.

Interface Name	Abstract Method Signature	Description
Consumer\<T>	`void accept(T t)`	Takes one input, returns nothing
Supplier\<T>	`T get()`	Takes no input, returns a value
Function\<T, R>	`R apply(T t)`	One input, one return
Predicate\<T>	`boolean test(T t)`	One input, returns boolean

BiConsumer\<T, U>	`void accept(T t, U u)`	Two inputs, no return
BiFunction\<T, U, R>\`	`R apply(T t, U u)`	Two inputs, one return
BiPredicate\<T, U>\`	`boolean test(T t, U u)`	Two inputs, returns boolean
UnaryOperator\<T>	`T apply(T t)`	Special form of Function (T to T)
BinaryOperator\<T>	`T apply(T t1, T t2)`	Two inputs of same type, return same

1. Consumer<T>

- **Method:** `void accept(T t)`

```
Consumer<String> printer = s -> System.out.println(s);
printer.accept("Hello");
```

2. Supplier<T>

- **Method:** `T get()`

```
Supplier<Double> random = () -> Math.random();
System.out.println(random.get());
```

3. Function\<T, R>

- **Method:** `R apply(T t)`

```
Function<String, Integer> lengthFinder = s -> s.length();
System.out.println(lengthFinder.apply("Java"));  // 4
```

4. Predicate<T>

- **Method:** `boolean test(T t)`

```
Predicate<Integer> isEven = x -> x % 2 == 0;
System.out.println(isEven.test(8));  // true
```

5. BiConsumer\<T, U>

- **Method:** `void accept(T t, U u)`

```
BiConsumer<String, Integer> printAge = (name, age) ->
    System.out.println(name + " is " + age + " years old");
printAge.accept("Kumar", 25);
```

---

6. BiFunction\<T, U, R>

- **Method:** `R apply(T t, U u)`

```
BiFunction<Integer, Integer, Integer> multiply = (a, b) -> a * b;
System.out.println(multiply.apply(4, 5));  // 20
```

---

7. BiPredicate\<T, U>

- **Method:** `boolean test(T t, U u)`

```
BiPredicate<String, String> equalsIgnoreCase = (s1, s2) -> s1.equalsIgnoreCase(s2);
System.out.println(equalsIgnoreCase.test("Java", "JAVA"));  // true
```

---

8. UnaryOperator<T>

- Extends: `Function<T, T>`
- **Method:** `T apply(T t)`

```
UnaryOperator<Integer> square = n -> n * n;
System.out.println(square.apply(6));  // 36
```

---

9. BinaryOperator<T>

- Extends: `BiFunction<T, T, T>`
- **Method:** `T apply(T t1, T t2)`

```
BinaryOperator<Integer> add = (a, b) -> a + b;
System.out.println(add.apply(10, 20));  // 30
```

Use Cases

- Functional Programming using **Streams**
- Event Handling
- Clean Strategy Pattern Implementation
- Callbacks and Filters in Collections

Custom Functional Interface Example

```
@FunctionalInterface
interface Greeting {
    void sayHello(String name);  // abstract method
}
```

Summary Table

Feature	Allowed in Functional Interface
One Abstract Method	Yes
More than one abstract method	No (Compile-time error)
Default Methods	Yes
Static Methods	Yes
`Object` class methods	Yes
`@FunctionalInterface`	Optional but Recommended

CHAPTER 36

Lambda Expressions in Java

Lambda Expressions in Java

Lambda expressions were introduced in **Java 8** as part of functional programming features. They allow us to write anonymous methods (functions without names), primarily to implement **functional interfaces** more concisely.

---

1. **What is a Lambda Expression?**

A **lambda expression** is a short block of code which takes in parameters and returns a value.
It provides a clear and concise way to represent **one method interface using an expression**.

Syntax:

```
(parameters) -> expression
```

or

```
(parameters) -> { statements; }
```

---

2. **Why Lambda?**

- Removes boilerplate code (no need for anonymous inner classes)
- Enables functional programming
- Makes code **cleaner**, **shorter**, and **more readable**
- Used heavily in **Streams**, **Collections**, **Functional Interfaces**, etc.

---

3. **Functional Interface (Prerequisite)**

A **Functional Interface** is an interface that has only **one abstract method**.

Example:

```
@FunctionalInterface
interface MyInterface {
```

```
    void show();  // only one method
}
```

4. **Basic Lambda Example**

```
@FunctionalInterface
interface Message {
    void sayHello();
}

public class TestLambda {
    public static void main(String[] args) {
        // Using Lambda
        Message msg = () -> System.out.println("Hello from Lambda!");
        msg.sayHello();
    }
}
```

5. **Lambda with Parameters**

```
@FunctionalInterface
interface Add {
    int sum(int a, int b);
}

public class LambdaWithParams {
    public static void main(String[] args) {
        Add ad = (a, b) -> a + b;
        System.out.println("Sum = " + ad.sum(5, 3));
    }
}
```

6. **Lambda with Block Body**

```
Multiply m = (a, b) -> {
    System.out.println("Multiplying " + a + " and " + b);
    return a * b;
};
```

7. **Using Lambda with Threads**

```
public class LambdaThread {
    public static void main(String[] args) {
        Runnable r = () -> {
            for (int i = 1; i <= 5; i++) {
                System.out.println("Child Thread: " + i);
            }
        };

        Thread t = new Thread(r);
        t.start();
    }
}
```

8. **Using Lambda with Collections**

```
import java.util.*;

public class LambdaCollections {
    public static void main(String[] args) {
        List<String> list = Arrays.asList("Banana", "Apple", "Mango");

        // Sorting with lambda
        Collections.sort(list, (s1, s2) -> s1.compareTo(s2));
        list.forEach(s -> System.out.println(s));
    }
}
```

9. **Common Functional Interfaces in `java.util.function`**

Interface	Method Signature	Example Use
`Predicate<T>`	`boolean test(T t)`	Test condition, used in filters
`Function<T,R>`	`R apply(T t)`	Converts from one type to another
`Consumer<T>`	`void accept(T t)`	Takes a value and returns nothing
`Supplier<T>`	`T get()`	Supplies a value, no input

Example: Predicate

```
Predicate<Integer> isEven = x -> x % 2 == 0;
System.out.println(isEven.test(10));  // true
```

10. **Limitations of Lambda**

- Can only be used with **functional interfaces**
- Cannot maintain **state** like traditional classes
- Limited debugging capabilities

11. **Comparison: Before vs Lambda**

Without Lambda (Anonymous Class):

```
Runnable r = new Runnable() {
    public void run() {
        System.out.println("Running...");
    }
};
```

With Lambda:

```
Runnable r = () -> System.out.println("Running...");
```

12. **Real-Life Analogy**

Think of lambda as a **"shortcut"** to write small methods without creating entire classes.

Just like:

- Instead of writing:

`"I will do this later."`

You say: `"Later!"`

Conclusion

Lambda expressions:

- Reduce code size
- Improve readability
- Enable functional-style programming
- Work with **functional interfaces** only

CHAPTER 37

Enum in Java

Enum in Java

What is `enum` in Java?

In Java, `enum` (short for *enumeration*) is a special data type that enables a variable to be a set of predefined constants.

```
enum Direction {
    NORTH, SOUTH, EAST, WEST;
}
```

Enums are type-safe and can be used to define a collection of constants that belong together, such as days of the week, states, directions, etc.

Why Use Enum?

- Improves **code readability** and **type safety**
- Avoids the use of **magic strings** or **integers**
- Can have **fields, methods, and constructors**
- Can be used in **switch** statements

Syntax of Enum

```
enum EnumName {
    CONSTANT1, CONSTANT2, CONSTANT3;
}
```

Example:

```
enum Day {
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY;
}
```

Using Enum in Code

```
public class TestEnum {
    enum Season {
        WINTER, SPRING, SUMMER, FALL
    }

    public static void main(String[] args) {
        Season s = Season.WINTER;
        System.out.println("Season is: " + s);
    }
}
```

Enums with Switch Case

```
public class EnumSwitchExample {
    enum Day {
        MONDAY, TUESDAY, WEDNESDAY
    }

    public static void main(String[] args) {
        Day day = Day.TUESDAY;

        switch (day) {
            case MONDAY:
                System.out.println("Start of the week");
                break;
            case TUESDAY:
                System.out.println("Second day");
                break;
            default:
                System.out.println("Other day");
        }
    }
}
```

Enum with Fields, Constructors, and Methods

```
enum Planet {
    EARTH(5.97), MARS(0.642), JUPITER(1898);

    private double mass; // in 10^24 kg

    // Constructor
    Planet(double mass) {
        this.mass = mass;
    }
```

```
    // Method
    public double getMass() {
        return mass;
    }
}
```

Usage:

```
public class TestPlanet {
    public static void main(String[] args) {
        for (Planet p : Planet.values()) {
            System.out.println(p + " mass is " + p.getMass() + " *10^24 kg");
        }
    }
}
```

Can Enum in Java Have Constructors?

Yes, enum in Java can have constructors.

But there are some **special rules**:

Enum Constructor Rules

1. **Enum constructors are always `private` or package-private (default)**.

You **cannot** use `public` or `protected` constructors in enums.

2. The **constructor is called automatically** when enum constants are created.
3. You **cannot create enum objects using `new` keyword** manually.

Example with Constructor

```
enum Level {
    LOW(1), MEDIUM(2), HIGH(3);

    private int code;

    // private constructor
    private Level(int code) {
```

```
        this.code = code;
        System.out.println("Constructor called for: " + this.name());
    }

    public int getCode() {
        return code;
    }
}

public class EnumConstructorDemo {
    public static void main(String[] args) {
        for (Level l : Level.values()) {
            System.out.println(l + " -> Code: " + l.getCode());
        }
    }
}
```

Output:

```
Constructor called for: LOW
Constructor called for: MEDIUM
Constructor called for: HIGH
LOW -> Code: 1
MEDIUM -> Code: 2
HIGH -> Code: 3
```

Invalid Example (Compile-Time Error):

```
enum Status {
    ACTIVE, INACTIVE;

    public Status() { }  //  Error: Modifier public not allowed here
}
```

Correction:

```
enum Status {
    ACTIVE, INACTIVE;

    Status() {  //  Valid: default or private
        System.out.println("Status constructor");
    }
}
```

Summary:

Rule	Explanation
Enum can have constructor	Yes
Constructor modifier allowed	Only `private` or package-private (default)
Can we use `new` to create enum?	No
When is constructor called?	Automatically at class loading
Purpose of constructor	To assign fields to enum constants

Commonly Used Methods in Enum

Method	Description
`values()`	Returns array of all constants in enum
`valueOf(String name)`	Returns enum constant with given name
`ordinal()`	Returns the position/index of enum constant
`name()`	Returns the name of the constant as declared

Example:

```
public class EnumMethods {
    enum Color {
        RED, GREEN, BLUE;
    }

    public static void main(String[] args) {
        Color c = Color.RED;

        System.out.println(c.name());     // RED
        System.out.println(c.ordinal());  // 0
        System.out.println(Color.valueOf("GREEN")); // GREEN

        for (Color col : Color.values()) {
            System.out.println(col);
        }
    }
}
```

Enum Can Implement Interfaces (But Not Extend Classes)

```
interface Greetable {
    void greet();
```

```
}

enum Mood implements Greetable {
    HAPPY, SAD, ANGRY;

    public void greet() {
        System.out.println("Hello, I am " + this);
    }
}
```

Enum as Singleton (Advanced Usage)

Enums are also used to implement **Singleton Pattern** in Java:

```
enum Singleton {
    INSTANCE;

    public void showMessage() {
        System.out.println("This is a singleton using enum.");
    }
}
```

Usage:

```
public class Main {
    public static void main(String[] args) {
        Singleton.INSTANCE.showMessage();
    }
}
```

Key Points to Remember

- Enum is implicitly final (cannot be extended).
- Enum constants are public, static, and final.
- Can include constructors, methods, and variables.
- Cannot extend a class because Java enums already extend `java.lang.Enum`.
- Can implement interfaces.

Real-Life Use Cases

| Scenario | Enum Example |

| --- | --- |
Direction	`NORTH, EAST, SOUTH, WEST`
Status	`PENDING, APPROVED, REJECTED`
Payment Method	`CASH, CARD, UPI`
Level	`LOW, MEDIUM, HIGH`
Days of Week	`MONDAY...SUNDAY`

CHAPTER 38

Switch 14+ in Java

What is Switch 14+ in Java?

Java 14 introduced a **new switch expression** syntax as a **preview feature** (and later standardized). It allows the `switch` to:

- Act as an **expression** (return a value)
- Use **arrow (`->`) syntax** instead of `case-break`
- Support **multiple labels (case values) in one line**
- Improve **readability**, **less boilerplate**, and **better error handling**

Traditional `switch` (Before Java 14)

```
int day = 3;
String dayName;

switch (day) {
    case 1:
        dayName = "Monday";
        break;
    case 2:
        dayName = "Tuesday";
        break;
    case 3:
        dayName = "Wednesday";
        break;
    default:
        dayName = "Invalid";
}
System.out.println(dayName);
```

Problems:

- Verbose (`break` needed every time)
- Can forget `break` → fall-through bugs
- Not an expression (can't return value directly)

Enhanced `switch` as Expression (Java 14+)

```
int day = 3;

String dayName = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    case 3 -> "Wednesday";
    default -> "Invalid";
};

System.out.println(dayName);
```

Improvements:

- Uses `->` arrow syntax
- No `break` needed
- Compact and clear
- Returns value directly (as **expression**)

Multiple Labels in a Single Case (Java 14+)

```
String result = switch (day) {
    case 1, 2, 3, 4, 5 -> "Weekday";
    case 6, 7 -> "Weekend";
    default -> "Invalid day";
};
```

Using `yield` in Blocked Case

If logic is complex and you need multiple lines, you can use **`yield`** to return a value.

```
int num = 10;

String result = switch (num % 2) {
    case 0 -> {
        System.out.println("Even number");
        yield "Even";
    }
    case 1 -> "Odd";
    default -> "Unknown";
};
System.out.println(result);
```

Summary Table: Traditional vs Enhanced Switch

Feature	Traditional `switch`	Enhanced `switch` (Java 14+)
Arrow syntax (`->`)		
Return value directly		(expression)
Multiple case labels		(`case 1, 2, 3`)
Fall-through	(can happen accidentally)	(no fall-through)
Use of `break`	(required)	
Use of `yield`		(for multi-line case)
Compact syntax		

Real-life Example: Grade System

```
char grade = 'B';

String remark = switch (grade) {
    case 'A' -> "Excellent";
    case 'B' -> "Good";
    case 'C' -> "Average";
    case 'D' -> "Pass";
    case 'F' -> "Fail";
    default -> "Invalid Grade";
};
System.out.println("Result: " + remark);
```

Notes:

- **Preview in Java 12 & 13**, finalized in **Java 14**
- The **old syntax is still valid**; both can coexist
- Great for **pattern matching (future)** and **cleaner code**

Object Class in Java

What is `Object` Class?

- The `Object` class is the **root (superclass) of all Java classes**.
- Every class in Java **implicitly or explicitly inherits** from `Object`.
- Defined in the `java.lang` package.
- If no `extends` keyword is used in a class, it automatically extends the `Object` class.

Why `Object` Class is Important?

- It provides **basic methods** that every Java object has.
- These methods can be **used or overridden** by derived classes.
- Examples include: `toString()`, `equals()`, `hashCode()`, `clone()`, etc.

`` **List of 11 Methods in `Object` Class**

S.No	Method Name	Description
1	`public String toString()`	Converts object to String
2	`public boolean equals(Object obj)`	Compares two objects
3	`public int hashCode()`	Returns object's hash code
4	`protected Object clone()`	Creates and returns a copy
5	`public final Class<?> getClass()`	Returns runtime class info
6	`protected void finalize()`	Cleanup before object destruction
7	`public final void wait()`	Pauses current thread (no timeout)
8	`public final void wait(long timeout)`	Waits with timeout
9	`public final void wait(long timeout, int nanos)`	High-precision wait
10	`public final void notify()`	Wakes up one waiting thread
11	`public final void notifyAll()`	Wakes up all waiting threads

1. `toString()`

Purpose:

Returns a string representing the object.

Default:

```
ClassName@HexHashCode
```

Syntax:

```
public String toString()
```

Example:

```
class Student {
    int id = 1;
    String name = "Ram";
}
Student s = new Student();
System.out.println(s.toString()); // Student@15db9742
```

Override to get readable output:

```
public String toString() {
    return id + " " + name;
}
```

2. `equals(Object obj)`

Purpose:

Compares objects. Default: checks references (==). Override to compare content.

Syntax:

```
public boolean equals(Object obj)
```

Example:

```
String s1 = new String("A");
String s2 = new String("A");

System.out.println(s1.equals(s2)); // true
System.out.println(s1 == s2);      // false
```

Override Example:

```
public boolean equals(Object obj) {
    Student s = (Student)obj;
    return id == s.id && name.equals(s.name);
}
```

3. `hashCode()`

Purpose:

Returns integer hash code, used in `HashMap`, `HashSet`.

Syntax:

```
public int hashCode()
```

Rule:

If `equals()` is overridden, then `hashCode()` **must also be overridden**.

Example:

```
System.out.println("A".hashCode()); // same hash for same content
```

Override:

```
public int hashCode() {
    return id;
}
```

4. `clone()`

Purpose:

Creates a **copy** (shallow) of the object.

Conditions:

- Class must implement `Cloneable`.
- Override `clone()` with `public`.

Syntax:

```
protected Object clone() throws CloneNotSupportedException
```

Example:

```
class Student implements Cloneable {
    int id = 10;
    public Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}
```

5. `getClass()`

Purpose:

Gives runtime class info. Useful in **Reflection API**.

Syntax:

```
public final Class<?> getClass()
```

Example:

```
Student s = new Student();
System.out.println(s.getClass().getName()); // Student
```

6. `finalize()`

Purpose:

Called by Garbage Collector **before destroying object**.

Syntax:

```
protected void finalize() throws Throwable
```

Example:

```
protected void finalize() {
    System.out.println("Object is garbage collected");
}
```

Note: `finalize()` is deprecated in Java 9+

7. `wait()`

Purpose:

Pauses the current thread **until another thread calls `notify()`**.

Syntax:

```
public final void wait() throws InterruptedException
```

Use:

Must be called inside `synchronized` block.

Example:

```
synchronized(obj) {
    obj.wait();
}
```

8. `wait(long timeout)`

Purpose:

Waits for a specified **timeout in milliseconds**.

Syntax:

```
public final void wait(long timeout) throws InterruptedException
```

9. `wait(long timeout, int nanos)`

Purpose:

Waits with millisecond and nanosecond **precision**.

Syntax:

```
public final void wait(long timeout, int nanos) throws InterruptedException
```

10. `notify()`

Purpose:

Wakes up **one** thread waiting on the object.

Syntax:

```
public final void notify()
```

Example:

```
synchronized(obj) {
    obj.notify();
}
```

11. `notifyAll()`

Purpose:

Wakes up **all** threads waiting on the object.

Syntax:

```
public final void notifyAll()
```

Summary Table

Method	Used For	Overridable	Notes
`toString()`	String form of object	Yes	Override for readability
`equals()`	Compare object content	Yes	Must override `hashCode()` too
`hashCode()`	Hash-based collections	Yes	Same object = same hash
`clone()`	Create object copy	Yes	Needs `Cloneable`
`getClass()`	Reflection info	No	Final method
`finalize()`	Cleanup before GC	Yes	Deprecated now
`wait()`	Thread sync	No	Used in `synchronized` block
`wait(long)`	Timed wait	No	Same as above with timeout
`wait(long, int)`	Nanosecond precision wait	No	Rarely used
`notify()`	Wake one thread	No	Must be in `synchronized`
`notifyAll()`	Wake all threads	No	Must be in `synchronized`

CHAPTER 40

Exception Handling in Java

Exception Handling in Java

1. **What is an Exception?**

- An **Exception** is an **unexpected event** that disrupts the normal flow of the program.
- It occurs during **runtime**.
- Java provides a robust mechanism to **handle runtime errors**, so the program doesn't crash.

---

2. **Hierarchy of Exceptions**

```
Object
 └── Throwable
      ├── Error (Not meant to handle)
      └── Exception
           ├── Checked Exceptions (Compile-time)
           └── Unchecked Exceptions (Runtime)
```

![Exception handling](https://rollbar.com/wp-content/uploads/2021/07/java-exceptions-hierarchy-example.png "Exception handling")

![Exception handling](https://lh6.googleusercontent.com/UuHJjdra7T1cHIuicA8GgpV9lPA0k7BkXmpgwXt68Eb_d29iEzBzKnPtX8r4Qs4a3GwT-_GW404bGv8F-VLhBfTer0hhbLnwtQwgC4g7GlCe1oL8naJ7c_j6as_po6DPfAQhxETOnQULknrZRA "Exception handling")

Checked Exceptions

- Detected at **compile time**
- Must be handled using **try-catch or throws**
- Examples:
  - `IOException`
  - `SQLException`
  - `FileNotFoundException`

Unchecked Exceptions

- Detected at **runtime**
- Program compiles without handling

- Examples:
  - `ArithmeticException`
  - `NullPointerException`
  - `ArrayIndexOutOfBoundsException`

---

3. **Common Java Exceptions**

Exception	Cause
`ArithmeticException`	Divide by zero
`NullPointerException`	Accessing object with null reference
`ArrayIndexOutOfBoundsException`	Invalid array index
`NumberFormatException`	Converting string to number improperly
`ClassNotFoundException`	Class not found at runtime
`IOException`	Input-output failure

---

4. **Exception Handling Keywords**

Keyword	Purpose
`try`	Encloses risky code
`catch`	Handles the exception
`finally`	Executes always (cleanup code)
`throw`	Used to explicitly throw an exception
`throws`	Declares exceptions for a method

---

5. **Basic Syntax**

```
try {
    // risky code
} catch (ExceptionType name) {
    // exception handler
} finally {
    // cleanup code
}
```

---

6. **Multiple Catch Blocks**

```
try {
    int a = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Arithmetic error");
} catch (Exception e) {
    System.out.println("General error");
}
```

> ***Note:** Catch blocks should be ordered from **specific to general**.*

---

7. **finally Block**

- Always executes — even if exception is not caught or `return` is used.

```
try {
    int x = 5 / 1;
} catch (Exception e) {
    System.out.println("Handled");
} finally {
    System.out.println("This will always execute");
}
```

---

8. **throw Keyword**

Used to **manually throw** an exception.

```
throw new ArithmeticException("Divide by zero");
```

---

9. **throws Keyword**

Used in method signature to declare exceptions.

```
public void readFile() throws IOException {
    FileReader f = new FileReader("file.txt");
}
```

---

10. **Custom Exception**

You can create your own exception by extending `Exception` or `RuntimeException`.

```
class AgeInvalidException extends Exception {
    AgeInvalidException(String msg) {
        super(msg);
    }
}
```

```
class Test {
    void checkAge(int age) throws AgeInvalidException {
        if (age < 18)
            throw new AgeInvalidException("Age is not valid");
    }
}
```

---

11. **Exception Propagation**

- If a method throws an exception and it is not handled there, it propagates to the **caller**.
- This continues up the **call stack**.

```
void methodA() {
    methodB();
}

void methodB() {
    int a = 10 / 0; // Exception
}
```

---

12. **Difference: throw vs throws**

throw	throws
Used to **throw** exception	Used to **declare** exception
Followed by exception instance	Followed by exception class
Used inside method	Used in method signature

---

13. **Best Practices**

- Catch specific exceptions instead of generic ones.
- Don't use exceptions for flow control.
- Close resources using `finally` or **try-with-resources**.
- Always log exceptions.

14. **try-with-resources (Java 7+)**

Automatically closes resources (like files, connections).

```
try (FileReader fr = new FileReader("data.txt")) {
    // use file
} catch (IOException e) {
    e.printStackTrace();
}
```

15. **Real-Time Analogy**

- **try block** = You try to withdraw money from ATM.
- **catch block** = If ATM runs out of cash, handle the problem.
- **finally block** = Whether transaction succeeds or fails, ATM card is returned.

Example: Full Exception Program

```
public class ExceptionDemo {
    public static void main(String[] args) {
        try {
            int a = 10, b = 0;
            int result = a / b;
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero: " + e);
        } finally {
            System.out.println("Program ended.");
        }
    }
}
```

CHAPTER 41

String in Java

String in Java

What is a String in Java?

- In Java, a `String` is **a sequence of characters**.
- `String` is a **class** in the `java.lang` package.
- Strings in Java are **immutable**, meaning **once created, they cannot be changed**.

```
String s = "Hello";  // String literal
```

Ways to Create a String

1. **Using String Literal**

```
String s1 = "Java";
```

- JVM stores the string in a **String Constant Pool (SCP)**.
- If the string already exists in SCP, the same reference is reused.

2. **Using `new` Keyword**

```
String s2 = new String("Java");
```

- Creates a new string in the heap memory.
- SCP is not reused unless interned.

Memory Allocation: SCP vs Heap

Aspect	SCP (String Constant Pool)	Heap
Location	Part of method area	Main memory
Duplicates	Avoids duplicate values	Allows duplicates

Usage	Efficient memory	Less efficient
Created using	String literals	`new String()`

Immutability in String

- Once a string is created, **its content cannot be changed**.
- Any modification results in a **new object**.

```
String s = "Java";
s.concat(" Programming");
System.out.println(s);  // Output: Java
```

`s.concat(" Programming")` returns a new String, but `s` still points to "Java".

Important String Methods

Method	Description	Example
`length()`	Returns length of the string	`s.length()`
`charAt(int index)`	Returns char at a given index	`s.charAt(0)`
`substring(int start)`	Returns substring from index	`s.substring(2)`
`substring(int start, int end)`	Substring from start to end-1	`s.substring(1, 4)`
`equals(String s2)`	Checks content equality	`"Java".equals("Java")`
`equalsIgnoreCase(String)`	Ignores case	`"java".equalsIgnoreCase("Java")`
`compareTo(String)`	Lexicographic compare	`"a".compareTo("b")`
`toLowerCase()` / `toUpperCase()`	Changes case	`"Java".toLowerCase()`
`trim()`	Removes leading/trailing spaces	`" Java ".trim()`
`replace(char old, char new)`	Replaces chars	`"java".replace('a', 'o')`
`contains(CharSequence s)`	Checks for substring	`"Java".contains("av")`
`startsWith(String)` / `endsWith(String)`	Prefix/suffix check	`"java".startsWith("j")`
`indexOf(char)` / `lastIndexOf(char)`	Finds index	`"java".indexOf('a')`
`isEmpty()`	Checks if string is empty	`"".isEmpty()`
`split(String regex)`	Splits string into array	`"a,b,c".split(",")`
`toCharArray()`	Converts string to char array	`"abc".toCharArray()`
`intern()`	Adds string to SCP if not already	`"new String("hi").intern()`

String Comparison

Using `==`

- Compares **reference (memory location)**.

```
String a = "Java";
String b = "Java";
System.out.println(a == b);  // true (same SCP ref)
```

Using `equals()`

- Compares **actual content**.

```
String a = new String("Java");
String b = "Java";
System.out.println(a.equals(b));  // true (same content)
```

String Concatenation

- You can use `+` operator or `concat()` method.

```
String a = "Hello";
String b = "World";
String c = a + " " + b;       // "Hello World"
String d = a.concat(b);       // "HelloWorld"
```

- At compile time, JVM **optimizes literals**.

String is Final Class

- `String` is a `final` class → cannot be extended.
- Ensures immutability and thread safety.

```
public final class String implements Serializable, Comparable<String>, CharSequence {
    ...
}
```

Why String is Immutable?

1. **Security**: Used in file paths, class loading, etc.
2. **Thread Safety**: Shared strings won't cause race conditions.
3. **Caching**: Enables **String Pooling** for memory efficiency.
4. **Hashcode Caching**: Used in HashMap/Set; reliable hash code.

String Pool (SCP)

- Maintains **a unique copy of strings**.
- Saves memory by **reusing literals**.

```
String s1 = "Java";
String s2 = "Java";
System.out.println(s1 == s2);  // true
```

`intern()` Method

- Forces the string into the SCP.

```
String s1 = new String("hello");
String s2 = s1.intern();  // s2 now points to SCP version
```

Examples

Reverse a String

```
String s = "Java";
String rev = "";
for (int i = s.length() - 1; i >= 0; i--) {
    rev += s.charAt(i);
}
System.out.println(rev);  // avaJ
```

Count vowels

```
String s = "Hello World";
int count = 0;
for (char ch : s.toLowerCase().toCharArray()) {
    if ("aeiou".indexOf(ch) != -1) count++;
}
System.out.println(count);  // 3
```

CHAPTER 42

StringBuilder and StringBuffer in Java

`StringBuilder` and `StringBuffer` in Java

Java provides two important classes for creating and manipulating **mutable (changeable) strings**:

- `StringBuilder`
- `StringBuffer`

Both are alternatives to the **immutable** `String` class.

Why Use `StringBuilder` or `StringBuffer`?

The `String` class creates **immutable** objects — once created, their content **cannot be changed**.
Every time a modification (append, insert, etc.) is made to a `String`, a **new object** is created, which can impact **memory and performance**.

To handle **frequent string modifications**, Java provides:

- `StringBuffer` (since JDK 1.0)
- `StringBuilder` (since JDK 1.5)

Difference Between `String`, `StringBuffer`, and `StringBuilder`

Feature	`String`	`StringBuffer`	`StringBuilder`
Mutability	Immutable	Mutable	Mutable
Thread-Safety	Not thread-safe	Thread-safe (synchronized)	Not thread-safe
Performance	Slow for changes	Slower due to sync	Faster (no sync)
Introduced In	JDK 1.0	JDK 1.0	JDK 1.5
Use Case	Few modifications	Multithreading environment	Single-threaded environment

Constructors

```
StringBuilder sb = new StringBuilder();                    // default capacity 16
StringBuilder sb2 = new StringBuilder("Hello");            // initializes with string
```

```
StringBuilder sb3 = new StringBuilder(50);              // custom capacity
```

Same for `StringBuffer`.

Common Methods (Same for Both Classes)

Method	Description
`append(String s)`	Adds the string at the end
`insert(int offset, s)`	Inserts at the specified index
`replace(int start, int end, s)`	Replaces from start to end
`delete(int start, int end)`	Deletes characters from range
`reverse()`	Reverses the content
`toString()`	Converts to `String`
`length()`	Returns length
`capacity()`	Returns capacity
`ensureCapacity(int min)`	Ensures a minimum capacity
`charAt(int index)`	Returns char at index
`setCharAt(int index, c)`	Sets character at given index

Example: Using `StringBuilder`

```
public class BuilderExample {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("Hello");

        sb.append(" Java");         // Hello Java
        sb.insert(6, "World ");     // Hello World Java
        sb.replace(6, 11, "Guru");  // Hello Guru Java
        sb.delete(5, 10);           // Hello Java
        sb.reverse();               // avaJ olleH

        System.out.println(sb);     // Output: avaJ olleH
    }
}
```

Example: Using `StringBuffer` (Thread-Safe)

```
public class BufferExample {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Welcome");

        sb.append(" to Java");         // Welcome to Java
        sb.insert(8, "Core ");         // Welcome Core to Java
        sb.delete(0, 8);               // Core to Java
        sb.reverse();                  // avaJ ot eroC

        System.out.println(sb);        // Output: avaJ ot eroC
    }
}
```

When to Use What?

Use Case	Best Choice
Frequent modifications, single-threaded code	`StringBuilder`
Frequent modifications, multi-threaded code	`StringBuffer`
Infrequent modifications or constant strings	`String`

Internal Working

- Both `StringBuilder` and `StringBuffer` use **character arrays** internally.
- Default capacity: **16 characters**.
- If exceeded, capacity is increased using:

```
newCapacity = (oldCapacity * 2) + 2;
```

Thread-Safety

- `StringBuffer` is **synchronized**:
  - All methods are thread-safe.
  - Slightly **slower** due to synchronization overhead.
- `StringBuilder` is **not synchronized**:
  - Faster but not safe in multithreading.

Summary Table

Feature	`String`	`StringBuffer`	`StringBuilder`
Mutable			
Thread-safe			
Performance		Medium	
Usage	Constants	Threaded env	Fast execution

CHAPTER 43

Multithreading in Java

1. Introduction to Multithreading

- **Definition**: Multithreading is the capability of a CPU, or a single core in a multi-core processor, to execute multiple threads concurrently.
- **Thread**: A thread is a lightweight sub-process; it's the smallest unit of processing.
- **Multitasking**: Executing multiple tasks at the same time.
  - **Process-based**: Each process has its own memory.
  - **Thread-based**: Multiple threads within the same process share memory.
- **Why Multithreading**:
  - Efficient CPU utilization
  - Simultaneous tasks like file download + media playback
  - Better performance in I/O operations

---

2. Creating Threads in Java

Java provides two main ways to create threads:

a) Extending `Thread` class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}
public class Demo {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start(); // starts the thread and calls run()
    }
}
```

b) Implementing `Runnable` interface

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread running");
    }
```

```
}
public class Demo {
    public static void main(String[] args) {
        Thread t = new Thread(new MyRunnable());
        t.start();
    }
}
```

3. Thread Lifecycle (States)

Java threads go through the following states:

1. **New**
2. **Runnable**
3. **Running**
4. **Blocked/Waiting**
5. **Terminated**

![Multithreading](https://media.licdn.com/dms/image/v2/D5612AQE2GE-5x02rcw/article-cover_image-shrink_720_1280/article-cover_image-shrink_720_1280/0/1691427612573?e=2147483647&v=beta&t=qUc8owAv8f6kurDR9sksZlZfAXxxYtAlodqwQqbRiGs "Mul")

Use methods like `start()`, `sleep()`, `join()`, `wait()`, `notify()`, `stop()` (deprecated) to control the states.

```
class LifecycleDemo extends Thread {
    public void run() {
        System.out.println("Thread running...");
    }

    public static void main(String[] args) throws InterruptedException {
        LifecycleDemo t = new LifecycleDemo();
        System.out.println("State before start: " + t.getState());
        t.start();
        System.out.println("State after start: " + t.getState());
        t.join();
        System.out.println("State after finish: " + t.getState());
    }
}
```

4. Thread Methods

Commonly used methods:

Method	Description
`start()`	Starts the thread
`run()`	Entry point of thread logic
`sleep(ms)`	Suspends thread temporarily
`join()`	Waits for a thread to finish execution
`isAlive()`	Checks if thread is still alive
`setPriority()`	Sets thread priority (1–10)
`getPriority()`	Gets thread priority
`yield()`	Pauses current thread for others

```
class ThreadMethods extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(Thread.currentThread().getName() + " " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {}
        }
    }

    public static void main(String[] args) {
        ThreadMethods t1 = new ThreadMethods();
        ThreadMethods t2 = new ThreadMethods();
        t1.setName("Thread-1");
        t2.setName("Thread-2");
        t1.start();
        t2.start();
    }
}
```

5. Thread Priorities

- Java uses integer values between 1 and 10.
  - `Thread.MIN_PRIORITY` = 1
  - `Thread.NORM_PRIORITY` = 5
  - `Thread.MAX_PRIORITY` = 10
- Schedulers may consider priority, but not guaranteed.

```
public class PriorityExample extends Thread {
    public void run() {
        System.out.println(getName() + " Priority: " + getPriority());
    }

    public static void main(String[] args) {
```

```
        PriorityExample t1 = new PriorityExample();
        PriorityExample t2 = new PriorityExample();
        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.MAX_PRIORITY);
        t1.start();
        t2.start();
    }
}
```

6. Daemon Threads

- **Daemon threads** are background service threads like garbage collection.
- Use `setDaemon(true)` before `start()`.

```
class DaemonExample extends Thread {
    public void run() {
        if (Thread.currentThread().isDaemon()) {
            System.out.println("Daemon thread running");
        } else {
            System.out.println("User thread running");
        }
    }

    public static void main(String[] args) {
        DaemonExample d = new DaemonExample();
        d.setDaemon(true);
        d.start();

        DaemonExample u = new DaemonExample();
        u.start();
    }
}
```

7. Synchronization in Threads

When multiple threads access shared resources, **race conditions** can occur.

a) `synchronized` keyword

```
synchronized void printData() {
    // only one thread at a time
}
```

b) Synchronizing block

```
synchronized(obj) {
    // critical section
}
```

c) Static synchronization

```
synchronized static void staticMethod() {
    // class-level lock
}
```

```
class Counter {
    int count = 0;

    synchronized void increment() {
        count++;
    }
}

public class SyncDemo extends Thread {
    static Counter c = new Counter();

    public void run() {
        for (int i = 0; i < 1000; i++) {
            c.increment();
        }
    }

    public static void main(String[] args) throws Exception {
        SyncDemo t1 = new SyncDemo();
        SyncDemo t2 = new SyncDemo();
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println("Final Count: " + c.count);
    }
}
```

8. Inter-Thread Communication

Useful in **producer-consumer** scenarios.

- Methods:
  - `wait()`: Causes the thread to wait.
  - `notify()`: Wakes up one waiting thread.
  - `notifyAll()`: Wakes up all waiting threads.

```
synchronized(obj) {
    obj.wait(); // wait on obj
    obj.notify(); // notify one
}
```

```
class Shared {
    synchronized void printTable(int n) {
        for (int i = 1; i <= 5; i++) {
            System.out.println(n * i);
            try {
                wait();
            } catch (Exception e) {}
        }
    }
}

public class ITCDemo {
    public static void main(String[] args) {
        Shared obj = new Shared();

        new Thread(() -> obj.printTable(5)).start();
        new Thread(() -> {
            try { Thread.sleep(1000); } catch (Exception e) {}
            synchronized (obj) {
                obj.notify();
            }
        }).start();
    }
}
```

9. Deadlock

Occurs when two or more threads wait for each other to release resources. Leads to infinite blocking.

Example:

```
synchronized(lock1) {
    synchronized(lock2) {
        // may cause deadlock
    }
```

```
}
```

```
public class DeadlockExample {
    static final Object lock1 = "LOCK1";
    static final Object lock2 = "LOCK2";

    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            synchronized (lock1) {
                try { Thread.sleep(100); } catch (Exception e) {}
                synchronized (lock2) {
                    System.out.println("Thread 1 acquired both locks");
                }
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (lock2) {
                try { Thread.sleep(100); } catch (Exception e) {}
                synchronized (lock1) {
                    System.out.println("Thread 2 acquired both locks");
                }
            }
        });

        t1.start();
        t2.start();
    }
}
```

10. Thread Groups

- A way to group multiple threads into a single unit.

```
ThreadGroup group = new ThreadGroup("GroupA");
Thread t1 = new Thread(group, runnable, "Thread1");
```

```
public class ThreadGroupDemo {
    public static void main(String[] args) {
        ThreadGroup group = new ThreadGroup("GroupA");

        Thread t1 = new Thread(group, () -> System.out.println("Thread 1"), "One");
        Thread t2 = new Thread(group, () -> System.out.println("Thread 2"), "Two");

        t1.start();
        t2.start();
```

```
        System.out.println("Active threads: " + group.activeCount());
    }
}
```

11. Thread Pools (`ExecutorService`)

Avoids creating many threads manually. Uses a pool of threads to execute tasks efficiently.

```
ExecutorService service = Executors.newFixedThreadPool(5);
service.submit(() -> System.out.println("Task executed"));
service.shutdown();
```

Types:

- `FixedThreadPool`
- `CachedThreadPool`
- `SingleThreadExecutor`
- `ScheduledThreadPool`

```
import java.util.concurrent.*;

public class ExecutorDemo {
    public static void main(String[] args) {
        ExecutorService pool = Executors.newFixedThreadPool(3);

        for (int i = 1; i <= 5; i++) {
            int task = i;
            pool.execute(() -> System.out.println("Task " + task));
        }

        pool.shutdown();
    }
}
```

12. Fork/Join Framework

For recursive task splitting and joining (Divide and Conquer).

```
class MyTask extends RecursiveTask<Integer> {
    // override compute()
}
ForkJoinPool pool = new ForkJoinPool();
pool.invoke(new MyTask());
```

```
import java.util.concurrent.*;

class MyTask extends RecursiveTask<Integer> {
    int start, end;

    MyTask(int s, int e) {
        start = s;
        end = e;
    }

    protected Integer compute() {
        if (end - start <= 10) {
            int sum = 0;
            for (int i = start; i <= end; i++) sum += i;
            return sum;
        } else {
            int mid = (start + end) / 2;
            MyTask left = new MyTask(start, mid);
            MyTask right = new MyTask(mid + 1, end);
            left.fork();
            return right.compute() + left.join();
        }
    }
}

public class ForkJoinDemo {
    public static void main(String[] args) {
        ForkJoinPool pool = new ForkJoinPool();
        int result = pool.invoke(new MyTask(1, 100));
        System.out.println("Sum: " + result);
    }
}
```

13. `Callable` and `Future`

- `Runnable` cannot return result or throw checked exceptions.
- `Callable` can return value and throw exceptions.

```
Callable<Integer> task = () -> 123;
Future<Integer> future = executor.submit(task);
Integer result = future.get(); // blocking
```

```
import java.util.concurrent.*;

public class CallableDemo {
    public static void main(String[] args) throws Exception {
        Callable<Integer> task = () -> 42;
```

```
        ExecutorService executor = Executors.newSingleThreadExecutor();
        Future<Integer> future = executor.submit(task);
        System.out.println("Result: " + future.get());
        executor.shutdown();
    }
}
```

14. CompletableFuture (Asynchronous Programming)

Non-blocking future with chaining.

```
CompletableFuture.supplyAsync(() -> "Hello")
                 .thenApply(str -> str + " World")
                 .thenAccept(System.out::println);
```

```
import java.util.concurrent.*;

public class CompletableDemo {
    public static void main(String[] args) {
        CompletableFuture.supplyAsync(() -> "Hello")
                         .thenApply(s -> s + " World")
                         .thenAccept(System.out::println);
    }
}
```

15. Volatile Keyword

- Ensures visibility of shared variable across threads.

```
volatile boolean running = true;
```

```
class VolatileDemo extends Thread {
    volatile boolean running = true;

    public void run() {
        while (running) {
            System.out.println("Running...");
        }
    }

    public static void main(String[] args) throws InterruptedException {
        VolatileDemo v = new VolatileDemo();
        v.start();
```

```
        Thread.sleep(1000);
        v.running = false;
    }
}
```

16. Thread Safety

- Making sure shared data is accessed and modified safely.
- Tools:
  - `synchronized`
  - `volatile`
  - `Atomic` classes
  - `Lock` interfaces

17. Locks in Java (`java.util.concurrent.locks`)

- More flexible than `synchronized`

```
Lock lock = new ReentrantLock();
lock.lock();
try {
    // critical section
} finally {
    lock.unlock();
}
```

```
import java.util.concurrent.locks.*;

public class LockExample {
    Lock lock = new ReentrantLock();

    void print() {
        lock.lock();
        try {
            System.out.println("Locked section");
        } finally {
            lock.unlock();
        }
    }

    public static void main(String[] args) {
        LockExample l = new LockExample();
```

```
        new Thread(() -> l.print()).start();
    }
}
```

18. BlockingQueue and Producer-Consumer Problem

Avoid manual synchronization for shared queues.

```
BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(10);
```

```
import java.util.concurrent.*;

public class ProducerConsumer {
    public static void main(String[] args) {
        BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(5);

        new Thread(() -> {
            try {
                for (int i = 0; i < 10; i++) {
                    queue.put(i);
                    System.out.println("Produced: " + i);
                }
            } catch (InterruptedException e) {}
        }).start();

        new Thread(() -> {
            try {
                for (int i = 0; i < 10; i++) {
                    System.out.println("Consumed: " + queue.take());
                }
            } catch (InterruptedException e) {}
        }).start();
    }
}
```

19. ThreadLocal

- Provides thread-specific variables.

```
ThreadLocal<Integer> local = new ThreadLocal<>();
local.set(100); // local to current thread
```

```
public class ThreadLocalDemo {
    public static void main(String[] args) {
        ThreadLocal<Integer> threadLocal = ThreadLocal.withInitial(() -> 1);

        Runnable r = () -> {
            System.out.println(Thread.currentThread().getName() + ": " + threadLocal.get());
            threadLocal.set(threadLocal.get() + 1);
        };

        new Thread(r, "Thread-A").start();
        new Thread(r, "Thread-B").start();
    }
}
```

20. Common Concurrency Utilities

- CountDownLatch
- CyclicBarrier
- Semaphore
- Phaser
- Exchanger

```
import java.util.concurrent.*;

public class LatchDemo {
    public static void main(String[] args) throws InterruptedException {
        CountDownLatch latch = new CountDownLatch(3);

        Runnable task = () -> {
            System.out.println(Thread.currentThread().getName() + " done");
            latch.countDown();
        };

        new Thread(task).start();
        new Thread(task).start();
        new Thread(task).start();

        latch.await();
        System.out.println("All threads finished!");
    }
}
```

CHAPTER 44

Introduction to Collection API

Introduction to Collection API in Java

The **Collection API** in Java is a **framework** that provides a set of **classes and interfaces** to store, retrieve, and manipulate groups of objects efficiently.

Why Collection API?

Before Java 1.2, we used **arrays** and **Vector**, but:

- Arrays have **fixed size**
- No in-built sorting/searching
- No standard mechanism to manipulate group of objects

To overcome these limitations, Java introduced the **Collection Framework** in **Java 1.2**.

What is a Collection?

A **Collection** is simply a group of **objects** (also called **elements**). It acts like a **container** that holds multiple objects together.

Collection Hierarchy

![Collection](https://miro.medium.com/v2/resize:fit:1400/1*2hgLstie4gxpXUUzgAkGgg.png "Collection")

Key Interfaces in Collection API

Interface	Description
`Collection`	Root interface of all collections
`List`	Ordered collection (allows duplicates)
`Set`	Unordered, no duplicates
`Queue`	For processing elements in FIFO order
`Deque`	Double-ended queue (insert/remove from both ends)
`Map` (not part of Collection but related)	Stores data in key-value pairs

---

Interfaces and Their Use

Interface	Description	Allows Duplicates	Ordered?
`List`	Ordered collection, index-based	Yes	Yes
`Set`	Unique elements	No	No (except LinkedHashSet)
`Queue`	FIFO-based collection	Yes	Partially
`Deque`	Double-ended queue	Yes	Yes
`Map`	Key-value pairs	No duplicate keys	Yes (LinkedHashMap), No (HashMap), Sorted (TreeMap)

Common Classes

Interface	Common Implementations
List	`ArrayList`, `LinkedList`, `Vector`, `Stack`
Set	`HashSet`, `LinkedHashSet`, `TreeSet`
Queue	`PriorityQueue`, `ArrayDeque`
Map	`HashMap`, `LinkedHashMap`, `TreeMap`, `Hashtable`

Features of Collection API

- Ready-made **data structures**
- **Dynamic sizing** (no fixed length like arrays)
- **Polymorphic behavior** via interfaces
- Support for **generics** (type safety)
- **Utility methods** in `Collections` class (e.g., sorting, shuffling)

Utility Class

- **`Collections`** – Class in `java.util` package
- Provides static methods like: `sort()`, `reverse()`, `shuffle()`, `min()`, `max()`, `binarySearch()`

Collection vs Collections

Term	Description
`Collection`	Interface representing a group of objects
`Collections`	Utility class with static methods (like sort, reverse, shuffle)

Conclusion

Java Collection API is **powerful and flexible**, making it easier to **store, search, sort, and manipulate** groups of data efficiently. Mastering it is essential for real-world Java development.

CHAPTER 45

Generics in Java

1. Generics in Java

Definition:
Generics allow you to write **type-safe** and **reusable code**. They enable you to specify a **type parameter** for classes, interfaces, and methods.

Example: Without Generics (Pre-Java 5)

```
List list = new ArrayList();
list.add("Hello");
String str = (String) list.get(0);  // Requires casting
```

Example: With Generics (Java 5+)

```
List<String> list = new ArrayList<>();
list.add("Hello");
String str = list.get(0);  // No need for casting
```

Advantages of Generics:

- Type safety
- No casting required
- Compile-time checking

Common Generic Usage:

```
List<Integer>
Map<String, Double>
Class<T>
<T extends Number>
```

---

2. Raw Types in Java

Definition:
A raw type is a **generic class or interface used without specifying a type argument**. This was used before Java introduced generics in Java 5.

Example of Raw Type

```
List list = new ArrayList();  // Raw type
list.add("Hello");
```

```
list.add(10); // Allowed at compile-time

String str = (String) list.get(1);  // Runtime error: ClassCastException
```

Generics vs Raw Types

Feature	Generics	Raw Types
Type Safety	Yes	No
Compile-Time Check	Yes	Limited
Casting Required	No	Yes
Introduced In	Java 5	Before Java 5

Important Notes

- **Using raw types disables type checking**, leading to possible `ClassCastException`.
- Mixing generics and raw types is allowed but **discouraged**.
- Compiler gives **unchecked warnings** when using raw types with generics.

Example: Mixing Raw and Generic

```
List<String> genericList = new ArrayList<>();
List rawList = genericList;         // Allowed
rawList.add(10);                    // Unsafe

String str = genericList.get(0);    // Runtime error if element is not a String
System.out.println(str);
```

CHAPTER 46

ArrayList in Java

What is `ArrayList` in Java?

`ArrayList` is a **resizable array implementation** of the `List` interface in Java. Unlike regular arrays, an `ArrayList` can **grow or shrink** dynamically as elements are added or removed.

It is part of **Java Collection Framework** and is present in the `java.util` package.

Declaration and Syntax

```
import java.util.ArrayList;

ArrayList<Type> listName = new ArrayList<>();
```

Example:

```
ArrayList<String> fruits = new ArrayList<>();
```

Key Features

Feature	Description
Dynamic sizing	Automatically resizes when elements are added/removed.
Maintains order	Insertion order is preserved.
Allows duplicates	Can store duplicate elements.
Indexed access	Elements can be accessed by index (like an array).
Not synchronized	Not thread-safe by default.

Internal Working

- Internally uses an **array** to store elements.
- Default capacity is **10**.
- When full, it creates a **new array with larger capacity** (usually 1.5x the old size) and **copies old elements** into it.

Commonly Used Methods

Method	Description
`add(E e)`	Adds element to end
`add(int index, E e)`	Adds element at specific index
`get(int index)`	Returns element at given index
`set(int index, E e)`	Replaces element at given index
`remove(int index)`	Removes element at given index
`remove(Object o)`	Removes first occurrence of object
`size()`	Returns number of elements
`isEmpty()`	Checks if list is empty
`clear()`	Removes all elements
`contains(Object o)`	Checks if element exists
`indexOf(Object o)`	Returns index of first occurrence
`lastIndexOf(Object o)`	Returns last occurrence index

Example Program

```
import java.util.ArrayList;

public class ArrayListExample {
    public static void main(String[] args) {
        ArrayList<String> students = new ArrayList<>();

        // Add elements
        students.add("Alice");
        students.add("Bob");
        students.add("Charlie");

        // Insert at index
        students.add(1, "David");

        // Access elements
        System.out.println("Student at index 2: " + students.get(2));

        // Replace element
        students.set(0, "Alex");

        // Remove element
        students.remove("Bob");

        // Display all students
        System.out.println("Students: " + students);
```

```
        // Size of list
        System.out.println("Total students: " + students.size());
    }
}
```

Iterating Over `ArrayList`

1. For Loop:

```
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i));
}
```

2. Enhanced For Loop:

```
for (String item : list) {
    System.out.println(item);
}
```

3. Iterator:

```
Iterator<String> itr = list.iterator();
while (itr.hasNext()) {
    System.out.println(itr.next());
}
```

Real-World Use Case

Example: Shopping Cart System

```
ArrayList<String> cart = new ArrayList<>();
cart.add("Milk");
cart.add("Bread");
cart.add("Eggs");
cart.remove("Bread"); // User removed item
System.out.println("Items in cart: " + cart);
```

Difference Between `ArrayList` and Array

Array	ArrayList
Fixed size	Dynamic size
Can store primitives	Only objects (Generics)
Faster	Slightly slower (due to resizing, boxing/unboxing)
Less flexible	More flexible

Thread Safety

- `ArrayList` is **not synchronized**.
- For multi-threaded programs, use:

```
List<String> syncList = Collections.synchronizedList(new ArrayList<>());
```

Generic Type in `ArrayList`

Using Generics ensures **type safety**.

```
ArrayList<Integer> numbers = new ArrayList<>();
numbers.add(10); // Only Integer allowed
```

Convert `ArrayList` to Array

```
String[] arr = list.toArray(new String[0]);
```

Interview Point

- **Time Complexity:**
  - `add()` → O(1) amortized
  - `get()` → O(1)

- `remove()` → O(n) (shifts elements)

Full Java Program Using All Major `ArrayList` Methods

```
import java.util.*;

public class ArrayListMethodsDemo {
    public static void main(String[] args) {
        // 1. Create an ArrayList
        ArrayList<String> list = new ArrayList<>();

        // 2. add(element)
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");

        // 3. add(index, element)
        list.add(1, "Blueberry");

        // 4. get(index)
        System.out.println("Element at index 2: " + list.get(2)); // Banana

        // 5. set(index, element)
        list.set(2, "Blackberry");
        System.out.println("Updated list: " + list);

        // 6. remove(index)
        list.remove(1);
        System.out.println("After removing index 1: " + list);

        // 7. remove(Object)
        list.remove("Apple");
        System.out.println("After removing 'Apple': " + list);

        // 8. contains(element)
        System.out.println("Contains 'Cherry'? " + list.contains("Cherry"));

        // 9. indexOf(element)
        System.out.println("Index of 'Blackberry': " + list.indexOf("Blackberry"));

        // 10. lastIndexOf(element)
        list.add("Cherry");
        System.out.println("Last index of 'Cherry': " + list.lastIndexOf("Cherry"));

        // 11. size()
        System.out.println("Size of list: " + list.size());

        // 12. isEmpty()
        System.out.println("Is list empty? " + list.isEmpty());
```

```
        // 13. clear()
        list.clear();
        System.out.println("List after clear(): " + list);

        // 14. addAll(Collection)
        ArrayList<String> fruits = new ArrayList<>(Arrays.asList("Mango", "Grapes", "Orange"));
        list.addAll(fruits);
        System.out.println("After addAll(): " + list);

        // 15. addAll(index, Collection)
        list.addAll(1, Arrays.asList("Papaya", "Pineapple"));
        System.out.println("After addAll at index 1: " + list);

        // 16. toArray()
        Object[] arr = list.toArray();
        System.out.println("Array from list: " + Arrays.toString(arr));

        // 17. sort()
        Collections.sort(list);
        System.out.println("Sorted list: " + list);

        // 18. iterator()
        System.out.print("Using Iterator: ");
        Iterator<String> itr = list.iterator();
        while (itr.hasNext()) {
            System.out.print(itr.next() + " ");
        }

        // 19. listIterator()
        System.out.print("\nUsing ListIterator (reverse): ");
        ListIterator<String> listItr = list.listIterator(list.size());
        while (listItr.hasPrevious()) {
            System.out.print(listItr.previous() + " ");
        }

        // 20. forEach()
        System.out.print("\nUsing forEach(): ");
        list.forEach(fruit -> System.out.print(fruit + " "));

        // 21. retainAll()
        list.retainAll(Arrays.asList("Mango", "Papaya"));
        System.out.println("\nAfter retainAll(): " + list);

        // 22. removeAll()
        list.removeAll(Arrays.asList("Papaya"));
        System.out.println("After removeAll(): " + list);

        // 23. clone()
        ArrayList<String> clonedList = (ArrayList<String>) list.clone();
        System.out.println("Cloned list: " + clonedList);
```

```
        // 24. ensureCapacity()
        list.ensureCapacity(20); // ensures minimum capacity (not visible effect)
        System.out.println("Capacity ensured to 20");

        // 25. trimToSize()
        list.trimToSize(); // trims size to actual elements
        System.out.println("Trimmed to size");

        // 26. subList()
        list.addAll(Arrays.asList("Kiwi", "Guava", "Lychee"));
        List<String> sub = list.subList(1, 3);
        System.out.println("Sublist (1 to 2): " + sub);
    }
}
```

Output Sample:

```
Element at index 2: Banana
Updated list: [Apple, Blueberry, Blackberry, Cherry]
After removing index 1: [Apple, Blackberry, Cherry]
After removing 'Apple': [Blackberry, Cherry]
Contains 'Cherry'? true
Index of 'Blackberry': 0
Last index of 'Cherry': 2
Size of list: 3
Is list empty? false
List after clear(): []
After addAll(): [Mango, Grapes, Orange]
After addAll at index 1: [Mango, Papaya, Pineapple, Grapes, Orange]
Array from list: [Mango, Papaya, Pineapple, Grapes, Orange]
Sorted list: [Grapes, Mango, Orange, Papaya, Pineapple]
Using Iterator: Grapes Mango Orange Papaya Pineapple
Using ListIterator (reverse): Pineapple Papaya Orange Mango Grapes
Using forEach(): Grapes Mango Orange Papaya Pineapple
After retainAll(): [Mango, Papaya]
After removeAll(): [Mango]
Cloned list: [Mango]
Capacity ensured to 20
Trimmed to size
Sublist (1 to 2): [Guava, Lychee]
```

Summary of Methods Covered:

- **CRUD**: `add()`, `remove()`, `get()`, `set()`
- **Bulk Ops**: `addAll()`, `removeAll()`, `retainAll()`, `clear()`

- **Search**: `contains()`, `indexOf()`, `lastIndexOf()`
- **Utilities**: `size()`, `isEmpty()`, `toArray()`, `clone()`, `subList()`
- **Iteration**: `forEach()`, `iterator()`, `listIterator()`
- **Sorting**: `Collections.sort()`
- **Capacity**: `ensureCapacity()`, `trimToSize()`

CHAPTER 47

LinkedList in Java

What is `LinkedList` in Java?

`LinkedList` is a **linear data structure** that stores elements in **nodes**, where each node contains:

- Data (element)
- Reference to the next (and previous) node

It is a **doubly linked list** implementation of the `List` and `Deque` interfaces in Java and is part of the **Java Collection Framework**.

Declaration and Syntax

```
import java.util.LinkedList;

LinkedList<Type> listName = new LinkedList<>();
```

Example:

```
LinkedList<String> cities = new LinkedList<>();
```

Key Features

Feature	Description
Doubly linked	Each node points to both previous and next nodes
Allows duplicates	Yes
Maintains order	Yes (insertion order)
Not synchronized	Not thread-safe
Efficient insertions/deletions	Especially at beginning/middle

Internal Working

- Each element (node) has:

  - `data`
  - `next` pointer
  - `previous` pointer
- Insertion or deletion doesn't require shifting (like arrays).
- **Slow random access** (no indexing like arrays).

Commonly Used Methods

Method	Description
`add(E e)`	Adds element at end
`addFirst(E e)`	Adds element at the beginning
`addLast(E e)`	Adds element at the end
`add(int index, E e)`	Adds at specific position
`remove()`	Removes first element
`removeFirst()` / `removeLast()`	Removes from ends
`get(int index)`	Returns element at index
`getFirst()` / `getLast()`	Returns end elements
`set(int index, E e)`	Replace element
`contains(Object o)`	Checks if element exists
`clear()`	Removes all elements
`size()`	Number of elements

Example Program

```
import java.util.LinkedList;

public class LinkedListExample {
    public static void main(String[] args) {
        LinkedList<String> tasks = new LinkedList<>();

        // Add elements
        tasks.add("Wake up");
        tasks.add("Brush");
        tasks.add("Exercise");

        // Add at beginning
        tasks.addFirst("Pray");

        // Add at end
```

```
        tasks.addLast("Start Work");

        // Get and Set
        System.out.println("First Task: " + tasks.getFirst());
        tasks.set(1, "Meditate");

        // Remove tasks
        tasks.remove("Brush");
        tasks.removeLast();

        // Print all tasks
        System.out.println("To-Do List: " + tasks);
    }
}
```

Iterating Over `LinkedList`

1. Enhanced For Loop

```
for (String item : list) {
    System.out.println(item);
}
```

2. Iterator

```
Iterator<String> itr = list.iterator();
while (itr.hasNext()) {
    System.out.println(itr.next());
}
```

3. For Loop (not recommended for `LinkedList`)

```
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i)); // Slower!
}
```

Real-World Use Case

Example: Train Compartment Queue

```
LinkedList<String> queue = new LinkedList<>();
queue.add("Passenger1");
queue.add("Passenger2");
queue.addFirst("VIP");
System.out.println("Boarding Order: " + queue);
```

Difference Between `ArrayList` and `LinkedList`

Feature	ArrayList	LinkedList
Backing Structure	Dynamic Array	Doubly Linked List
Access time	Fast (O(1))	Slow (O(n))
Insertion/Deletion	Slow (O(n))	Fast at head/middle (O(1) or O(n))
Memory Usage	Less	More (node pointers)
Use Case	Read-heavy	Insert/delete-heavy

Thread Safety

- Not synchronized
- Use:

```
List<String> syncList = Collections.synchronizedList(new LinkedList<>());
```

Generic Type Support

```
LinkedList<Integer> marks = new LinkedList<>();
marks.add(90);
marks.add(85);
```

Convert `LinkedList` to Array

```
String[] arr = list.toArray(new String[0]);
```

Interview Points

- **When to use LinkedList over ArrayList?**

Use `LinkedList` when:

  - You have **frequent insertions/deletions**
  - You don't need random access

- **Time Complexity:**

Operation	LinkedList
Add at End	O(1)
Add at Middle	O(n)
Remove	O(n)
Access (get/set)	O(n)

Full Java Program Using All Major `LinkedList` Methods

```
import java.util.*;

public class LinkedListMethodsDemo {
    public static void main(String[] args) {
        // 1. Create a LinkedList
        LinkedList<String> list = new LinkedList<>();

        // 2. add(element) - adds to end
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");

        // 3. add(index, element)
        list.add(1, "Blueberry");

        // 4. get(index)
        System.out.println("Element at index 2: " + list.get(2)); // Banana

        // 5. set(index, element)
        list.set(2, "Blackberry");
        System.out.println("Updated list: " + list);

        // 6. remove(index)
        list.remove(1);
        System.out.println("After removing index 1: " + list);

        // 7. remove(Object)
        list.remove("Apple");
```

```
        System.out.println("After removing 'Apple': " + list);

        // 8. contains(element)
        System.out.println("Contains 'Cherry'? " + list.contains("Cherry"));

        // 9. indexOf(element)
        System.out.println("Index of 'Blackberry': " + list.indexOf("Blackberry"));

        // 10. lastIndexOf(element)
        list.add("Cherry");
        System.out.println("Last index of 'Cherry': " + list.lastIndexOf("Cherry"));

        // 11. size()
        System.out.println("Size of list: " + list.size());

        // 12. isEmpty()
        System.out.println("Is list empty? " + list.isEmpty());

        // 13. clear()
        list.clear();
        System.out.println("List after clear(): " + list);

        // 14. addAll(Collection)
        LinkedList<String> fruits = new LinkedList<>(Arrays.asList("Mango", "Grapes", "Orange"));
        list.addAll(fruits);
        System.out.println("After addAll(): " + list);

        // 15. addAll(index, Collection)
        list.addAll(1, Arrays.asList("Papaya", "Pineapple"));
        System.out.println("After addAll at index 1: " + list);

        // 16. toArray()
        Object[] arr = list.toArray();
        System.out.println("Array from list: " + Arrays.toString(arr));

        // 17. sort()
        Collections.sort(list);
        System.out.println("Sorted list: " + list);

        // 18. iterator()
        System.out.print("Using Iterator: ");
        Iterator<String> itr = list.iterator();
        while (itr.hasNext()) {
            System.out.print(itr.next() + " ");
        }

        // 19. listIterator()
        System.out.print("\nUsing ListIterator (reverse): ");
        ListIterator<String> listItr = list.listIterator(list.size());
        while (listItr.hasPrevious()) {
            System.out.print(listItr.previous() + " ");
        }
```

```
        // 20. forEach()
        System.out.print("\nUsing forEach(): ");
        list.forEach(fruit -> System.out.print(fruit + " "));

        // 21. retainAll()
        list.retainAll(Arrays.asList("Mango", "Papaya"));
        System.out.println("\nAfter retainAll(): " + list);

        // 22. removeAll()
        list.removeAll(Arrays.asList("Papaya"));
        System.out.println("After removeAll(): " + list);

        // 23. clone()
        LinkedList<String> clonedList = (LinkedList<String>) list.clone();
        System.out.println("Cloned list: " + clonedList);

        // 24. addFirst()
        list.addFirst("Kiwi");
        System.out.println("After addFirst(): " + list);

        // 25. addLast()
        list.addLast("Guava");
        System.out.println("After addLast(): " + list);

        // 26. getFirst(), getLast()
        System.out.println("First: " + list.getFirst() + ", Last: " + list.getLast());

        // 27. removeFirst(), removeLast()
        list.removeFirst();
        list.removeLast();
        System.out.println("After removeFirst and removeLast: " + list);

        // 28. peek(), peekFirst(), peekLast()
        System.out.println("peek(): " + list.peek());
        System.out.println("peekFirst(): " + list.peekFirst());
        System.out.println("peekLast(): " + list.peekLast());

        // 29. poll(), pollFirst(), pollLast()
        System.out.println("poll(): " + list.poll());
        System.out.println("pollFirst(): " + list.pollFirst());
        System.out.println("pollLast(): " + list.pollLast());

        // 30. push(), pop() (as Stack)
        list.push("Lemon");
        list.push("Strawberry");
        System.out.println("After push(): " + list);
        list.pop();
        System.out.println("After pop(): " + list);

        // 31. descendingIterator()
        System.out.print("Using descendingIterator(): ");
```

```
        Iterator<String> descItr = list.descendingIterator();
        while (descItr.hasNext()) {
            System.out.print(descItr.next() + " ");
        }
    }
}
```

Output Sample (Partial):

```
Element at index 2: Banana
Updated list: [Apple, Blueberry, Blackberry, Cherry]
After removing index 1: [Apple, Blackberry, Cherry]
After removing 'Apple': [Blackberry, Cherry]
Contains 'Cherry'? true
Index of 'Blackberry': 0
Last index of 'Cherry': 2
Size of list: 3
Is list empty? false
List after clear(): []
After addAll(): [Mango, Grapes, Orange]
After addAll at index 1: [Mango, Papaya, Pineapple, Grapes, Orange]
Array from list: [Mango, Papaya, Pineapple, Grapes, Orange]
Sorted list: [Grapes, Mango, Orange, Papaya, Pineapple]
...
Cloned list: [Mango]
After addFirst(): [Kiwi, Mango]
After addLast(): [Kiwi, Mango, Guava]
First: Kiwi, Last: Guava
After removeFirst and removeLast: [Mango]
...
```

Summary of Methods Used in `LinkedList`:

Category	Methods Used
CRUD	`add()`, `remove()`, `get()`, `set()`
Bulk Ops	`addAll()`, `removeAll()`, `retainAll()`, `clear()`
Search	`contains()`, `indexOf()`, `lastIndexOf()`
Iteration	`iterator()`, `listIterator()`, `forEach()`, `descendingIterator()`

Stack Methods	`push()`, `pop()`
Queue/Deque	`addFirst()`, `addLast()`, `removeFirst()`, `removeLast()`, `peek()`, `peekFirst()`, `peekLast()`, `poll()`, `pollFirst()`, `pollLast()`
Utilities	`clone()`, `toArray()`, `size()`, `isEmpty()`
Sorting	`Collections.sort()`

CHAPTER 48

Vector in Java

What is `Vector` in Java?

`Vector` is a **legacy class** in Java that implements the `List` interface. It is **similar to `ArrayList`**, but with one major difference:

> ***Vector is synchronized (thread-safe).***

It is part of the **Java Collection Framework** and uses a **dynamic array** to store elements.

Declaration and Syntax

```
import java.util.Vector;

Vector<Type> vectorName = new Vector<>();
```

Example:

```
Vector<String> books = new Vector<>();
```

Key Features

Feature	Description
Dynamic array	Grows when needed
Thread-safe	All methods are synchronized
Allows duplicates	Yes
Maintains order	Yes (insertion order)
Legacy class	Introduced in Java 1.0
Implements List	Compatible with modern collections

Internal Working

- Internally uses an **Object\[] array**.
- Default capacity is **10**.
- When full, it **doubles** its size (2x old capacity).
- All public methods are **synchronized**, making it thread-safe but **slower** in single-threaded contexts.

Commonly Used Methods

Method	Description
`add(E e)`	Adds element to end
`add(int index, E e)`	Adds at index
`get(int index)`	Returns element
`set(int index, E e)`	Updates element
`remove(int index)`	Removes by index
`remove(Object o)`	Removes by object
`size()`	Total number of elements
`contains(Object o)`	Checks existence
`clear()`	Empties the vector
`isEmpty()`	Checks if vector is empty
`firstElement()` / `lastElement()`	Retrieves ends

Example Program

```
import java.util.Vector;

public class VectorExample {
    public static void main(String[] args) {
        Vector<String> languages = new Vector<>();

        // Adding elements
        languages.add("Java");
        languages.add("Python");
        languages.add("C++");

        // Insert at position
        languages.add(1, "JavaScript");

        // Get and Set
        System.out.println("Language at index 2: " + languages.get(2));
        languages.set(2, "C#");
```

```
        // Remove
        languages.remove("Python");

        // Print all
        System.out.println("Languages: " + languages);
        System.out.println("Total count: " + languages.size());
    }
}
```

Iterating Over `Vector`

1. Enhanced For Loop

```
for (String item : vector) {
    System.out.println(item);
}
```

2. Iterator

```
Iterator<String> itr = vector.iterator();
while (itr.hasNext()) {
    System.out.println(itr.next());
}
```

3. Enumeration (Legacy)

```
Enumeration<String> e = vector.elements();
while (e.hasMoreElements()) {
    System.out.println(e.nextElement());
}
```

Real-World Use Case

> ***Example: Banking Transaction Log (Thread-Safe)***

```
Vector<String> logs = new Vector<>();
logs.add("Transaction 1");
logs.add("Transaction 2");
logs.add("Transaction 3");
```

```
System.out.println("Logs: " + logs);
```

Difference: `Vector` vs `ArrayList` vs `LinkedList`

Feature	Vector	ArrayList	LinkedList
Thread-safe	Yes	No	No
Grow Rate	Doubles	1.5x	N/A (linked)
Access	Fast (O(1))	Fast (O(1))	Slow (O(n))
Insert/Delete Mid	Slow	Slow	Fast
Legacy?	Yes	Modern	Modern
Backing Structure	Array	Array	Linked Nodes

Thread Safety

- All methods in `Vector` are **synchronized**, making it **safe in multi-threaded environments**.
- However, synchronization causes **performance overhead** in single-threaded applications.

Modern Alternative:

```
List<String> syncList = Collections.synchronizedList(new ArrayList<>());
```

Generics in Vector

```
Vector<Integer> marks = new Vector<>();
marks.add(75);
marks.add(88);
```

Convert `Vector` to Array

```
String[] arr = vector.toArray(new String[0]);
```

Interview Points

- `Vector` is rarely used today unless synchronization is a **must**.
- For modern development:
  - Use `ArrayList` if **thread safety is not required**
  - Use `Collections.synchronizedList()` or `CopyOnWriteArrayList` for **thread safety**
- **Time Complexity:**

Operation	Time
`add()`	O(1)
`get()`	O(1)
`remove()`	O(n)
`insert()`	O(n)

Summary

Feature	Vector
Introduced in	Java 1.0
Thread-safe	Yes
Ordered?	Yes
Allows nulls?	Yes
Allows duplicates?	Yes
Use Case	Rare – mainly when synchronization is needed

CHAPTER 49

Cursor in Java

Cursor in Java (Collection Cursor)

What is a Cursor?

In Java, a **cursor** is an object used to **traverse (navigate)** through elements in a **collection** (like `ArrayList`, `LinkedList`, `Vector`).

It helps to **iterate**, **read**, **remove**, or even **modify** elements.

Types of Cursors in Java

Java provides **3 types of cursors**:

Cursor	Direction	Can Remove?	Can Update/Add?	Works With
1. Enumeration	Forward only	No	No	Legacy classes (`Vector`, `Stack`)
2. Iterator	Forward only	Yes	No	All collections
3. ListIterator	Forward & Backward	Yes	Yes	Only `List` implementations

1. Enumeration Cursor

- Used with **legacy classes** like `Vector`, `Stack`, `Hashtable`.
- Only allows **reading** elements.

```
import java.util.*;

public class EnumCursor {
    public static void main(String[] args) {
        Vector<String> v = new Vector<>();
        v.add("Java");
        v.add("Python");

        Enumeration<String> e = v.elements();

        while (e.hasMoreElements()) {
            System.out.println(e.nextElement());
        }
    }
}
```

Limitations:

- Cannot remove or add elements.
- **Read-only** and **forward-only**.

2. Iterator Cursor

- Used with all collections (like `ArrayList`, `LinkedList`, `HashSet`, etc.)
- Allows **removal** while traversing.

```
import java.util.*;

public class IteratorCursor {
    public static void main(String[] args) {
        ArrayList<String> list = new ArrayList<>();
        list.add("Java");
        list.add("C++");
        list.add("Python");

        Iterator<String> it = list.iterator();
        while (it.hasNext()) {
            String lang = it.next();
            if (lang.equals("C++")) {
                it.remove();  // Safe removal
            } else {
                System.out.println(lang);
            }
        }
        System.out.println("Final List: " + list);
    }
}
```

Limitations:

- **Only forward direction**.
- Cannot **add** or **update**.

3. ListIterator Cursor

- Used only with **List implementations** (`ArrayList`, `LinkedList`, etc.)
- Can **traverse in both directions** (forward and backward).
- Can **add, remove, or update** elements.

```
import java.util.*;
```

```
public class ListIteratorCursor {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.add("Apple");
        list.add("Banana");

        ListIterator<String> lit = list.listIterator();

        while (lit.hasNext()) {
            String fruit = lit.next();
            if (fruit.equals("Banana")) {
                lit.set("Mango");  // Replace
                lit.add("Cherry"); // Add after Mango
            }
        }

        System.out.println("Modified List: " + list);
    }
}
```

Features:

- Forward & backward navigation
- Add/Remove/Update elements
- Get index using `nextIndex()` and `previousIndex()`

Quick Comparison Table

Feature	Enumeration	Iterator	ListIterator
Package	`java.util`	`java.util`	`java.util`
Direction	Forward	Forward	Forward + Backward
Remove element	No	Yes	Yes
Add element	No	No	Yes
Modify element	No	No	Yes
Used in	Legacy (Vector, Hashtable)	All collections	Only List (ArrayList, LinkedList)

CHAPTER 50

Stack in Java

What is `Stack` in Java?

`Stack` is a **class** in Java that represents a **Last In First Out (LIFO)** data structure — the element that is added last is removed first.

Java's `Stack` is a **legacy class** and is a subclass of `Vector`.

Declaration and Syntax

```
import java.util.Stack;

Stack<Type> stackName = new Stack<>();
```

Example:

```
Stack<Integer> numbers = new Stack<>();
```

Key Features

Feature	Description
LIFO order	Last-in, first-out
Thread-safe	Inherited from `Vector` (synchronized methods)
Allows nulls	Yes
Part of	Legacy Java Collection Framework
Implements	`List` interface (indirectly via `Vector`)

Internal Working

- Internally uses a **Vector** (i.e., a dynamic array).
- Inherits all `Vector` methods.
- Adds 5 stack-specific methods:

- `push()`, `pop()`, `peek()`, `empty()`, `search()`

Stack Operations

Method	Description
`push(E e)`	Adds element to top of the stack
`pop()`	Removes and returns top element
`peek()`	Returns top element without removing
`empty()`	Returns `true` if stack is empty
`search(Object o)`	Returns 1-based position from top of stack, or -1

Example Program

```
import java.util.Stack;

public class StackExample {
    public static void main(String[] args) {
        Stack<String> books = new Stack<>();

        // Push elements
        books.push("Math");
        books.push("Physics");
        books.push("Chemistry");

        // Peek at top
        System.out.println("Top Book: " + books.peek());

        // Pop top
        books.pop();

        // Check if empty
        System.out.println("Is stack empty? " + books.empty());

        // Search
        System.out.println("Position of Math: " + books.search("Math"));

        // Print stack
        System.out.println("Stack: " + books);
    }
}
```

Stack Output Example

```
Top Book: Chemistry
Is stack empty? false
Position of Math: 2
Stack: [Math, Physics]
```

Real-World Use Cases

> *Example 1: Undo Operation in Text Editor*
>
> *Each action is pushed onto a stack; `undo` pops the latest action.*

> *Example 2: Web Browser Back Navigation*
>
> *Each visited page is pushed, and the back button pops the stack.*

> *Example 3: Expression Evaluation*
>
> *Used in converting/evaluating **postfix/prefix expressions**.*

Difference: `Stack` vs `Queue`

Feature	Stack	Queue
Order	LIFO	FIFO
Key Methods	`push()`, `pop()`	`add()`, `remove()`
Use Case	Undo, Back nav	Scheduling, Print jobs

Stack vs ArrayList vs LinkedList vs Vector

Feature	Stack	ArrayList	LinkedList	Vector
Order	LIFO	Insertion	Insertion	Insertion
Thread-safe	Yes	No	No	Yes
Backing Data	Vector	Array	Nodes	Array
Performance	Slower (sync)	Fast	Good for insert/delete	Slower (sync)
Preferred For	LIFO behavior	Random access	Insert/delete	Legacy multithreading

Limitations of Java `Stack`

- It's a **legacy class**.
- Thread-safe but **slower in single-threaded apps**.
- Modern alternative:

```
Deque<String> stack = new ArrayDeque<>();
```

Use `Deque` interface (with `ArrayDeque` or `LinkedList`) for **better performance** in modern Java.

```
Deque<String> stack = new ArrayDeque<>();
stack.push("Java");
stack.pop();
```

Interview Points

- `pop()` throws `EmptyStackException` if the stack is empty.
- `search()` returns 1-based index from top (not zero-based).
- Always use `peek()` to **safely check top** before `pop()`.

Summary

Feature	Stack
Type	LIFO
Thread-safe	Yes (legacy sync)
Methods	push(), pop(), peek(), empty(), search()
Null allowed	Yes
Alternatives	Deque / ArrayDeque (preferred)

CHAPTER 51

Queue in Java

Queue in Java

Definition

A **Queue** is a **First-In-First-Out (FIFO)** linear data structure where:

- **Elements are added** from the **rear (tail)**.
- **Elements are removed** from the **front (head)**.

It is part of the **Java Collections Framework**, declared in `java.util`.

Common Implementations

Class	Characteristics
`LinkedList`	General-purpose FIFO queue. Allows null elements.
`PriorityQueue`	Orders by priority instead of insertion.
`ArrayDeque`	Faster alternative to Stack and LinkedList.

Queue Syntax

```
Queue<String> queue = new LinkedList<>();
```

Queue Interface Methods

Method	Description
`add(e)`	Inserts element. Throws exception if queue is full.
`offer(e)`	Inserts element. Returns `false` if insertion fails.
`remove()`	Removes and returns head. Throws `NoSuchElementException` if empty.
`poll()`	Removes and returns head. Returns `null` if empty.
`element()`	Returns head without removing. Throws exception if empty.
`peek()`	Returns head without removing. Returns `null` if empty.
`isEmpty()`	Checks if queue is empty.

| `size()` | Returns number of elements. |
| `clear()` | Removes all elements. |

Queue Example

```
import java.util.*;

public class QueueDemo {
    public static void main(String[] args) {
        Queue<String> queue = new LinkedList<>();

        queue.add("A");
        queue.offer("B");
        queue.offer("C");

        System.out.println("Queue: " + queue);         // [A, B, C]
        System.out.println("Peek: " + queue.peek());   // A
        System.out.println("Removed: " + queue.remove()); // A
        System.out.println("Poll: " + queue.poll());   // B
        System.out.println("Now: " + queue);           // [C]
        System.out.println("Element: " + queue.element()); // C

        queue.clear();
        System.out.println("Is Empty? " + queue.isEmpty()); // true
    }
}
```

Method Pair	**Difference**
`add()` vs `offer()`	`add()` throws an exception (`IllegalStateException`) if the queue is full. `offer()` returns `false` instead of throwing an exception.
`remove()` vs `poll()`	`remove()` throws an exception (`NoSuchElementException`) if the queue is empty. `poll()` returns `null` if the queue is empty.
`element()` vs `peek()`	`element()` throws an exception if the queue is empty. `peek()` returns `null` if the queue is empty.

PriorityQueue in Java

Definition

A `PriorityQueue` is a **special queue** where:

- **Elements are ordered by priority**, not by insertion.
- **Smallest or highest-priority element** is always at the head.

> *Internally implemented using a **Min-Heap** (binary heap).*

Declaration

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

Characteristics

Feature	Value
Ordering	Natural order or custom comparator
Null allowed?	No (throws `NullPointerException`)
Thread-safe?	No
Duplicates?	Yes
Internal Structure	Min-Heap

Methods of PriorityQueue

Method	Description
`add(e)` / `offer(e)`	Inserts element (heap is restructured).
`poll()`	Removes and returns top-priority element.
`peek()`	Returns top element without removing.
`remove(Object o)`	Removes specific object.

`contains(Object o)`	Checks if element exists.
`size()`	Number of elements.
`clear()`	Removes all elements.

Basic Example

```
import java.util.PriorityQueue;

public class PriorityQueueExample {
    public static void main(String[] args) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();

        pq.add(40);
        pq.add(10);
        pq.add(30);
        pq.add(20);

        System.out.println("Top Element: " + pq.peek()); // 10

        while (!pq.isEmpty()) {
            System.out.println(pq.poll()); // 10, 20, 30, 40
        }
    }
}
```

PriorityQueue with Custom Comparator (Max-Heap)

```
PriorityQueue<Integer> maxPQ = new PriorityQueue<>((a, b) -> b - a);
maxPQ.add(10);
maxPQ.add(30);
maxPQ.add(20);
System.out.println(maxPQ.poll()); // 30 (highest first)
```

PriorityQueue with Custom Class

```
import java.util.*;

class Task {
    String name;
```

```
    int priority;

    Task(String name, int priority) {
        this.name = name;
        this.priority = priority;
    }

    public String toString() {
        return name + " (Priority: " + priority + ")";
    }
}

public class CustomPQ {
    public static void main(String[] args) {
        PriorityQueue<Task> taskQueue = new PriorityQueue<>(
            (t1, t2) -> t1.priority - t2.priority // Ascending priority
        );

        taskQueue.add(new Task("Low", 5));
        taskQueue.add(new Task("High", 1));
        taskQueue.add(new Task("Medium", 3));

        while (!taskQueue.isEmpty()) {
            System.out.println(taskQueue.poll());
        }
    }
}
```

Real-Life Use Cases

Use Case	How It Works
CPU Job Scheduling	Higher-priority jobs run first.
Dijkstra's Algorithm	Nodes with shortest distance first.
A* Pathfinding	Best node explored based on cost.
Customer Support Systems	VIP tickets handled before others.
Live Leaderboard Systems	Top scores maintained in heap.

PriorityQueue vs Other Collections

Feature	PriorityQueue	Queue (LinkedList)	Stack	ArrayList

Order	By priority	FIFO	LIFO	Insertion order
Null allowed	No	Yes	Yes	Yes
Thread-safe	No	No	(Legacy)	No
Random Access	No	No	No	Yes

Interview Highlights

- Uses **Min-Heap** by default.
- Custom comparator = **Max-Heap** possible.
- `peek()` = O(1), `add()` and `poll()` = O(log n).
- No nulls allowed.
- Used in many **algorithmic problems**.

Summary Table

Property	PriorityQueue
Order	Priority-based (Min-Heap)
Thread-safe	No
Null allowed	No
Duplicates allowed	Yes
Time Complexity	add/poll = O(log n), peek = O(1)

Deque and ArrayDeque in Java

Deque (Double Ended Queue)

Definition

A `Deque` (pronounced “deck”) is a **double-ended queue** that allows insertion and deletion at both **front and rear**.

> *It's an **interface** in the `java.util` package.*

Declaration

```
Deque<Integer> deque = new ArrayDeque<>();
```

> *Common implementation: `ArrayDeque`, `LinkedList`*

Characteristics of Deque

Feature	Value
Insertion/removal	Both ends (front and rear)
Null allowed?	No in `ArrayDeque`
Thread-safe?	No (use `ConcurrentLinkedDeque`)
Duplicates?	Yes
Interface	Deque (extends `Queue`)
Implementations	`ArrayDeque`, `LinkedList`

Key Methods of Deque

Method	Description
`addFirst(e)` / `offerFirst(e)`	Adds at front
`addLast(e)` / `offerLast(e)`	Adds at rear
`removeFirst()` / `pollFirst()`	Removes from front
`removeLast()` / `pollLast()`	Removes from rear
`getFirst()` / `peekFirst()`	Gets first element (no remove)
`getLast()` / `peekLast()`	Gets last element (no remove)
`contains(Object o)`	Checks if element exists
`size()`	Returns number of elements
`clear()`	Removes all elements

ArrayDeque in Java

Definition

`ArrayDeque` is a **resizable array implementation** of the `Deque` interface.

> *Faster than `Stack` and `LinkedList` for stack/queue operations.*

Declaration

```
ArrayDeque<String> stack = new ArrayDeque<>();
```

Characteristics of ArrayDeque

Feature	Value
Null allowed?	No (`NullPointerException`)
Thread-safe?	No
Implements	`Deque`, `Queue`
Faster than	`Stack` and `LinkedList`
Backed by	Resizable array (no capacity limit)

Basic Example – Deque as Queue

```
import java.util.*;

public class DequeAsQueue {
    public static void main(String[] args) {
        Deque<Integer> queue = new ArrayDeque<>();

        queue.offer(10); // addLast
        queue.offer(20);
        queue.offer(30);

        System.out.println("Front: " + queue.peek()); // 10

        while (!queue.isEmpty()) {
            System.out.println(queue.poll()); // 10 20 30
        }
    }
}
```

Basic Example – Deque as Stack

```
import java.util.*;

public class DequeAsStack {
    public static void main(String[] args) {
        Deque<String> stack = new ArrayDeque<>();

        stack.push("A"); // addFirst
        stack.push("B");
        stack.push("C");

        System.out.println("Top: " + stack.peek()); // C

        while (!stack.isEmpty()) {
            System.out.println(stack.pop()); // C B A
        }
    }
}
```

Real-Life Use Cases

Use Case	How It Works
Undo/Redo Operations	Stack behavior with push/pop
Palindrome Checking	Compare front and rear elements
Sliding Window Problems	Maintain window elements efficiently
Browser History	Navigate back and forward
Job Scheduling Queues	Add/remove tasks from both ends

Deque vs Queue vs Stack vs ArrayDeque

Feature	Deque	Queue	Stack	ArrayDeque
Direction	Both ends	FIFO	LIFO	Both (Deque impl)
Null allowed?	(ArrayDeque)	(LinkedList)		
Thread-safe?		(Legacy)		
Performance	Better than Stack/LinkedList			
Random Access				

Interview Highlights

- `Deque` allows **double-ended** operations.
- `ArrayDeque` is **faster** than `Stack`/`LinkedList`.
- No `null` elements in `ArrayDeque`.
- Used for **stack and queue** operations.
- Can be used for **sliding window**, **palindrome**, and **LIFO/FIFO** needs.

Summary Table

Property	ArrayDeque
Implements	`Deque`, `Queue`
Backed by	Array (Resizable)
Null Allowed	No
Thread-Safe	No
Use as Stack	Yes (`push()`, `pop()`)
Use as Queue	Yes (`offer()`, `poll()`)

| Performance | Fast for insertion/removal ends |

CHAPTER 54

Set in Java

`Set` in Java

What is `Set` in Java?

- `Set` is a **collection** that **does not allow duplicate elements**.
- It is part of the **Java Collection Framework** and is an **interface** under the `java.util` package.
- Sets are mainly used when **uniqueness** of elements is required.

Set Declaration & Initialization

```
Set<String> set = new HashSet<>();
Set<Integer> set = new LinkedHashSet<>();
Set<Double> set = new TreeSet<>();
```

Key Properties of Set

Feature	Description
Duplicates	Not allowed
Ordering	Depends on implementation (`HashSet`, `LinkedHashSet`, `TreeSet`)
Null Values	Allowed (HashSet, LinkedHashSet – 1 null; TreeSet – No nulls allowed)
Performance	HashSet > LinkedHashSet > TreeSet (in general)

Types of Set in Java

1. HashSet

- Uses **hashing**.
- **No order is guaranteed**.

- Allows **only one null**.

```
Set<String> hs = new HashSet<>();
hs.add("Apple");
hs.add("Banana");
hs.add("Apple"); // Duplicate ignored
```

2. LinkedHashSet

- Maintains **insertion order**.
- Slower than `HashSet`, but predictable.

```
Set<String> lhs = new LinkedHashSet<>();
lhs.add("Dog");
lhs.add("Cat");
lhs.add("Dog"); // Duplicate ignored
```

3. TreeSet

- Implements **SortedSet**.
- Stores elements in **natural sorted order** (ascending).
- Does **not allow nulls**.

```
Set<Integer> ts = new TreeSet<>();
ts.add(50);
ts.add(20);
ts.add(30); // Output: 20, 30, 50
```

Common Set Methods

Method	Description
`add(E e)`	Adds element
`addAll(Collection c)`	Adds all elements from another collection
`remove(Object o)`	Removes an element
`clear()`	Clears the set
`contains(Object o)`	Checks if element exists
`isEmpty()`	Checks if the set is empty
`size()`	Returns number of elements
`iterator()`	Returns an iterator
`retainAll(Collection c)`	Keeps only common elements

| `removeAll(Collection c)` | Removes all common elements |

Iterating over a Set

1. Using for-each loop

```
for (String s : set) {
    System.out.println(s);
}
```

2. Using Iterator

```
Iterator<String> itr = set.iterator();
while (itr.hasNext()) {
    System.out.println(itr.next());
}
```

Set vs List vs Map

Feature	Set	List	Map
Duplicates	Not allowed	Allowed	(keys) (values)
Order	Depends on type	Maintains order	Depends on implementation
Null values	1 or (depends)		One null key, multiple values

Use Cases of Set

- Storing **unique student IDs**
- Removing duplicates from a list
- Checking **membership** quickly
- Maintaining **natural order** (using `TreeSet`)

Set Example – Complete Program

```java
import java.util.*;

public class SetExample {
    public static void main(String[] args) {
        Set<String> fruits = new HashSet<>();
        fruits.add("Mango");
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Apple");  // Duplicate ignored

        System.out.println("Fruits: " + fruits);
    }
}
```

Summary Table

Set Type	Ordered?	Duplicates	Null	Sorted?	Best Use
`HashSet`	No order	No	One		Fast lookup, unique values
`LinkedHashSet`	Insertion	No	One		Unique values + maintain order
`TreeSet`	Sorted	No	No	Ascending	Unique + sorted data

Let me know if you'd like a **Set vs List vs Map chart**, **Set interview questions**, or **practice problems with solutions**.

CHAPTER 55

HashSet in Java

What is `HashSet` in Java?

`HashSet` is a part of the **Java Collection Framework** and implements the `Set` interface. It represents a **collection of unique elements** and is **backed by a `HashMap`** internally.

It is found in the `java.util` package.

Declaration and Syntax

```
import java.util.HashSet;

HashSet<Type> setName = new HashSet<>();
```

Example:

```
HashSet<String> countries = new HashSet<>();
```

Key Features

Feature	Description
No duplicates	Stores only unique elements.
Unordered	No guaranteed order of elements.
Allows null	Allows a single null element.
Backed by HashMap	Internally uses a `HashMap` for storage.
Fast operations	Offers constant time for basic operations like `add`, `remove`.
Not synchronized	Not thread-safe by default.

Internal Working

- Internally uses a **`HashMap`**.
- When you add an element to `HashSet`, it's stored as a key in the map with a dummy value.

- Uses **hashing** to store and retrieve elements.

Commonly Used Methods

Method	Description
`add(E e)`	Adds an element if it's not already present.
`remove(Object o)`	Removes the specified element.
`contains(Object o)`	Checks if element exists.
`isEmpty()`	Checks if set is empty.
`size()`	Returns number of elements.
`clear()`	Removes all elements.
`iterator()`	Returns an iterator over the set.
`addAll(Collection)`	Adds all elements from another collection.
`removeAll(Collection)`	Removes all matching elements.
`retainAll(Collection)`	Retains only common elements.
`toArray()`	Converts set to array.
`clone()`	Returns a shallow copy of the set.

Example Program

```
import java.util.*;

public class HashSetExample {
    public static void main(String[] args) {
        HashSet<String> fruits = new HashSet<>();

        // Add elements
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Mango");
        fruits.add("Apple"); // Duplicate, ignored

        // Check size and content
        System.out.println("Fruits: " + fruits);
        System.out.println("Contains Mango? " + fruits.contains("Mango"));

        // Remove element
        fruits.remove("Banana");

        // Iterate using enhanced for-loop
```

```
        for (String fruit : fruits) {
            System.out.println("Fruit: " + fruit);
        }

        // Clear set
        fruits.clear();
        System.out.println("Is set empty? " + fruits.isEmpty());
    }
}
```

Iterating Over `HashSet`

1. Enhanced For Loop:

```
for (String item : set) {
    System.out.println(item);
}
```

2. Iterator:

```
Iterator<String> itr = set.iterator();
while (itr.hasNext()) {
    System.out.println(itr.next());
}
```

Real-World Use Case

> ***Example: Storing unique visitor IP addresses to a website***

```
HashSet<String> visitorIPs = new HashSet<>();
visitorIPs.add("192.168.0.1");
visitorIPs.add("192.168.0.2");
visitorIPs.add("192.168.0.1"); // Duplicate won't be added
System.out.println("Unique visitors: " + visitorIPs.size());
```

Difference Between `HashSet` and `ArrayList`

Feature	`ArrayList`	`HashSet`
Order	Maintains insertion order	No order guaranteed
Duplicates	Allows duplicates	Does **not** allow duplicates
Access	Indexed access	No indexing, only element-based
Performance	Slower for search	Faster for search using hashing
Underlying DS	Resizable array	HashMap (keys only)

Thread Safety

- `HashSet` is **not synchronized**.
- Use `Collections.synchronizedSet()` for thread-safe operations.

```
Set<String> syncSet = Collections.synchronizedSet(new HashSet<>());
```

Generic Type in `HashSet`

```
HashSet<Integer> numbers = new HashSet<>();
numbers.add(100); // Only Integer allowed
```

Convert `HashSet` to Array

```
String[] arr = set.toArray(new String[0]);
```

Interview Point

- **Time Complexity:**
  - `add()` → O(1) average
  - `remove()` → O(1) average
  - `contains()` → O(1) average

Full Java Program Using Major `HashSet` Methods

```
import java.util.*;

public class HashSetMethodsDemo {
    public static void main(String[] args) {
        HashSet<String> set = new HashSet<>();

        // 1. add()
        set.add("A");
        set.add("B");
        set.add("C");

        // 2. add() duplicate
        set.add("A");

        // 3. contains()
        System.out.println("Contains B? " + set.contains("B"));

        // 4. remove()
        set.remove("C");

        // 5. size()
        System.out.println("Size: " + set.size());

        // 6. isEmpty()
        System.out.println("Is empty? " + set.isEmpty());

        // 7. iterator()
        Iterator<String> itr = set.iterator();
        while (itr.hasNext()) {
            System.out.print(itr.next() + " ");
        }

        // 8. addAll()
        HashSet<String> newSet = new HashSet<>();
        newSet.add("D");
        newSet.add("E");
        set.addAll(newSet);
        System.out.println("\nAfter addAll: " + set);

        // 9. retainAll()
        set.retainAll(Arrays.asList("A", "E"));
        System.out.println("After retainAll: " + set);

        // 10. removeAll()
        set.removeAll(Arrays.asList("A"));
        System.out.println("After removeAll: " + set);

        // 11. toArray()
```

```
        Object[] array = set.toArray();
        System.out.println("Array: " + Arrays.toString(array));

        // 12. clone()
        HashSet<String> cloned = (HashSet<String>) set.clone();
        System.out.println("Cloned: " + cloned);

        // 13. clear()
        set.clear();
        System.out.println("After clear, isEmpty? " + set.isEmpty());
    }
}
```

Summary of Methods Covered

- **CRUD**: `add()`, `remove()`, `clear()`
- **Search**: `contains()`, `size()`, `isEmpty()`
- **Bulk Ops**: `addAll()`, `removeAll()`, `retainAll()`
- **Utilities**: `iterator()`, `toArray()`, `clone()`

CHAPTER 56

LinkedHashSet in Java

What is `LinkedHashSet` in Java?

`LinkedHashSet` is a class in the **Java Collection Framework** that implements the `Set` interface and maintains a **linked list of the entries** in the set, preserving the **insertion order**.

It is a **hybrid** of `HashSet` (for uniqueness and hashing) and a **LinkedList-like structure** (for maintaining order).

Declaration and Syntax

```
import java.util.LinkedHashSet;

LinkedHashSet<Type> setName = new LinkedHashSet<>();
```

Example:

```
LinkedHashSet<String> cities = new LinkedHashSet<>();
```

Key Features

Feature	Description
Unique elements	Does not allow duplicate elements.
Preserves order	Maintains the **insertion order** of elements.
Allows null	Allows one `null` value.
Hashing + Linking	Combines **hash table** for speed and **linked list** for order.
Not synchronized	Not thread-safe by default.

Internal Working

- Internally backed by a **LinkedHashMap**.
- Maintains a **doubly-linked list** to preserve insertion order.
- Each element is stored as a **key** in the map with a dummy value.

Commonly Used Methods

Method	Description
`add(E e)`	Adds element if not already present
`remove(Object o)`	Removes the specified element
`contains(Object o)`	Checks if element exists
`isEmpty()`	Checks if set is empty
`size()`	Returns number of elements
`clear()`	Removes all elements
`iterator()`	Returns iterator in insertion order
`addAll(Collection)`	Adds all elements from another collection
`removeAll(Collection)`	Removes all matching elements
`retainAll(Collection)`	Keeps only common elements
`toArray()`	Converts set to array
`clone()`	Returns shallow copy

Example Program

```java
import java.util.*;

public class LinkedHashSetExample {
    public static void main(String[] args) {
        LinkedHashSet<String> animals = new LinkedHashSet<>();

        animals.add("Dog");
        animals.add("Cat");
        animals.add("Horse");
        animals.add("Dog"); // duplicate ignored

        System.out.println("Animals: " + animals); // Maintains order

        animals.remove("Cat");
        System.out.println("After removal: " + animals);

        System.out.println("Contains Horse? " + animals.contains("Horse"));
        System.out.println("Is Empty? " + animals.isEmpty());
        System.out.println("Size: " + animals.size());

        animals.clear();
        System.out.println("After clear: " + animals);
    }
}
```

Iterating Over `LinkedHashSet`

1. Enhanced For Loop:

```
for (String item : set) {
    System.out.println(item);
}
```

2. Iterator:

```
Iterator<String> itr = set.iterator();
while (itr.hasNext()) {
    System.out.println(itr.next());
}
```

Real-World Use Case

> ***Example: Preserving unique student names in registration order***

```
LinkedHashSet<String> students = new LinkedHashSet<>();
students.add("Alice");
students.add("Bob");
students.add("Charlie");
students.add("Bob"); // ignored
System.out.println("Registered Students: " + students);
```

Difference Between `HashSet` and `LinkedHashSet`

Feature	`HashSet`	`LinkedHashSet`
Order	Unordered	Maintains **insertion order**
Performance	Slightly faster	Slightly slower (due to linking)
Internal DS	`HashMap`	`LinkedHashMap`
Use case	Fast lookup	Fast lookup + predictable order

Thread Safety

- `LinkedHashSet` is **not synchronized**.
- Use thread-safe version as:

```
Set<String> syncSet = Collections.synchronizedSet(new LinkedHashSet<>());
```

Generic Type in `LinkedHashSet`

```
LinkedHashSet<Integer> numbers = new LinkedHashSet<>();
numbers.add(10); // Only Integers allowed
```

Convert `LinkedHashSet` to Array

```
String[] arr = set.toArray(new String[0]);
```

Interview Point

- **Time Complexity:**
  - `add()` → O(1)
  - `contains()` → O(1)
  - `remove()` → O(1)
  - Maintains **predictable iteration order**

Full Java Program Using Major `LinkedHashSet` Methods

```
import java.util.*;

public class LinkedHashSetMethodsDemo {
    public static void main(String[] args) {
        LinkedHashSet<String> set = new LinkedHashSet<>();

        // 1. add()
```

```
        set.add("One");
        set.add("Two");
        set.add("Three");

        // 2. Duplicate
        set.add("One");

        // 3. contains()
        System.out.println("Contains 'Two'? " + set.contains("Two"));

        // 4. remove()
        set.remove("Three");

        // 5. size()
        System.out.println("Size: " + set.size());

        // 6. isEmpty()
        System.out.println("Is empty? " + set.isEmpty());

        // 7. iterator()
        Iterator<String> itr = set.iterator();
        while (itr.hasNext()) {
            System.out.print(itr.next() + " ");
        }

        // 8. addAll()
        LinkedHashSet<String> newSet = new LinkedHashSet<>();
        newSet.add("Four");
        newSet.add("Five");
        set.addAll(newSet);
        System.out.println("\nAfter addAll: " + set);

        // 9. retainAll()
        set.retainAll(Arrays.asList("One", "Five"));
        System.out.println("After retainAll: " + set);

        // 10. removeAll()
        set.removeAll(Arrays.asList("One"));
        System.out.println("After removeAll: " + set);

        // 11. toArray()
        Object[] array = set.toArray();
        System.out.println("Array: " + Arrays.toString(array));

        // 12. clone()
        LinkedHashSet<String> cloned = (LinkedHashSet<String>) set.clone();
        System.out.println("Cloned: " + cloned);

        // 13. clear()
        set.clear();
        System.out.println("After clear, isEmpty? " + set.isEmpty());
    }
```

```
}
```

Summary of Methods Covered

- **CRUD**: `add()`, `remove()`, `clear()`
- **Search**: `contains()`, `size()`, `isEmpty()`
- **Bulk Ops**: `addAll()`, `removeAll()`, `retainAll()`
- **Utilities**: `iterator()`, `toArray()`, `clone()`

CHAPTER 57

TreeSet in Java

What is `TreeSet` in Java?

`TreeSet` is a part of the **Java Collection Framework** that implements the `NavigableSet` interface (which extends `SortedSet`), and stores **unique elements in sorted (ascending) order**.

It uses a **Red-Black Tree** (a self-balancing binary search tree) internally to maintain the order.

Declaration and Syntax

```
import java.util.TreeSet;

TreeSet<Type> setName = new TreeSet<>();
```

Example:

```
TreeSet<Integer> numbers = new TreeSet<>();
```

Key Features

Feature	Description
Unique elements	Does not allow duplicates.
Sorted order	Elements are stored in **ascending order** by default.
No null	Does **not allow null** (throws `NullPointerException`).
Backed by Tree	Internally uses a **Red-Black Tree** (balanced BST).
NavigableSet methods	Supports methods like `higher()`, `lower()`, `ceiling()`.
Not synchronized	Not thread-safe by default.

Internal Working

- Implements **Self-Balancing Red-Black Tree**.
- Elements are **sorted automatically** based on **natural ordering** or **custom comparator**.

- Maintains log(n) time for most operations.

Commonly Used Methods

Method	Description
`add(E e)`	Adds an element in sorted position
`remove(Object o)`	Removes specified element
`contains(Object o)`	Checks for presence
`isEmpty()`	Checks if empty
`size()`	Returns element count
`clear()`	Removes all elements
`first()`	Returns lowest element
`last()`	Returns highest element
`higher(E e)`	Element strictly greater than given
`lower(E e)`	Element strictly less than given
`ceiling(E e)`	Greater than or equal to
`floor(E e)`	Less than or equal to
`pollFirst()`	Retrieves and removes the first element
`pollLast()`	Retrieves and removes the last element
`iterator()`	Returns iterator in ascending order
`descendingIterator()`	Returns iterator in descending order

Example Program

```
import java.util.*;

public class TreeSetExample {
    public static void main(String[] args) {
        TreeSet<Integer> ts = new TreeSet<>();

        ts.add(50);
        ts.add(20);
        ts.add(30);
        ts.add(10);
        ts.add(50); // Duplicate ignored

        System.out.println("TreeSet: " + ts); // Sorted order

        ts.remove(20);
```

```
        System.out.println("After removal: " + ts);

        System.out.println("First: " + ts.first());
        System.out.println("Last: " + ts.last());
        System.out.println("Higher than 30: " + ts.higher(30));
        System.out.println("Lower than 30: " + ts.lower(30));
    }
}
```

Iterating Over `TreeSet`

1. Enhanced For Loop:

```
for (int num : treeSet) {
    System.out.println(num);
}
```

2. Iterator:

```
Iterator<Integer> itr = treeSet.iterator();
while (itr.hasNext()) {
    System.out.println(itr.next());
}
```

3. Descending Iterator:

```
Iterator<Integer> descItr = treeSet.descendingIterator();
while (descItr.hasNext()) {
    System.out.println(descItr.next());
}
```

Real-World Use Case

> ***Example: Ranking system that auto-sorts scores***

```
TreeSet<Integer> scores = new TreeSet<>();
scores.add(78);
scores.add(95);
scores.add(88);
```

```
System.out.println("Top score: " + scores.last());
System.out.println("2nd highest: " + scores.lower(scores.last()));
```

Difference Between `HashSet`, `LinkedHashSet`, and `TreeSet`

Feature	`HashSet`	`LinkedHashSet`	`TreeSet`
Order	No order	Maintains insertion	Sorted (natural/custom order)
Performance	Fastest	Slower than `HashSet`	Slowest (uses Red-Black Tree)
Allows null	One null allowed	One null allowed	Throws exception
Duplicate values	Not allowed	Not allowed	Not allowed
Underlying DS	`HashMap`	`LinkedHashMap`	`TreeMap` (Red-Black Tree)
Use case	Unordered fast set	Ordered fast set	Sorted set

Thread Safety

- `TreeSet` is **not synchronized**.
- Use this for thread-safe version:

```
Set<Integer> syncSet = Collections.synchronizedSet(new TreeSet<>());
```

Custom Sorting Using Comparator

```
TreeSet<String> set = new TreeSet<>(Comparator.reverseOrder());
set.add("Banana");
set.add("Apple");
set.add("Mango");
System.out.println("Custom sorted set: " + set);
```

Interview Point

- **Time Complexity**:
  - `add()` → O(log n)

- `remove()` → O(log n)
- `contains()` → O(log n)

- Does not allow `null` elements.

Full Java Program Using Major `TreeSet` Methods

```
import java.util.*;

public class TreeSetMethodsDemo {
    public static void main(String[] args) {
        TreeSet<Integer> set = new TreeSet<>();

        // 1. add()
        set.add(40);
        set.add(10);
        set.add(30);
        set.add(20);

        // 2. contains()
        System.out.println("Contains 30? " + set.contains(30));

        // 3. remove()
        set.remove(10);

        // 4. first(), last()
        System.out.println("First: " + set.first());
        System.out.println("Last: " + set.last());

        // 5. ceiling(), floor()
        System.out.println("Ceiling of 25: " + set.ceiling(25)); // 30
        System.out.println("Floor of 25: " + set.floor(25));     // 20

        // 6. higher(), lower()
        System.out.println("Higher than 20: " + set.higher(20)); // 30
        System.out.println("Lower than 20: " + set.lower(20));   // null

        // 7. pollFirst(), pollLast()
        System.out.println("Poll First: " + set.pollFirst()); // 20
        System.out.println("Poll Last: " + set.pollLast());   // 40

        // 8. iterator()
        System.out.print("Ascending: ");
        Iterator<Integer> itr = set.iterator();
        while (itr.hasNext()) {
            System.out.print(itr.next() + " ");
        }
```

```
        // 9. descendingIterator()
        System.out.print("\nDescending: ");
        Iterator<Integer> descItr = set.descendingIterator();
        while (descItr.hasNext()) {
            System.out.print(descItr.next() + " ");
        }

        // 10. clear()
        set.clear();
        System.out.println("\nSet after clear: " + set);
    }
}
```

Summary of Methods Covered

- **CRUD**: `add()`, `remove()`, `clear()`
- **Search**: `contains()`, `first()`, `last()`
- **Navigation**: `higher()`, `lower()`, `ceiling()`, `floor()`, `pollFirst()`, `pollLast()`
- **Iteration**: `iterator()`, `descendingIterator()`
- **Custom Comparator Support**

CHAPTER 58

Map in Java

What is `Map` in Java?

`Map` is a part of the **Java Collection Framework** that stores **key-value pairs**. Unlike other collections, a `Map` does **not allow duplicate keys**, but **values can be duplicated**.

It is found in the `java.util` package and is an **interface** implemented by classes like `HashMap`, `LinkedHashMap`, `TreeMap`, etc.

Declaration and Syntax

```
import java.util.Map;
import java.util.HashMap;

Map<KeyType, ValueType> mapName = new HashMap<>();
```

Example:

```
Map<Integer, String> studentMap = new HashMap<>();
```

Key Features

Feature	Description
Key-value structure	Each entry contains a unique key and a value.
Unique keys	Keys must be unique.
Allows nulls	One null key and multiple null values allowed (in `HashMap`).
Various implementations	Like `HashMap`, `LinkedHashMap`, `TreeMap`
Not synchronized	Not thread-safe by default.

Commonly Used Methods

| Method | Description |

`put(K key, V value)`	Inserts or updates key-value pair
`get(Object key)`	Returns value for given key
`remove(Object key)`	Removes mapping by key
`containsKey(Object key)`	Checks if key exists
`containsValue(Object value)`	Checks if value exists
`size()`	Number of entries
`isEmpty()`	Checks if map is empty
`clear()`	Removes all entries
`keySet()`	Returns `Set` of keys
`values()`	Returns `Collection` of values
`entrySet()`	Returns `Set` of key-value pairs
`putAll(Map)`	Copies all mappings from another map
`getOrDefault(Object key, V defaultValue)`	Returns value or default if not found

Example Program

```
import java.util.*;

public class MapExample {
    public static void main(String[] args) {
        Map<Integer, String> map = new HashMap<>();

        // Add entries
        map.put(1, "Apple");
        map.put(2, "Banana");
        map.put(3, "Cherry");

        // Access
        System.out.println("Value for key 2: " + map.get(2));

        // Update value
        map.put(2, "Blueberry");

        // Remove key
        map.remove(3);

        // Check
        System.out.println("Contains key 1? " + map.containsKey(1));
        System.out.println("Contains value 'Apple'? " + map.containsValue("Apple"));

        // Iterate over entries
        for (Map.Entry<Integer, String> entry : map.entrySet()) {
            System.out.println("Key: " + entry.getKey() + ", Value: " + entry.getValue());
        }
```

```
        // Print all keys and values
        System.out.println("Keys: " + map.keySet());
        System.out.println("Values: " + map.values());
    }
}
```

Iterating Over `Map`

1. Using `entrySet()`

```
for (Map.Entry<Integer, String> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
```

2. Using `keySet()`

```
for (Integer key : map.keySet()) {
    System.out.println("Key: " + key + ", Value: " + map.get(key));
}
```

3. Using `forEach()` (Java 8+)

```
map.forEach((key, value) -> System.out.println(key + " = " + value));
```

Real-World Use Case

> ***Example: Storing student ID and name mapping***

```
Map<Integer, String> studentMap = new HashMap<>();
studentMap.put(101, "Alice");
studentMap.put(102, "Bob");
studentMap.put(103, "Charlie");

System.out.println("Student 102: " + studentMap.get(102));
```

Difference Between `Map`, `HashMap`, `LinkedHashMap`, `TreeMap`

Feature	`Map` (Interface)	`HashMap`	`LinkedHashMap`	`TreeMap`
Order	N/A	No order	Insertion order	Sorted by keys
Null Keys	N/A	1 null key allowed	1 null key allowed	No null key allowed
Performance	N/A	Fastest	Slightly slower	Slowest (uses BST)
Synchronization	N/A	Not synchronized	Not synchronized	Not synchronized
Underlying DS	N/A	Hash table	Hash table + LinkedList	Red-Black Tree (SortedMap)

Thread Safety

- `Map` and its implementations are **not synchronized** by default.
- Use `Collections.synchronizedMap()`:

```
Map<String, Integer> syncMap = Collections.synchronizedMap(new HashMap<>());
```

- Or use **ConcurrentHashMap** for thread-safe and performance-friendly operations.

Interview Point

- **Map** is not part of the `Collection` interface hierarchy.
- Keys are **unique**, values can be **duplicated**.
- Order depends on implementation.

Full Java Program Using All Major `Map` Methods

```
import java.util.*;

public class MapMethodsDemo {
    public static void main(String[] args) {
        Map<String, Integer> map = new HashMap<>();

        // 1. put()
        map.put("A", 10);
        map.put("B", 20);
        map.put("C", 30);
```

```
        // 2. get()
        System.out.println("Value for 'B': " + map.get("B"));

        // 3. getOrDefault()
        System.out.println("Value for 'D': " + map.getOrDefault("D", 0));

        // 4. containsKey / containsValue
        System.out.println("Has key 'A'? " + map.containsKey("A"));
        System.out.println("Has value 40? " + map.containsValue(40));

        // 5. size / isEmpty
        System.out.println("Size: " + map.size());
        System.out.println("Is empty? " + map.isEmpty());

        // 6. remove()
        map.remove("C");

        // 7. putAll()
        Map<String, Integer> newMap = new HashMap<>();
        newMap.put("X", 100);
        newMap.put("Y", 200);
        map.putAll(newMap);

        // 8. keySet()
        System.out.println("Keys: " + map.keySet());

        // 9. values()
        System.out.println("Values: " + map.values());

        // 10. entrySet()
        for (Map.Entry<String, Integer> entry : map.entrySet()) {
            System.out.println(entry.getKey() + " => " + entry.getValue());
        }

        // 11. forEach
        map.forEach((k, v) -> System.out.println("Key: " + k + ", Value: " + v));

        // 12. clear()
        map.clear();
        System.out.println("Map after clear: " + map);
    }
}
```

Summary of Methods Covered

- **CRUD**: `put()`, `get()`, `remove()`, `clear()`
- **Search**: `containsKey()`, `containsValue()`, `getOrDefault()`
- **Bulk Ops**: `putAll()`, `keySet()`, `values()`, `entrySet()`

- **Utilities**: `size()`, `isEmpty()`, `forEach()`

CHAPTER 59

Hashtable in Java

What is `Hashtable` in Java?

`Hashtable` is a part of the **Java Collection Framework** that implements the `Map` interface and stores data in **key-value pairs**. It is similar to `HashMap` but is **synchronized** and **thread-safe**.

It is found in the `java.util` package and was one of the **original legacy classes** in Java before `Map` was introduced.

Declaration and Syntax

```
import java.util.Hashtable;

Hashtable<KeyType, ValueType> table = new Hashtable<>();
```

Example:

```
Hashtable<Integer, String> ht = new Hashtable<>();
```

Key Features

Feature	Description
Key-value pairs	Stores entries in (key, value) format
No null	Does **not allow null keys or null values**
Thread-safe	Synchronized methods (safe for multi-threaded apps)
Unordered	No guarantee of any order (like `HashMap`)
Legacy class	Exists before Java 1.2 Collections framework

Commonly Used Methods

Method	Description

`put(K key, V value)`	Adds/updates a key-value pair
`get(Object key)`	Gets value associated with the key
`remove(Object key)`	Removes entry by key
`containsKey(Object key)`	Checks if a key exists
`containsValue(Object value)`	Checks if a value exists
`isEmpty()`	Checks if the table is empty
`size()`	Returns number of key-value pairs
`clear()`	Removes all mappings
`keySet()`	Returns set of all keys
`values()`	Returns collection of all values
`entrySet()`	Returns set of key-value pairs
`elements()`	Returns values as Enumeration (legacy)
`keys()`	Returns keys as Enumeration (legacy)

Example Program

```
import java.util.Hashtable;

public class HashtableExample {
    public static void main(String[] args) {
        Hashtable<Integer, String> ht = new Hashtable<>();

        // Add entries
        ht.put(1, "Apple");
        ht.put(2, "Banana");
        ht.put(3, "Cherry");

        System.out.println("Hashtable: " + ht);

        // Get value by key
        System.out.println("Value for key 2: " + ht.get(2));

        // Check for key and value
        System.out.println("Contains key 3? " + ht.containsKey(3));
        System.out.println("Contains value 'Banana'? " + ht.containsValue("Banana"));

        // Remove an entry
        ht.remove(1);

        System.out.println("After removal: " + ht);
    }
}
```

Iterating Over `Hashtable`

1. Using `entrySet()`

```
for (Map.Entry<Integer, String> entry : ht.entrySet()) {
    System.out.println(entry.getKey() + " => " + entry.getValue());
}
```

2. Using `Enumeration` (Legacy way)

```
Enumeration<Integer> keys = ht.keys();
while (keys.hasMoreElements()) {
    Integer key = keys.nextElement();
    System.out.println("Key: " + key + ", Value: " + ht.get(key));
}
```

Real-World Use Case

> ***Example: Storing configuration properties in a multithreaded application***

```
Hashtable<String, String> config = new Hashtable<>();
config.put("url", "http://example.com");
config.put("timeout", "5000");
System.out.println("App URL: " + config.get("url"));
```

Difference Between `HashMap` and `Hashtable`

Feature	`HashMap`	`Hashtable`
Thread-safe	Not thread-safe	Thread-safe (synchronized)
Null key/value	1 null key, many null values	No null key or value allowed
Performance	Faster (non-synchronized)	Slower (due to synchronization)
Legacy	Modern (`Map` introduced in 1.2)	Legacy (pre-Java 1.2)
Fail-fast iterator	Throws `ConcurrentModificationException`	Fail-safe iterator (not guaranteed)

Thread Safety

- All methods in `Hashtable` are **synchronized**, making it safe for use in multithreaded environments.
- However, it is recommended to use `ConcurrentHashMap` in modern apps due to better performance.

Full Java Program Using Major `Hashtable` Methods

```
import java.util.*;

public class HashtableMethodsDemo {
    public static void main(String[] args) {
        Hashtable<String, Integer> table = new Hashtable<>();

        // 1. put()
        table.put("A", 10);
        table.put("B", 20);
        table.put("C", 30);

        // 2. get()
        System.out.println("Value of B: " + table.get("B"));

        // 3. containsKey(), containsValue()
        System.out.println("Has key 'A'? " + table.containsKey("A"));
        System.out.println("Has value 30? " + table.containsValue(30));

        // 4. remove()
        table.remove("C");

        // 5. size(), isEmpty()
        System.out.println("Size: " + table.size());
        System.out.println("Is empty? " + table.isEmpty());

        // 6. keySet(), values(), entrySet()
        System.out.println("Keys: " + table.keySet());
        System.out.println("Values: " + table.values());

        // 7. Enumeration (keys and elements)
        Enumeration<String> keys = table.keys();
        while (keys.hasMoreElements()) {
            String key = keys.nextElement();
            System.out.println("Key: " + key + ", Value: " + table.get(key));
        }

        // 8. clear()
        table.clear();
        System.out.println("After clear: " + table);
    }
```

```
}
```

Summary of Methods Covered

- **CRUD**: `put()`, `get()`, `remove()`, `clear()`
- **Check**: `containsKey()`, `containsValue()`, `isEmpty()`, `size()`
- **Views**: `keySet()`, `values()`, `entrySet()`
- **Legacy Enumeration**: `keys()`, `elements()`

CHAPTER 60

HashMap in Java

What is `HashMap` in Java?

`HashMap` is a part of the **Java Collection Framework** that implements the `Map` interface and stores data as **key-value pairs**. It allows **one null key**, multiple null values, and provides **constant-time performance** for basic operations like `get()` and `put()`.

It is found in the `java.util` package.

Declaration and Syntax

```
import java.util.HashMap;

HashMap<KeyType, ValueType> map = new HashMap<>();
```

Example:

```
HashMap<Integer, String> map = new HashMap<>();
```

Key Features

Feature	Description
Key-Value pairs	Stores data as `(key, value)`
Unique Keys	Keys must be unique
Null Allowed	1 null key, multiple null values
Unordered	No ordering guarantee
Fast Performance	`O(1)` time for basic operations
Not Thread-Safe	Not synchronized
Backed by	Uses **Hash Table + Linked List** (or Tree in JDK 8+)

Internal Working

- Internally uses an **array of buckets**.
- Each bucket uses **Linked List** (or Tree structure for high collisions).
- **Hashing** is used to compute index from the key's `hashCode()`.
- If many elements fall into the same bucket, a **Linked List** or **Tree** is formed.

Commonly Used Methods

Method	Description
`put(K key, V value)`	Inserts or updates a key-value pair
`get(Object key)`	Gets value for the key
`remove(Object key)`	Removes the key-value pair
`containsKey(Object key)`	Checks if a key exists
`containsValue(Object value)`	Checks if a value exists
`isEmpty()`	Returns true if map is empty
`size()`	Returns the number of key-value pairs
`clear()`	Clears all entries
`keySet()`	Returns `Set` of keys
`values()`	Returns `Collection` of values
`entrySet()`	Returns `Set<Map.Entry>` for iteration
`putIfAbsent(K key, V value)`	Adds only if key is not present
`getOrDefault(key, default)`	Gets value or default if key doesn't exist
`replace(K key, V newValue)`	Replaces value for the given key

Example Program

```
import java.util.HashMap;

public class HashMapExample {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>();

        map.put(1, "Apple");
        map.put(2, "Banana");
        map.put(3, "Cherry");
        map.put(null, "NullKeyAllowed");

        System.out.println("Map: " + map);
        System.out.println("Value for key 2: " + map.get(2));
```

```
        map.put(2, "Blueberry"); // update
        map.remove(3);           // remove entry

        System.out.println("After update & removal: " + map);
        System.out.println("Contains key 1? " + map.containsKey(1));
    }
}
```

Iterating Over `HashMap`

1. Using `entrySet()`:

```
for (Map.Entry<Integer, String> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " => " + entry.getValue());
}
```

2. Using `keySet()`:

```
for (Integer key : map.keySet()) {
    System.out.println("Key: " + key + ", Value: " + map.get(key));
}
```

3. Using `forEach()` (Java 8+):

```
map.forEach((key, value) -> System.out.println(key + " = " + value));
```

Real-World Use Case

> ***Example: Student Roll Numbers and Names***

```
HashMap<Integer, String> studentMap = new HashMap<>();
studentMap.put(101, "Alice");
studentMap.put(102, "Bob");
studentMap.put(103, "Charlie");

System.out.println("Student 102: " + studentMap.get(102));
```

Difference Between `HashMap` vs `Hashtable`

Feature	`HashMap`	`Hashtable`
Thread-safe	Not thread-safe	Thread-safe (synchronized)
Null support	1 null key, many null values	No null key or value allowed
Performance	Faster	Slower (due to locking)
Legacy	Modern class	Legacy class (pre-Java 1.2)
Fail-fast iterator	Yes	Not fail-fast

Thread Safety

- Not synchronized by default.
- Use `Collections.synchronizedMap()`:

```
Map<String, Integer> syncMap = Collections.synchronizedMap(new HashMap<>());
```

- Or use `ConcurrentHashMap` for better thread-safe performance.

Full Java Program Using All Major `HashMap` Methods

```
import java.util.*;

public class HashMapMethodsDemo {
    public static void main(String[] args) {
        HashMap<String, Integer> map = new HashMap<>();

        // 1. put()
        map.put("A", 100);
        map.put("B", 200);
        map.put("C", 300);
        map.put(null, 999);

        // 2. get()
        System.out.println("Get B: " + map.get("B"));

        // 3. getOrDefault()
        System.out.println("Get D (default): " + map.getOrDefault("D", 0));

        // 4. containsKey(), containsValue()
        System.out.println("Contains A? " + map.containsKey("A"));
```

```
        System.out.println("Contains value 300? " + map.containsValue(300));

        // 5. remove()
        map.remove("C");

        // 6. putIfAbsent()
        map.putIfAbsent("B", 500); // won't replace
        map.putIfAbsent("D", 400); // will insert

        // 7. replace()
        map.replace("A", 111);

        // 8. size(), isEmpty()
        System.out.println("Size: " + map.size());
        System.out.println("Is empty? " + map.isEmpty());

        // 9. keySet(), values(), entrySet()
        System.out.println("Keys: " + map.keySet());
        System.out.println("Values: " + map.values());

        // 10. forEach
        map.forEach((k, v) -> System.out.println(k + " -> " + v));

        // 11. clear()
        map.clear();
        System.out.println("After clear: " + map);
    }
}
```

Summary of Methods Covered

- **CRUD**: `put()`, `get()`, `remove()`, `clear()`
- **Search**: `containsKey()`, `containsValue()`, `getOrDefault()`, `putIfAbsent()`, `replace()`
- **Views**: `keySet()`, `values()`, `entrySet()`
- **Utilities**: `size()`, `isEmpty()`, `forEach()`

CHAPTER 61

LinkedHashMap in Java

What is `LinkedHashMap` in Java?

`LinkedHashMap` is a subclass of `HashMap` that maintains a **doubly-linked list** to preserve the **insertion order** of keys.

It is a part of the **Java Collection Framework** and implements the `Map` interface.

Declaration and Syntax

```
import java.util.LinkedHashMap;

LinkedHashMap<KeyType, ValueType> map = new LinkedHashMap<>();
```

Example:

```
LinkedHashMap<Integer, String> map = new LinkedHashMap<>();
```

Key Features

Feature	Description
Key-value pairs	Stores entries as (key, value)
Maintains order	Preserves **insertion order**
Unique keys	Only one entry per key
Allows nulls	One null key and multiple null values allowed
Not thread-safe	Use `Collections.synchronizedMap()` for synchronization
Faster than HashMap in iteration	Because of predictable order
Extends	`HashMap`, adds ordering logic with Linked List

Internal Working

- Internally uses:

  - **Hash Table** for storing entries
  - **Doubly Linked List** for maintaining **insertion order**
- Inherits hashing mechanism from `HashMap`, and adds **linking logic** to maintain order.

Commonly Used Methods

Method	Description
`put(K key, V value)`	Adds or updates entry
`get(Object key)`	Gets value for the given key
`remove(Object key)`	Removes mapping for the key
`containsKey(Object key)`	Checks if key exists
`containsValue(Object value)`	Checks if value exists
`isEmpty()`	Checks if map is empty
`size()`	Number of key-value pairs
`clear()`	Clears the map
`keySet()`	Returns set of keys
`values()`	Returns collection of values
`entrySet()`	Returns set of key-value pairs
`putIfAbsent()`	Adds only if key is not already present
`getOrDefault()`	Gets value or default
`replace()`	Replaces value for given key

Example Program

```
import java.util.LinkedHashMap;

public class LinkedHashMapExample {
    public static void main(String[] args) {
        LinkedHashMap<Integer, String> map = new LinkedHashMap<>();

        map.put(3, "Cherry");
        map.put(1, "Apple");
        map.put(2, "Banana");

        System.out.println("Map: " + map); // Maintains insertion order

        map.put(2, "Blueberry"); // Update value
        map.remove(3);           // Remove by key
```

```
        System.out.println("Updated map: " + map);
    }
}
```

Iterating Over `LinkedHashMap`

1. Using `entrySet()`:

```
for (Map.Entry<Integer, String> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
```

2. Using `keySet()`:

```
for (Integer key : map.keySet()) {
    System.out.println("Key: " + key + ", Value: " + map.get(key));
}
```

3. Using `forEach()` (Java 8+):

```
map.forEach((k, v) -> System.out.println(k + " = " + v));
```

Real-World Use Case

> ***Example: LRU Cache Implementation***

```
LinkedHashMap<Integer, String> cache = new LinkedHashMap<>(16, 0.75f, true);
cache.put(1, "Page1");
cache.put(2, "Page2");
cache.get(1); // Recently accessed
```

Difference: `HashMap` vs `LinkedHashMap`

Feature	`HashMap`	`LinkedHashMap`

Order	No order guaranteed	Maintains **insertion order**
Performance	Slightly faster	Slightly slower (due to linked list)
Null Support	1 null key, many null values	Same as HashMap
Use Case	Fast access, no order needed	Ordered iteration needed
Backed By	Hash table	Hash table + Doubly Linked List

Thread Safety

- Not synchronized.
- For multithreaded environments, use:

```
Map<String, String> syncMap = Collections.synchronizedMap(new LinkedHashMap<>());
```

Full Java Program Using All Major `LinkedHashMap` Methods

```
import java.util.*;

public class LinkedHashMapMethodsDemo {
    public static void main(String[] args) {
        LinkedHashMap<String, Integer> map = new LinkedHashMap<>();

        // 1. put()
        map.put("One", 1);
        map.put("Two", 2);
        map.put("Three", 3);

        // 2. get()
        System.out.println("Get 'Two': " + map.get("Two"));

        // 3. getOrDefault()
        System.out.println("Get 'Four' or default: " + map.getOrDefault("Four", 0));

        // 4. containsKey() / containsValue()
        System.out.println("Contains key 'One'? " + map.containsKey("One"));
        System.out.println("Contains value 3? " + map.containsValue(3));

        // 5. remove()
        map.remove("Three");

        // 6. putIfAbsent()
        map.putIfAbsent("Four", 4);
        map.putIfAbsent("Two", 22); // Won't replace
```

```
        // 7. replace()
        map.replace("One", 10);

        // 8. size / isEmpty
        System.out.println("Size: " + map.size());
        System.out.println("Is empty? " + map.isEmpty());

        // 9. keySet, values, entrySet
        System.out.println("Keys: " + map.keySet());
        System.out.println("Values: " + map.values());

        // 10. forEach
        map.forEach((k, v) -> System.out.println(k + " => " + v));

        // 11. clear()
        map.clear();
        System.out.println("Map after clear(): " + map);
    }
}
```

Summary of Methods Covered

- **CRUD**: `put()`, `get()`, `remove()`, `clear()`, `replace()`
- **Search**: `containsKey()`, `containsValue()`, `getOrDefault()`, `putIfAbsent()`
- **Views**: `keySet()`, `values()`, `entrySet()`
- **Utilities**: `size()`, `isEmpty()`, `forEach()`

CHAPTER 62

TreeMap in Java

What is `TreeMap` in Java?

`TreeMap` is a part of the **Java Collection Framework** that implements the `NavigableMap` and `SortedMap` interfaces. It stores key-value pairs **in sorted (ascending) order of keys** using a **Red-Black Tree** (self-balancing BST).

It belongs to the `java.util` package.

Declaration and Syntax

```
import java.util.TreeMap;

TreeMap<KeyType, ValueType> map = new TreeMap<>();
```

Example:

```
TreeMap<Integer, String> map = new TreeMap<>();
```

Key Features

Feature	Description
Key-Value Pairs	Stores entries in (key, value) format
Sorted Order	Maintains **ascending** (natural) **order of keys**
Unique Keys	Only one value per key
No Null Keys	Does **not allow null keys**
Null Values	Allows **multiple null values**
Thread-Safety	Not synchronized (use `Collections.synchronizedMap()` or `ConcurrentSkipListMap`)
Based On	Self-balancing **Red-Black Tree**

Internal Working

- Uses a **Red-Black Tree** (balanced binary search tree).
- Keys are automatically sorted using:
  - **Natural ordering** (if keys implement `Comparable`)
  - Or **custom `Comparator`** if passed in constructor.

Commonly Used Methods

Method	Description
`put(K key, V value)`	Adds or updates a key-value pair
`get(Object key)`	Returns value for given key
`remove(Object key)`	Removes mapping for the key
`firstKey()`	Returns the lowest key
`lastKey()`	Returns the highest key
`floorKey(K key)`	Greatest key ≤ given key
`ceilingKey(K key)`	Smallest key ≥ given key
`higherKey(K key)`	Next higher key
`lowerKey(K key)`	Next lower key
`pollFirstEntry()`	Removes and returns first entry
`pollLastEntry()`	Removes and returns last entry
`descendingMap()`	Returns reversed view of map
`headMap(K toKey)`	Keys strictly less than `toKey`
`tailMap(K fromKey)`	Keys ≥ `fromKey`
`subMap(K fromKey, K toKey)`	Keys between `fromKey` (inclusive) and `toKey` (exclusive)

Example Program

```
import java.util.TreeMap;

public class TreeMapExample {
    public static void main(String[] args) {
        TreeMap<Integer, String> map = new TreeMap<>();

        map.put(3, "C");
        map.put(1, "A");
        map.put(2, "B");
        map.put(4, "D");

        System.out.println("TreeMap: " + map); // Keys sorted automatically
```

```
        System.out.println("First Key: " + map.firstKey());
        System.out.println("Last Key: " + map.lastKey());

        System.out.println("Floor Key of 3: " + map.floorKey(3));
        System.out.println("Ceiling Key of 3: " + map.ceilingKey(3));
    }
}
```

Iterating Over `TreeMap`

1. Using `entrySet()`:

```
for (Map.Entry<Integer, String> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " => " + entry.getValue());
}
```

2. Using `forEach()`:

```
map.forEach((k, v) -> System.out.println(k + ": " + v));
```

3. Descending Order View:

```
NavigableMap<Integer, String> desc = map.descendingMap();
System.out.println("Descending Map: " + desc);
```

Real-World Use Case

> ***Example: Sorted leaderboard of scores***

```
TreeMap<Integer, String> scores = new TreeMap<>();
scores.put(90, "Alice");
scores.put(95, "Bob");
scores.put(85, "Charlie");

System.out.println("Top scorer: " + scores.lastEntry());
```

Difference: `HashMap` vs `TreeMap` vs `LinkedHashMap`

Feature	`HashMap`	`LinkedHashMap`	`TreeMap`
Order	No order	Insertion order	Sorted order (ascending)
Null Keys	1 allowed	1 allowed	Not allowed
Null Values	Allowed	Allowed	Allowed
Performance	Fastest (O(1))	Slightly slower	Slower (O(log n))
Backed By	Hash Table	Hash Table + Linked List	Red-Black Tree
Use Case	Lookup by key	Order-sensitive access	Sorted key access

Thread Safety

- Not thread-safe by default.
- To make it synchronized:

```
Map<Integer, String> syncTree = Collections.synchronizedSortedMap(new TreeMap<>());
```

- For concurrent access, use `ConcurrentSkipListMap`.

Full Java Program Using All Major `TreeMap` Methods

```
import java.util.*;

public class TreeMapMethodsDemo {
    public static void main(String[] args) {
        TreeMap<String, Integer> map = new TreeMap<>();

        // 1. put()
        map.put("Banana", 2);
        map.put("Apple", 1);
        map.put("Cherry", 3);

        // 2. get()
        System.out.println("Get Apple: " + map.get("Apple"));

        // 3. containsKey(), containsValue()
        System.out.println("Contains 'Banana'? " + map.containsKey("Banana"));
        System.out.println("Contains value 3? " + map.containsValue(3));

        // 4. remove()
```

```
        map.remove("Cherry");

        // 5. firstKey(), lastKey()
        System.out.println("First key: " + map.firstKey());
        System.out.println("Last key: " + map.lastKey());

        // 6. floorKey(), ceilingKey()
        System.out.println("Floor of 'Blueberry': " + map.floorKey("Blueberry"));
        System.out.println("Ceiling of 'Blueberry': " + map.ceilingKey("Blueberry"));

        // 7. headMap(), tailMap(), subMap()
        System.out.println("Head map (< 'Cherry'): " + map.headMap("Cherry"));
        System.out.println("Tail map (≥ 'Banana'): " + map.tailMap("Banana"));
        System.out.println("Sub map ('Apple' to 'Cherry'): " + map.subMap("Apple", "Cherry"));

        // 8. descendingMap()
        System.out.println("Descending map: " + map.descendingMap());

        // 9. keySet(), values(), entrySet()
        System.out.println("Keys: " + map.keySet());
        System.out.println("Values: " + map.values());

        // 10. clear()
        map.clear();
        System.out.println("After clear(): " + map);
    }
}
```

Summary of Methods Covered

- **CRUD**: `put()`, `get()`, `remove()`, `clear()`
- **Check**: `containsKey()`, `containsValue()`
- **Navigation**: `firstKey()`, `lastKey()`, `floorKey()`, `ceilingKey()`, `higherKey()`, `lowerKey()`
- **Views**: `keySet()`, `values()`, `entrySet()`, `descendingMap()`, `subMap()`, `headMap()`, `tailMap()`

CHAPTER 63

Comparable vs Comparator in Java

`Comparable` vs `Comparator` in Java

Both `Comparable` and `Comparator` are used for **sorting** objects in Java, but they are used in **different ways** and for **different purposes**.

1. `Comparable` Interface (java.lang)

Purpose:

Used to define the **natural/default sorting order** of objects.

Syntax:

```
public interface Comparable<T> {
    public int compareTo(T o);
}
```

Key Method:

- `compareTo(T o)`

Returns:

  - Negative → this < o
  - Zero → this == o
  - Positive → this > o

Example:

```
class Student implements Comparable<Student> {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int compareTo(Student s) {
        return this.id - s.id; // sorting by id (ascending)
    }
```

```
}
```

Usage:

```
List<Student> list = new ArrayList<>();
list.add(new Student(2, "Ravi"));
list.add(new Student(1, "Amit"));
Collections.sort(list); // uses compareTo
```

2. `Comparator` Interface (java.util)

Purpose:

Used to define **custom sorting** (outside the class), or **multiple ways of sorting**.

Syntax:

```
public interface Comparator<T> {
    public int compare(T o1, T o2);
}
```

Key Method:

- `compare(T o1, T o2)`

Returns:

  - Negative → o1 < o2
  - Zero → o1 == o2
  - Positive → o1 > o2

Example:

```
class SortByName implements Comparator<Student> {
    public int compare(Student a, Student b) {
        return a.name.compareTo(b.name); // sorting by name
    }
}
```

Usage:

```
Collections.sort(list, new SortByName()); // uses compare
```

Comparable vs Comparator: Key Differences

Feature	Comparable	Comparator
Package	`java.lang`	`java.util`
Method	`compareTo(Object o)`	`compare(Object o1, Object o2)`
Sorting	Natural (default) order	Custom or multiple sorting orders
Implementation	Class must implement it	Separate class or anonymous class
Changes required	Modify the original class	No need to modify the class
Used by	`Collections.sort(list)`	`Collections.sort(list, comparator)`

Real-World Use Case

Scenario	Use `Comparable` or `Comparator`?
Default sorting by roll number	`Comparable`
Sorting by name or marks (custom)	`Comparator`
Sorting objects from 3rd-party class	`Comparator`

Summary Table

Interface	compareTo() / compare()	Sorting Style	Customization	Used in
Comparable	`compareTo()`	Natural order	No	`Collections.sort(list)`
Comparator	`compare()`	Multiple/custom	Yes	`Collections.sort(list, comparator)`

CHAPTER 64

Collectors

`Collectors.toList()`

Purpose:

Collects stream elements into a **`List`**.

Syntax:

```
List<T> list = stream.collect(Collectors.toList());
```

Example:

```
List<String> names = List.of("John", "Jane", "Jack");
List<String> result = names.stream()
                           .collect(Collectors.toList());
System.out.println(result);
```

Output:

```
[John, Jane, Jack]
```

`Collectors.toSet()`

Purpose:

Collects stream elements into a **`Set`** (removes duplicates).

Syntax:

```
Set<T> set = stream.collect(Collectors.toSet());
```

Example:

```
List<String> names = List.of("John", "Jane", "John");
Set<String> result = names.stream()
```

```
                            .collect(Collectors.toSet());
System.out.println(result);
```

Output:

```
[Jane, John] // order not guaranteed
```

`Collectors.toMap()`

Purpose:

Collects stream elements into a **`Map<K, V>`**.

Syntax:

```
Map<K, V> map = stream.collect(Collectors.toMap(
    keyMapper, valueMapper
));
```

Example:

```
List<String> names = List.of("John", "Jane");
Map<String, Integer> nameLengthMap = names.stream()
    .collect(Collectors.toMap(
        name -> name,              // Key: name
        name -> name.length()      // Value: length
    ));
System.out.println(nameLengthMap);
```

Output:

```
{Jane=4, John=4}
```

If duplicates exist, use merge function:

```
Collectors.toMap(keyMapper, valueMapper, (v1, v2) -> v1)
```

`Collectors.joining(CharSequence delimiter)`

Purpose:

Concatenates string elements in the stream with a **delimiter**.

Syntax:

```
String result = stream.collect(Collectors.joining(", "));
```

Example:

```
List<String> names = List.of("John", "Jane", "Jack");
String result = names.stream()
                     .collect(Collectors.joining(", "));
System.out.println(result);
```

Output:

```
John, Jane, Jack
```

`Collectors.counting()`

Purpose:

Counts the number of elements in the stream.

Syntax:

```
long count = stream.collect(Collectors.counting());
```

Example:

```
List<String> names = List.of("John", "Jane", "Jack");
long count = names.stream()
                  .collect(Collectors.counting());
System.out.println(count);
```

Output:

```
3
```

`Collectors.groupingBy()`

Purpose:

Groups elements by a **classifier function** into a `Map<K, List<T>>`.

Syntax:

```
Map<K, List<T>> grouped = stream.collect(
    Collectors.groupingBy(classifierFunction)
);
```

Example:

Group names by length:

```
List<String> names = List.of("John", "Jane", "Jack", "Bob");
Map<Integer, List<String>> grouped = names.stream()
    .collect(Collectors.groupingBy(String::length));
System.out.println(grouped);
```

Output:

```
{3=[Bob], 4=[John, Jane, Jack]}
```

`Collectors.partitioningBy()`

Purpose:

Partitions elements into two groups based on a **predicate** — returns `Map<Boolean, List<T>>`.

Syntax:

```
Map<Boolean, List<T>> partitioned = stream.collect(
    Collectors.partitioningBy(predicate)
);
```

Example:

Partition numbers into even and odd:

```
List<Integer> nums = List.of(1, 2, 3, 4, 5);
Map<Boolean, List<Integer>> partitioned = nums.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
System.out.println(partitioned);
```

Output:

```
{false=[1, 3, 5], true=[2, 4]}
```

Summary Table

Collector	Description	Output Type
`toList()`	Collects to a List	`List<T>`
`toSet()`	Collects to a Set (no duplicates)	`Set<T>`
`toMap()`	Collects to a Map	`Map<K, V>`
`joining(delim)`	Joins strings with delimiter	`String`
`counting()`	Counts elements	`Long`
`groupingBy(classifier)`	Groups by classifier	`Map<K, List<T>>`
`partitioningBy(pred)`	Partitions by predicate	`Map<Boolean, List<T>>`

CHAPTER 65

Streams in Java

Streams in Java

What is Stream in Java?

A **Stream** in Java is a **sequence of elements** that supports **aggregate operations**. It allows functional-style processing of collections (like List, Set, Map) and arrays using **declarative** (what to do) rather than **imperative** (how to do) programming.

Key Characteristics:

- No storage: Doesn't store elements, it processes them.
- Functional: Uses lambda expressions and functional-style operations.
- Lazy: Operations are lazy and optimized.
- Parallelizable: Can run operations in parallel.

Stream API Introduction

Introduced in **Java 8**, in the package:

```
java.util.stream
```

Stream Pipeline

```
Data Source → Stream → Intermediate Operations → Terminal Operation → Result
```

Example Pipeline:

```
List<String> names = List.of("John", "Jane", "Jack");

names.stream()
     .filter(n -> n.startsWith("J"))
     .map(String::toUpperCase)
     .forEach(System.out::println);
```

Types of Stream

Type	Description
Sequential Stream	Processes elements one by one
Parallel Stream	Processes elements concurrently

Creating Streams

```
// From Collection
List<Integer> list = List.of(1, 2, 3);
Stream<Integer> stream1 = list.stream();

// From Arrays
Stream<String> stream2 = Arrays.stream(new String[]{"a", "b", "c"});

// From values
Stream<String> stream3 = Stream.of("x", "y", "z");

// Infinite Stream
Stream<Integer> infinite = Stream.iterate(0, n -> n + 1);
```

Stream Operations

1. **Intermediate Operations** (returns a Stream)

Operation	Description
`filter()`	Filters elements based on a predicate
`map()`	Transforms elements
`flatMap()`	Flattens nested structures
`sorted()`	Sorts elements
`distinct()`	Removes duplicates
`limit(n)`	Limits the number of elements
`skip(n)`	Skips the first n elements
`peek()`	Performs action without changing elements

Example:

```
list.stream()
    .filter(e -> e > 10)
    .map(e -> e * 2)
    .sorted()
    .distinct();
```

---

2. **Terminal Operations** (returns a result)

Operation	Description
`forEach()`	Performs action on each element
`collect()`	Converts stream to List, Set, Map, etc
`count()`	Returns number of elements
`reduce()`	Reduces to a single value
`toArray()`	Converts stream to array
`min()`, `max()`	Finds minimum or maximum value
`anyMatch()`, `allMatch()`, `noneMatch()`	Matching conditions

Example:

```
List<String> result = list.stream()
    .filter(s -> s.length() > 3)
    .collect(Collectors.toList());
```

Collectors Utility

Located in: `java.util.stream.Collectors`

Common collectors:

```
Collectors.toList()
Collectors.toSet()
Collectors.toMap()
Collectors.joining(", ")
Collectors.counting()
Collectors.groupingBy()
Collectors.partitioningBy()
```

Example:

```
List<String> names = List.of("Ram", "Sam", "Tom", "Tim");

Map<Integer, List<String>> map = names.stream()
    .collect(Collectors.groupingBy(String::length));
```

Stream vs Collection

Feature	Stream	Collection
Storage	No	Yes
Traversal	One-time (unless recreated)	Multiple
Lazy Evaluation	Yes	No
Functional-style	Yes	No
Parallel support	Yes	No (needs manual effort)

Real-World Use Cases

Scenario	Stream Usage
Filtering active users	`users.stream().filter(...).collect(...)`
Aggregating sales by region	`sales.stream().collect(groupingBy(...))`
Reading file lines	`Files.lines(Paths.get("file.txt"))`
Removing duplicates from list	`list.stream().distinct().collect(...)`
Counting words in paragraph	`Arrays.stream(str.split(" ")).count()`

Example Programs

Example 1: Find even numbers

```
List<Integer> list = List.of(1, 2, 3, 4, 5, 6);
list.stream()
    .filter(n -> n % 2 == 0)
    .forEach(System.out::println);
```

Example 2: Count frequency of words

```
String text = "java python java c java";
Map<String, Long> freq = Arrays.stream(text.split(" "))
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));
System.out.println(freq); // {c=1, python=1, java=3}
```

Parallel Stream

```
List<Integer> bigList = IntStream.range(1, 1000000)
    .boxed()
    .collect(Collectors.toList());

long count = bigList.parallelStream()
    .filter(n -> n % 2 == 0)
    .count();
```

Use **parallelStream()** only when:

- Data set is large
- Independent operations
- Multicore CPUs available

Summary Table

Concept	Example / Notes
Stream Source	List, Set, Array, Files
Intermediate Ops	`filter()`, `map()`, `sorted()`, `distinct()`
Terminal Ops	`forEach()`, `collect()`, `reduce()`
Collector Utility	`Collectors.toList()`, `groupingBy()`
Parallel Stream	`collection.parallelStream()`
Functional Interface	Lambda expressions used

15 practical and diverse Java Stream examples, covering filtering, mapping, sorting, collecting, grouping, reducing.

1. **Filter Even Numbers**

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6);

numbers.stream()
       .filter(n -> n % 2 == 0)
       .forEach(System.out::println);
// Output: 2 4 6
```

2. **Convert to Uppercase**

```
List<String> names = List.of("john", "jane", "jack");

names.stream()
     .map(String::toUpperCase)
     .forEach(System.out::println);
// Output: JOHN JANE JACK
```

3. **Sort List of Strings**

```
List<String> words = List.of("banana", "apple", "cherry");

words.stream()
     .sorted()
     .forEach(System.out::println);
// Output: apple banana cherry
```

4. **Find First Element**

```
List<Integer> list = List.of(10, 20, 30);

Optional<Integer> first = list.stream().findFirst();
first.ifPresent(System.out::println);
// Output: 10
```

5. **Count Strings Starting with "A"**

```
List<String> names = List.of("Alice", "Bob", "Ankit", "Alex");

long count = names.stream()
                  .filter(name -> name.startsWith("A"))
                  .count();
System.out.println(count);
// Output: 3
```

6. **Remove Duplicates**

```
List<Integer> nums = List.of(1, 2, 2, 3, 4, 4, 5);

nums.stream()
    .distinct()
    .forEach(System.out::println);
// Output: 1 2 3 4 5
```

7. **Sum of Numbers using reduce()**

```
List<Integer> nums = List.of(1, 2, 3, 4, 5);

int sum = nums.stream()
              .reduce(0, Integer::sum);
System.out.println(sum);
// Output: 15
```

8. **Convert List to Map**

```
List<String> names = List.of("Ram", "Sam", "Tim");

Map<String, Integer> map = names.stream()
    .collect(Collectors.toMap(
        name -> name,
        name -> name.length()
    ));

System.out.println(map);
// Output: {Ram=3, Sam=3, Tim=3}
```

9. **Group by String Length**

```
List<String> names = List.of("John", "Jane", "Max", "Tom", "Tim");

Map<Integer, List<String>> grouped = names.stream()
    .collect(Collectors.groupingBy(String::length));

System.out.println(grouped);
// Output: {3=[Max, Tom, Tim], 4=[John, Jane]}
```

10. **Join Strings**

```
List<String> names = List.of("Apple", "Banana", "Mango");

String result = names.stream()
    .collect(Collectors.joining(", "));

System.out.println(result);
// Output: Apple, Banana, Mango
```

11. **Find Max Element**

```
List<Integer> nums = List.of(5, 10, 20, 7);

int max = nums.stream()
              .max(Integer::compare)
              .orElseThrow();
System.out.println(max);
// Output: 20
```

12. **FlatMap Example (2D List → 1D List)**

```
List<List<String>> data = List.of(
    List.of("A", "B"),
    List.of("C", "D")
);

List<String> flat = data.stream()
```

```
    .flatMap(List::stream)
    .collect(Collectors.toList());

System.out.println(flat);
// Output: [A, B, C, D]
```

13. **Partition Numbers into Even and Odd**

```
List<Integer> nums = List.of(1, 2, 3, 4, 5);

Map<Boolean, List<Integer>> partitioned = nums.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));

System.out.println(partitioned);
// Output: {false=[1, 3, 5], true=[2, 4]}
```

14. **Words Frequency Count**

```
String input = "java python java c java";

Map<String, Long> frequency = Arrays.stream(input.split(" "))
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));

System.out.println(frequency);
// Output: {c=1, python=1, java=3}
```

15. **Filter and Collect to Set**

```
List<String> names = List.of("Anna", "Ankit", "John", "Ankit");

Set<String> filtered = names.stream()
    .filter(n -> n.startsWith("A"))
    .collect(Collectors.toSet());

System.out.println(filtered);
// Output: [Ankit, Anna]
```

CHAPTER 66

Records in Java

Records in Java

1. Definition

A **Record** in Java is a **special kind of class** introduced in **Java 14 (Preview), officially in Java 16** that acts as a **compact, immutable data carrier**.

> *"Record is a quick way to create classes whose main purpose is to hold data (POJOs / DTOs) with less boilerplate."*

---

2. Declaration Syntax

```
record ClassName(type1 field1, type2 field2, ...) { }
```

> *Automatically generates:*

- `private final` fields
- Constructor
- Getters (no `get` prefix)
- `equals()`, `hashCode()`, `toString()`

---

3. Simple Example

```
record Student(int id, String name) { }

public class RecordDemo {
    public static void main(String[] args) {
        Student s = new Student(101, "Vijay");
        System.out.println(s.id());     // 101
        System.out.println(s.name());   // Vijay
        System.out.println(s);          // Student[id=101, name=Vijay]
    }
}
```

---

4. Comparison with Class

Feature	Regular Class	Record
Boilerplate code	High (need constructor, getters, etc.)	Low (auto-generated)
Mutability	Mutable (fields can change)	Immutable (fields are final)
Inheritance	Can extend other classes	Cannot extend any class
Default methods	Not auto-generated	Constructor, getters, equals, hashCode, toString
Use case	Complex logic or behavior	Data Transfer Objects (DTOs), Models

5. Internal Working (Compiler-Generated Code)

```
record Student(int id, String name) { }
```

Roughly becomes:

```
final class Student {
    private final int id;
    private final String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int id() { return id; }
    public String name() { return name; }

    public boolean equals(Object o) { ... }
    public int hashCode() { ... }
    public String toString() { return "Student[id=" + id + ", name=" + name + "]"; }
}
```

6. Custom Methods in Records

```
record Product(int id, String name, double price) {
    public String display() {
        return id + " - " + name + " - ₹" + price;
    }
}
```

7. Compact Constructor (Validation Logic)

```
record Person(String name, int age) {
    Person {
        if (age < 0) throw new IllegalArgumentException("Invalid age");
    }
}
```

8. Immutability of Records

```
Student s = new Student(101, "Ravi");
// s.id = 102;  Error – final fields can't be reassigned
```

9. Limitations of Records

- Cannot extend other classes
- Fields are always `private final`
- Cannot declare instance fields
- Not suitable for mutable entities or business logic

10. Real-World Use Cases

Use Case	Example
DTOs (Data Transfer Objects)	Sending user/product/order info in APIs
JSON Serialization	Mapping JSON data to Java
Records in Java Streams	Grouping/filtering records in functional style
Domain Models in Spring Boot	Entities for clean code

11. Records with Collections

```
record Employee(int id, String name) {}

public class Main {
    public static void main(String[] args) {
        List<Employee> list = List.of(
            new Employee(1, "Arun"),
            new Employee(2, "Bala")
        );
```

```
        list.stream()
            .filter(e -> e.id() > 1)
            .forEach(System.out::println);
    }
}
```

12. Summary Table

Aspect	Record in Java
Introduced In	Java 14 (Preview), Java 16 (Standard)
Use	Compact, immutable data holders
Fields	`private final`
Inheritance	Cannot extend other classes
Mutability	Immutable
Auto-Generated	Constructor, `equals()`, `hashCode()`, `toString()`, Getters
Suitable For	DTOs, Value Classes, JSON models

CHAPTER 67

Optional Class in Java

Optional Class in Java – Full Guide

1. What is `Optional`?

`Optional` is a container object introduced in **Java 8** to **represent the presence or absence of a value**.

- It **avoids `NullPointerException`** by explicitly representing "no value".
- Part of the **`java.util`** package.

> *Think of it as a **wrapper** for a value that might be `null`.*

---

2. Why use `Optional`?

Problem	Solution
Traditional `null` leads to `NullPointerException`.	`Optional` provides methods to safely deal with missing values.
Hard to understand intent of `null` return types.	`Optional` makes absence of value **explicit**.

---

3. Optional Declaration

```
Optional<String> name = Optional.of("John");
Optional<String> emptyName = Optional.empty();
```

---

4. Optional Class Diagram

```
java.lang.Object
        ↑
java.util.Optional<T>
```

---

5. Important Static Methods

Method	Description
`of(T value)`	Returns an `Optional` with the specified non-null value.
`ofNullable(T value)`	Returns `Optional` if value is non-null, else empty.
`empty()`	Returns an empty `Optional`.

6. Important Instance Methods

Method	Description
`isPresent()`	Returns `true` if value is present.
`isEmpty()`	Returns `true` if value is absent.
`get()`	Returns value if present, else throws `NoSuchElementException`.
`orElse(T other)`	Returns value if present, else returns `other`.
`orElseGet(Supplier)`	Returns value if present, else computes it using Supplier.
`orElseThrow()`	Throws `NoSuchElementException` if value is not present.
`ifPresent(Consumer)`	Runs consumer if value is present.
`filter(Predicate)`	Returns Optional if value matches predicate.
`map(Function)`	Transforms value inside Optional.
`flatMap(Function)`	Similar to `map` but avoids nested Optionals.

7. Optional Example Code

Basic Usage

```
Optional<String> name = Optional.of("Alice");
System.out.println(name.get()); // Alice
```

Null Safety

```
Optional<String> name = Optional.ofNullable(null);
System.out.println(name.orElse("Default")); // Default
```

`ifPresent()`

```
name.ifPresent(n -> System.out.println("Hello " + n));
```

`map()` and `flatMap()`

```
Optional<String> name = Optional.of("john");
```

```
Optional<String> upper = name.map(String::toUpperCase);
System.out.println(upper.get()); // JOHN

Optional<Optional<String>> nested = name.map(n -> Optional.of(n.toUpperCase()));
System.out.println(nested); // Optional[Optional[JOHN]]

Optional<String> flat = name.flatMap(n -> Optional.of(n.toUpperCase()));
System.out.println(flat); // Optional[JOHN]
```

8. Real-World Use Case

Before Java 8

```
public String getUserEmail(User user) {
    if (user != null && user.getEmail() != null) {
        return user.getEmail();
    } else {
        return "default@example.com";
    }
}
```

With Optional

```
public Optional<String> getUserEmail(User user) {
    return Optional.ofNullable(user)
                   .map(User::getEmail)
                   .or(() -> Optional.of("default@example.com"));
}
```

9. Comparison: `null` vs `Optional`

Feature	`null`	`Optional`
Expresses absence	Implicit	Explicit
Null checks	Manual	Built-in methods
Risk of `NullPointerException`	High	Low
Chaining operations	Error-prone	Safe with `map`, `flatMap`
Readability	Poor	Better intent

10. Do's and Don'ts

Use `Optional`:

- As return types for methods that may return no result
- To improve code readability and avoid `null` checks

Don't use `Optional`:

- For fields (due to serialization issues and unnecessary overhead)
- In method parameters (use method overloading or null-checks instead)

11. Summary Table

Method	Return Type	Description
`Optional.of(value)`	`Optional<T>`	Wraps non-null value
`Optional.empty()`	`Optional<T>`	Empty optional
`Optional.ofNullable(value)`	`Optional<T>`	Wraps value or returns empty
`get()`	`T`	Gets value or throws exception
`isPresent()`	`boolean`	Checks if value exists
`orElse(default)`	`T`	Returns value or default
`orElseGet(Supplier)`	`T`	Returns value or supplier result
`orElseThrow()`	`T`	Throws if empty
`map(Function)`	`Optional<R>`	Transforms value
`flatMap(Function)`	`Optional<R>`	Transforms and flattens
`filter(Predicate)`	`Optional<T>`	Returns filtered optional
`ifPresent(Consumer)`	`void`	Runs if present

12. Sample Interview Question

Q: How is `Optional` different from `null`?
A: `Optional` makes the absence of a value explicit, reduces null-check boilerplate, and provides safe operations like `map`, `flatMap`, and `orElse`. It prevents `NullPointerException` and improves readability.

13. Quick Code Snippet

```
Optional<String> city = Optional.of("Hyderabad");

city.map(String::toUpperCase)
```

```
    .filter(c -> c.startsWith("H"))
    .ifPresent(System.out::println); // HYDERABAD
```

CHAPTER 68

File Handling in Java

File Handling in Java

File handling is a mechanism in Java to **create**, **read**, **write**, **update**, and **delete** files. Java supports file operations through the `java.io`, `java.util`, and `java.nio.file` packages.

Why File Handling?

Java programs use file handling to:

- Store data **permanently**
- Read/write **text or data files**
- Exchange data between applications
- Save user input/output (logs, reports)
- Work with configurations (e.g., `.properties`)

Java File Handling Packages

Package	Description
`java.io`	Legacy, stream-based API for reading/writing files
`java.nio.file`	Modern API for file system interaction using `Path`, `Files`
`java.util`	Includes classes like `Scanner` for reading files conveniently

Commonly Used Classes

Class	Package	Purpose
`File`	`java.io`	Represents file/directory
`FileReader`	`java.io`	Reads characters from file
`BufferedReader`	`java.io`	Efficient line-by-line reading
`FileWriter`	`java.io`	Writes characters to file
`BufferedWriter`	`java.io`	Efficient buffered writing

`PrintWriter`	`java.io`	Easy way to write lines
`Scanner`	`java.util`	Reads file line-by-line
`Files`, `Paths`	`java.nio.file`	Modern file I/O

The `File` Class

Used to handle file and directory paths.

Constructors

```
File file1 = new File("file.txt");                  // Relative path
File file2 = new File("C:\\Users\\file.txt");       // Absolute path
File file3 = new File("folder", "file.txt");        // Folder + File
```

Common Methods

Method	Description
`createNewFile()`	Creates new file
`mkdir()`	Creates a directory
`mkdirs()`	Creates nested directories
`exists()`	Checks file/directory existence
`delete()`	Deletes file/directory
`length()`	Returns size in bytes
`getName()`	Returns file name
`getAbsolutePath()`	Returns absolute file path
`list()` / `listFiles()`	List directory contents
`canRead()` / `canWrite()`	Check file permissions

File Creation Example

```
import java.io.*;

public class FileCreation {
    public static void main(String[] args) {
        try {
            File file = new File("demo.txt");
            if (file.createNewFile()) {
                System.out.println("Created: " + file.getName());
```

```
            } else {
                System.out.println("Already exists.");
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Reading Files

Using `BufferedReader`

```
try (BufferedReader reader = new BufferedReader(new FileReader("demo.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

Using `Scanner`

```
try (Scanner sc = new Scanner(new File("demo.txt"))) {
    while (sc.hasNextLine()) {
        System.out.println(sc.nextLine());
    }
} catch (Exception e) {
    e.printStackTrace();
}
```

Using `Files.readString()` (Java 11+)

```
import java.nio.file.*;

Path path = Paths.get("demo.txt");
String content = Files.readString(path);
System.out.println(content);
```

Writing to Files

Using `BufferedWriter`

```
try (BufferedWriter writer = new BufferedWriter(new FileWriter("demo.txt"))) {
    writer.write("Hello!");
    writer.newLine();
    writer.write("Second line.");
} catch (IOException e) {
    e.printStackTrace();
}
```

Using `PrintWriter`

```
try (PrintWriter pw = new PrintWriter("output.txt")) {
    pw.println("Line 1");
    pw.println("Line 2");
} catch (Exception e) {
    e.printStackTrace();
}
```

Append Mode

```
FileWriter fw = new FileWriter("demo.txt", true); // true enables append
```

Try-With-Resources (Java 7+)

Automatically closes streams:

```
try (BufferedWriter writer = new BufferedWriter(new FileWriter("demo.txt"))) {
    writer.write("Try-with-resources example");
}
```

Deleting a File

```
File file = new File("demo.txt");
if (file.delete()) {
    System.out.println("Deleted: " + file.getName());
```

```
} else {
    System.out.println("Delete failed.");
}
```

Directory Handling

```
File dir = new File("folder");
if (dir.mkdir()) {
    System.out.println("Directory created.");
}

String[] files = dir.list();
for (String name : files) {
    System.out.println(name);
}
```

File Permissions

```
File file = new File("demo.txt");
System.out.println("Readable: " + file.canRead());
System.out.println("Writable: " + file.canWrite());
System.out.println("Executable: " + file.canExecute());
```

Java NIO – Modern File API

Example: Create, Write, Read

```
import java.nio.file.*;

public class NIOExample {
    public static void main(String[] args) throws IOException {
        Path path = Paths.get("niofile.txt");

        // Write
        Files.write(path, "Hello from NIO!".getBytes());

        // Read
        String data = Files.readString(path);
```

```
        System.out.println(data);
    }
}
```

Exceptions in File Handling

Exception	Cause
`FileNotFoundException`	File doesn't exist
`IOException`	General I/O error
`SecurityException`	No permission to access file
`EOFException`	End of file reached unexpectedly

Use `try-catch` or `throws IOException` with file operations.

Real-World Use Cases

Scenario	Example
Logging	Save logs in `log.txt`
Reading configurations	Load from `.properties` file
Report generation	Write to `.csv` or `.txt` report files
User data storage	Save form inputs to files
File uploads/downloads	Store uploaded files in web applications

Summary Table

Operation	Class(es) Used	Method/Notes
Create File	`File`, `Files`	`createNewFile()`, `Files.createFile()`
Read File	`BufferedReader`, `Scanner`, `Files`	`readLine()`, `nextLine()`, `Files.readString()`
Write File	`BufferedWriter`, `PrintWriter`, `Files`	`write()`, `append`, `println()`
Delete File	`File`	`delete()`
Directory Ops	`File`, `Files`	`mkdir()`, `list()`, `listFiles()`
Permissions	`File`	`canRead()`, `canWrite()`, `canExecute()`

Best Practices

- Always **close file streams** (use try-with-resources)
- Check if file exists before reading/writing
- Use `BufferedReader/Writer` for efficiency
- Use `java.nio.file` for simplicity (Java 7+)
- Handle exceptions carefully (file may not exist or be accessible)